

Hindi Dependency Parsing and Treebank Validation

Thesis submitted in partial fulfillment
of the requirements for the degree of

Master of Science (by Research)
in
Computer Science & Engineering

by

Bharat Ram Ambati

200502004

ambati@research.iiit.ac.in



Language Technologies Research Centre
International Institute of Information Technology

Hyderabad - 500 032, INDIA

February 2011

Copyright © Bharat Ram Ambati, 2011
All Rights Reserved

International Institute of Information Technology
Hyderabad, India

CERTIFICATE

It is certified that the work contained in this thesis, titled “Hindi Dependency Parsing and Treebank Validation” by Bharat Ram Ambati, has been carried out under my supervision and is not submitted elsewhere for a degree.

Date

Advisers: Prof. Rajeev Sangal
and
Dr. Dipti Misra Sharma

To my family and friends

Acknowledgments

I would like to thank my supervisors Prof. Rajeev Sangal and Dr. Dipti Misra Sharma without whom, this thesis would not have been possible. Dr. Dipti's deep linguistics insights and Prof. Sangal's valuable suggestions on using linguistics in machine learning led to this thesis. I admire Prof. Sangal as a good adviser and a great human. Starting from choosing LTRC as research lab till now and hopefully in future also, he is the motivating factor for my research. He is the one who taught us how to give a wonderful presentation.

Special thanks to Prof. Joakim Nivre for his suggestions during later stages of my thesis work. He showed new directions to my research, especially in terms of writing a good research paper. I would like to thank Ryan McDonald for clarifying doubts related to MST parser. I would also like to thank the reviewers of my papers, who improved my writing skills. Thanks to Markus Dickinson, Bolyd, Attardi, Daniel Zeman, Owen Rambow, Martha Palmer, Fei Xia and Rajesh Butt for their comments on my presentations during LREC-2010 and ACL-2010 conferences. Thanks to Soma mam, Lakshmi Bai mam, Vineet Chaitanyaji for productive discussions during LTRC meetings.

Among all my project partners in IIIT, I liked working with Sambhav Jain. We both started the work on Data-driven dependency parsing for Hindi. We used to share work equally based on our interests and strengths. When I started the work on Hindi dependency parsing during winter school, I hardly know anything about it. Along with Prof. Sangal, Dr. Srinivas Bangalore was there giving his valuable feedback every day. Special thanks to Samar Husain who has been guiding me from the start. Without his guidance, I wouldn't have completed my thesis. I would like to thank Anil sir, because of whom i got interested in java and tools for NLP.

I loved working in the lab. This is because of the wonderful work environment in the lab. Technical discussions, funny discussions, gossips, and so on. I would like to thank my seniors Sriram, Rafiya, Suman, Prashanth Mannem, Krathik Gali, Jagadeesh Gorla, Avinesh, Ravi Kiran, Vishwanath Naidu, Himani and Itisree for helping me whenever I am in trouble. Thanks to Phani Gadde, Meher, Pujitha, GSK, Aswarth and other juniors who worked with me on projects related to parsing. Thanks to my friends SRP, Gani, Sivareddy, Abhilash, Mridul, Mathur and Vipul who made my journey in LTRC most memorable.

Annotated treebank is the most important resource for my work. I would like to thank all the annotators of both the treebanks for the data. To name a few of them are Rafiya, Preeti, Nandini, Preeti Shukla.

I thank the LTRC staff, who made my research journey at LTRC most comfortable. Srinivas sir for wonderful lab, Rambabu sir for administrative issues, Mr. Satish and Mr. Kumara Swamy for lab related issues and Mr. Lakshmi Narayan and Aswini Nanda for general issues. Thanks to Appaji sir, Kishore sir, BLN sir, and Bala sir, for making my life easier in IIIT.

My friends played a very key role in my life. Thanks to Vijay Bharath, Kranthi, Srirang and Gopal for their support during initial days of my hostel life in IIIT. I would like to acknowledge my friends Charan, Gani, Pudi, SRP, Harsha, Girish, SCP, Praveen, Samish, Janga, Siddu, Siva, Abhilash, Raghudeep, and Divya who are there with me throughout my journey in IIIT.

I would like to acknowledge two special persons in my life, who were there to share both my happiness and sadness. Thanks to Ragasudha and Anupama Gali for your moral support.

Finally, I would like to thank my father, mother, grand mother and sister for their support and strong belief in me.

Abstract

Hindi is a morphologically rich, free word order language. Parsing morphologically rich, free word order languages (MoR-FWO) is a challenging task. In this work we present our experiments which lead to state-of-the-art dependency parser for Hindi. We do a series of experiments exploring the role of different morphological and syntactic features in Hindi dependency parsing using two data-driven parsers, Malt and MST. With just 1500 sentences training data, we are able to build a dependency parser with state-of-the-art accuracy of 74.5% Labeled Attachment Score (LAS) and 90.1% Unlabeled Attachment Score (UAS). We also do a detailed error analysis and suggest possible remedies for these problems. During the course of experiments, we realized that some of the basic linguistic constraints were violated by these data driven parsers. We built a linguistically sound parser without compromising on the accuracy. Consider a simple linguistic constraint that a verb should not have multiple karta karakas (roughly, subjects) as its children in the dependency tree. We propose two approaches to handle this constraint and evaluate them on the state-of-the-art Hindi dependency parser.

After building state-of-the-art inter-chunk dependency parser for Hindi, we also present our preliminary work on sentence level parsing of Hindi. We extract 1000 sentences, which are word level annotated, from a new multi-layered and multi-representational Hindi Treebank which is being developed. We do a step by step analysis of the importance of different features like, Part-Of-Speech, morph and chunk information for sentence level parsing of Hindi. We could achieve 75.4% Labeled Attachment Score (LAS) and 85.5% Unlabeled Attachment Score (UAS), which is the state-of-the-art performance for sentence level parsing.

We also propose an new error detection tool for Treebank validation. Majority of the available error detection tools don't work for small treebanks, or for the treebanks which are under development. But our tool handles the data sparsity issues and also helps in validation of the treebanks which are being developed. Based on the nature of errors, the proposed tool uses either rule-based or hybrid or both systems to detect errors. We present our results on Hindi dependency treebank data. With our preliminary experiments, we are able to detect 75%, 62.5% and 76.63% of errors in POS, chunk and dependency annotation respectively.

Contents

Chapter	Page
1 Introduction	1
1.1 Contributions	2
1.2 Outline	3
2 Dependency Parsing	5
2.1 Introduction	5
2.2 Approaches	6
2.3 MaltParser (A Transition-based Dependency Parser)	7
2.3.1 Transition Systems	7
2.3.2 Classifiers	9
2.4 MST Parser (A Graph-based Dependency Parser)	10
2.4.1 Maximum Spanning Trees	10
2.4.2 Parsing Algorithm	11
2.4.3 Learning and Feature Selection	14
2.5 Summary	15
3 Hindi Treebank	16
3.1 Hindi Language	16
3.2 Paninian Grammatical Model	17
3.3 Treebanks	19
3.3.1 HyDT-Hindi	19
3.3.2 Hindi Treebank (under development)	20
3.4 Representation	21
3.4.1 SSF Format	21
3.4.2 CoNLL Format	23
4 Hindi Dependency Parsing: Chunk-level	25
4.1 Introduction	25
4.2 Approach	25
4.3 Settings	26
4.3.1 Data Settings	26
4.3.2 Malt: General Settings	27
4.3.3 Malt: Feature Selection	27
4.3.4 MST+MaxEnt: MST Settings	29
4.3.5 MST+MaxEnt: MaxEnt Settings	30

4.4	Experiments and Results	30
4.5	Error Analysis	31
4.5.1	Simple Sentences	33
4.5.2	Embedded Clauses	33
4.5.3	Coordination	34
4.5.4	Complex Predicates	35
4.5.5	Non-Projectivity	35
4.5.6	Long-Distance Dependencies	35
4.6	Summary	36
5	Linguistic Constraints in Dependency Parsing	37
5.1	Introduction	37
5.2	Motivation	37
5.3	Approaches	40
5.3.1	Naive Approach (NA)	40
5.3.2	Probabilistic Approach (PA)	41
5.4	Experiments	42
5.4.1	Hindi	42
5.4.2	Czech	43
5.5	Discussion and Future Work	44
5.6	Conclusion	44
6	Hindi Dependency Parsing: Down to Word level	45
6.1	Introduction	45
6.2	Getting the best linguistic features	46
6.2.1	Using POS as feature (PaF):	46
6.2.2	Using Morph as feature (MaF):	47
6.2.3	Using local morphosyntax as feature (LMSaF)	48
6.2.4	An alternative approach to use best features: A 2-stage setup (2stage)	49
6.3	Experimental Setup	50
6.3.1	Data	50
6.3.2	Parsers and settings	51
6.4	Results and Analysis	51
6.4.1	Feature comparison: PaF, MaF vs. LMSaF	53
6.4.2	Approach comparison: LMSaF vs. 2stage	54
6.4.3	Parser comparison: MST vs. Malt	55
6.5	Discussion and Future Work	55
6.6	Conclusion	56
7	Error Detection for Treebank Validation	57
7.1	Related Work	58
7.2	Hindi Dependency Annotation	58
7.2.1	Part-Of-Speech (POS):	59
7.2.2	Morph:	59
7.2.3	Chunk:	59
7.2.4	Dependency Relations:	60
7.2.5	Other Features:	60

7.3	Approaches	60
7.3.1	Rule-Based System	60
7.3.2	Hybrid System	63
7.3.2.1	Frequency Based Approach	63
7.3.2.2	Probability Based Hybrid Approach	64
7.4	Experiments and Results	67
7.5	Discussion and Future Work	68
7.6	Conclusion	69
8	Conclusions and Future Work	70
	<i>Appendix A: Old Tagsets</i>	73
A.1	Old POS Tagset	73
A.2	Old Chunk Tag Set	74
	<i>Appendix B: New Tagsets</i>	75
B.1	New POS Tagset	75
B.2	New Chunk Tagset	76
	<i>Appendix C: Dependency Tag Set</i>	77
	Bibliography	80

List of Figures

Figure	Page
1.1 Dependency Structure and Phrase Structure for the English sentence “Abhay ate a mango”	1
2.1 An example dependency graph.	5
2.2 A non-projective dependency graph.	6
2.3 An example of a labeled dependency graph.	7
2.4 Dependency graph for an English sentence from the Penn Treebank.	8
2.5 Arc-eager transition sequence for the English sentence in Figure 2.4.	9
2.6 Chu-Liu-Edmonds algorithm for finding maximum spanning trees in directed graphs .	11
2.7 Features used by MSTParser. where x_i is the head and x_j the modifier	14
3.1 Levels of representation/analysis in the Paninian model	17
4.1 UAS and LAS of experiments 1-10; 5-fold cross-validation on training and development data of the ICON09 tools contest	28
4.2 Precision and Recall of some important dependency labels	32
4.3 Dependency arc precision/recall relative to dependency length, where the length of a dependency from w_i to w_j is $ i - j $ and roots are assumed to have distance 0 to their head	36
5.1 Telugu to Hindi MT system	38
5.2 Hindi to English MT system	39
5.3 Approaches: Naive Approach and Probabilistic Approach	41
6.1 Dependency parsing using only POS information from a shallow parser	47
6.2 Dependency parsing using shallow parser information	49
6.3 Dependency parsing using only POS information from a shallow parser	50
6.4 F-measure of top 6, inter-chunk and intra-chunk labels for PaF, MaF and LMSaF . . .	54
6.5 Dependency arc f-measure relative to dependency length	55
7.1 Error detection framework.	61
7.2 Error detection at POS level by rule-based approach.	61
7.3 Error detection at chunk level by rule-based approach.	62
7.4 Error detection at dependency level by rule-based approach.	62
7.5 Error detection in inter-chunk dependencies by frequency based hybrid approach. . . .	65
7.6 Algorithm employed for PBHA.	66
7.7 Cycle - Improving guidelines for better annotation	69

List of Tables

Table	Page
3.1 Columns in CoNLL format	23
4.1 Feature pool used for arc-eager algorithm of Malt.	27
4.2 MaxEnt Settings (CN: W; represents lexical item (W) of the current node (CN))	31
4.3 Results of both Malt and MST+MaxEnt on cross-validated and test data sets.	31
4.4 Confusion matrix for important labels. The diagonal under ‘Incorrect’ represents attachment errors.	32
5.1 No. of instances of multiple subjects/objects in the output of the state-of-the-art Hindi parser.	42
5.2 Comparison of NA and PA with previous best results for Hindi.	43
5.3 Comparison of NA and PA with previous best results for Czech.	44
6.1 Results of all the four approaches using gold-standard shallow parser information. . . .	51
6.2 Results of all the four experiments using automatic shallow parser information.	52
7.1 Error Detection using rule-based system at different levels.	67
7.2 Error Detection at dependency level using both the frequency based and probability based hybrid approaches.	68
7.3 Error Detection at dependency level using both the frequency based and probability based hybrid approaches.	68

Chapter 1

Introduction

The *syntactic parsing* of a sentence consists of finding the correct syntactic structure of that sentence in a given formalism. Formalisms are called *grammars*, and contain the structural constraints of the language. Dependency grammar and phrase structure grammar are two such formalisms. Figure 1 (a) and (b) shows the dependency structure and a simplified phrase structure for the sentence “Abhay ate a mango”

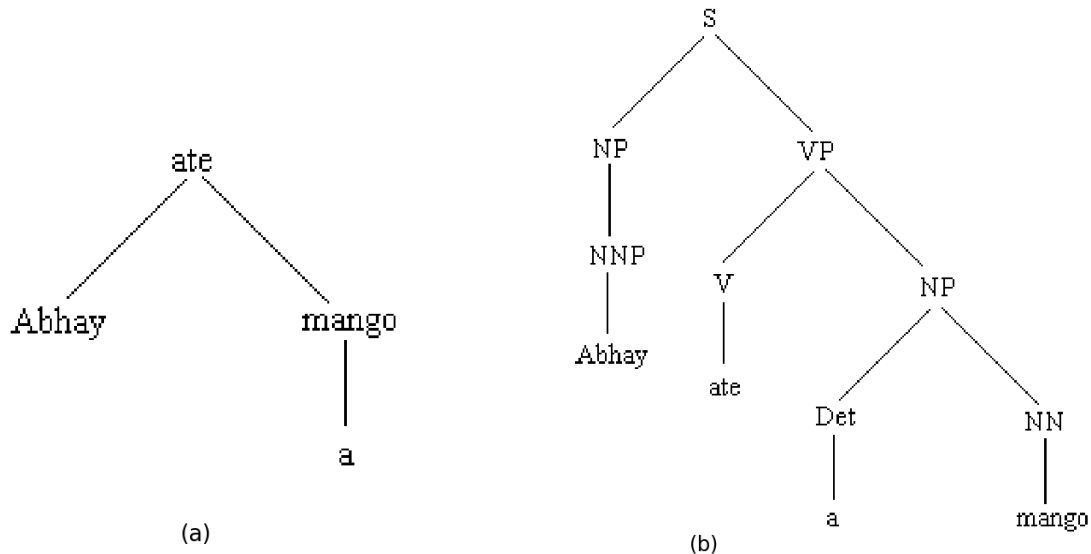


Figure 1.1 Dependency Structure and Phrase Structure for the English sentence “Abhay ate a mango”

Parsing is one of the major tasks which helps in understanding the natural language. It is useful in several natural language applications. Machine translation, anaphora resolution, word sense disambiguation, question answering, summarization are few of them. This led to the development of grammar-driven, data-driven and hybrid parsers. Due to the availability of annotated corpora in recent years, data driven parsing has achieved considerable success. The availability of phrase structure treebanks for English (Marcus et al., 1993) has seen the development of many efficient parsers. Using the dependency analysis, a similar large scale annotation effort for Czech, has been the Prague Dependency Treebank

(Hajičová, 1998). Unlike English, Czech is a free word order language and is also morphologically very rich. It has been suggested that free word order languages can be handled better using the dependency based framework than the constituency based one (Hudson, 1984; Shieber, 1985; Mel’čuk, 1988; Bharati et al., 1995). Consequently, most of the parsers for free word order languages are dependency based. The basic difference between a constituent based representation and a dependency representation is the lack of non-terminal nodes in the latter. It has also been noted that use of appropriate edge labels gives a level of semantics. It is perhaps due to these reasons that the recent past has seen a surge in the development of dependency based treebanks.

Parsing morphologically rich, free word order languages (MoR-FWO) is a challenging task. In spite of the availability of such treebanks for some MoR-FWO languages now, the state-of-the-art parsers for these languages perform lower than that of fixed word order language like English (see, Nivre et al. (2007b) and the references therein). Past experiments on parser evaluation and parser adaptation for MoR-FWO languages have shown that there are a number of factors which contribute to the performance of a parser (Nivre et al., 2007a; Hall et al., 2007; McDonald and Nivre, 2007).

Indian Languages are morphologically rich, free word order languages. As free word order languages can be handled better using the dependency based framework, dependency annotation using paninian framework is started for Indian Languages (Begum et al., 2008). There have been some previous attempts at parsing Hindi following a constraint based approach (Bharati and Sangal, 1993; Bharati et al., 2002; Bharati et al., 2008b). Recently, Gorla et al. (2008) have proposed an unlabeled dependency parser for IL using a semi-supervised graph-based approach. This thesis is an attempt towards building a state-of-the-art dependency parser for Hindi using supervised approaches.

1.1 Contributions

There are two major contributions of this thesis,

- State-of-the-art dependency parser for Hindi
- Error detection tool for Hindi Treebank Validation

Towards building the state-of-the-art dependency parser for Hindi, we used two data-driven parsers Malt and MST. We did a series of experiments exploring the role of different morphological and syntactic features in Hindi dependency parsing. With just 1500 sentences training data, we were able to build a dependency parser with state-of-the-art accuracy of 74.5% Labeled Attachment Score (LAS) and 90.1% Unlabeled Attachment Score (UAS). We did a detailed error analysis isolating specific linguistic phenomena and/or other factors that impede the overall parsing performance, and suggested possible remedies for these problems. During error analysis we realized that some of the basic linguistic constraints are violated by these parsers. As an initial step towards incorporating linguistic constraints into statistical dependency parsers, we proposed two approaches and evaluated them on the state-of-the-art

Hindi dependency parser. The goal of this work is to build a linguistically sound parser without compromising on the accuracy.

All these efforts are at finding inter-chunk dependency relations, given gold-standard POS and chunk tags as the treebank used only has the inter-chunk dependency information. A new multi-layered and multi-representational Hindi Treebank (Bhatt et al., 2009) is being developed. We have extracted 1000 sentences from this treebank which are completely annotated down to word level for our experiments. We did a step by step analysis of the importance of different features like, Part-Of-Speech, morph and chunk information for sentence level parsing of Hindi. With 1000 sentences for training, we could achieve 75.4% Labeled Attachment Score (LAS) and 85.5% Unlabeled Attachment Score (UAS). This is preliminary work in sentence level parsing of Hindi.

The second major contribution of this thesis is error detection tool for Hindi Treebank validation. Validation is very important for error-free treebank. But, it is the most time consuming process. Some tools are available for error detection in treebanks, but most of those approaches don't work for small treebanks, or the treebanks which are under development, like Hindi Treebank. We proposed a new tool which uses both rule-based and hybrid systems to detect errors. We tested it on Hindi dependency treebank data and were able to detect 75%, 62.5% and 76.63% of errors in POS, chunk and dependency annotation respectively. Based on nature of the errors, either rule-based or hybrid or both the systems.

1.2 Outline

Chapter 2: In this chapter, we first describe the dependency parsing and different approaches commonly followed for dependency parsing. Then a detailed description of two data-driven dependency parsers, Malt and MST, is given.

Chapter 3: We describe the details of the data used for experiments in this chapter. We first brief about the Hindi language and the framework used for dependency representation of Hindi. Following this, we describe different treebanks available for Hindi and different formats used for representing these treebanks.

Chapter 4: In this chapter, we describe a series of experiments done aiming towards building a state-of-the-art dependency parser for Hindi. Details of the data, parsers and different kinds of features used can be seen in this chapter. A detailed error analysis and possible remedies for these problems is also provided.

Chapter 5: This chapter focuses on importance of basic linguistic constraints in statistical dependency parsing. We consider a simple constraint that a verb should not have multiple subjects/direct objects as its children in the dependency tree. We see the importance of this constraint taking machine translation system which uses dependency parser output as an example application. We propose two approaches to handle this case and evaluate our approaches on the state-of-the-art dependency parsers for Hindi and Czech and analyze the results.

Chapter 6: In chapters 4 and 5, we explored different features and approaches towards building a state-of-the-art dependency parser for Hindi. But, all these efforts, are finding inter-chunk dependency relations, given gold-standard POS and chunk tags. But, there isn't any attempt at complete sentence level parsing for Hindi, that too using automatic tags/features rather than gold-standard ones. In this chapter, we describe our experiments on complete sentence level parsing for Hindi, which is the first known attempt in this area.

Chapter 7: In this chapter, we propose a new tool which uses both rule-based and hybrid systems to detect errors at Part-Of-Speech, chunk and dependency annotations in the Hindi dependency treebank. We categorize the errors into different groups for the convenience of the validator. We evaluate our tool on the new Hindi treebank which is under development.

Chapter 8: We conclude our thesis in this chapter providing summary of the thesis. We also put forward certain topics for future work.

Chapter 2

Dependency Parsing

2.1 Introduction

Dependency graphs represent words and their relationship to syntactic modifiers using directed edges. Figure 2.1 shows a dependency graph for the sentence, “John hit the ball with the bat”. This example belongs to the special class of dependency graphs that only contain projective (also known as nested or non-crossing) edges. Assuming a unique root as the left most word in the sentence, a projective graph is one that can be written with all words in a predefined linear order and all edges drawn on the plane above the sentence, with no edge crossing another. Figure 2.1 shows this construction for the example sentence. Equivalently, we can say a dependency graph is projective if and only if an edge from word w to word u implies that there exists a directed path in the graph from w to every word between w and u in the sentence.

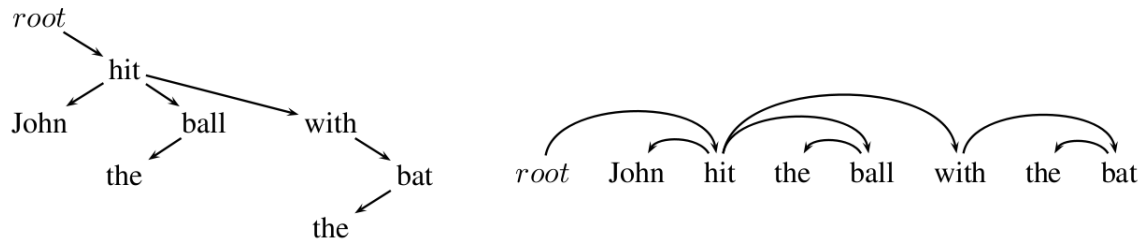


Figure 2.1 An example dependency graph.

Due to English’s rigid word order, projective graphs are sufficient to analyze most English sentences. In fact, the largest source of English dependencies is automatically generated from the Penn Treebank and is by construction exclusively projective. However, there are certain examples in which a non-projective graph is preferable. Consider the sentence, “John saw a dog yesterday which was a Yorkshire Terrier”. Here the relative clause “which was a Yorkshire Terrier” and the noun it modifies (the *dog*) are separated by a temporal modifier of the main verb. There is no way to draw the dependency graph for this sentence in the plane with no crossing edges, as illustrated in Figure 2.2. In languages with

flexible word, such as Czech, Dutch, German, and Indian languages like Hindi, Telugu, Bangla etc. non-projective dependencies are more frequent. Rich inflection systems reduce the demands on word order for expressing grammatical relations, leading to non-projective dependencies that we need to represent and parse efficiently.



Figure 2.2 A non-projective dependency graph.

Formally, a dependency structure for a given sentence is a directed graph originating out of a unique and artificially inserted root node, which we always insert as the left most word. In the most common case, every valid dependency graph has the following properties,

1. Each word has exactly one incoming edge in the graph (except the root, which has no incoming edge).
2. It is a weakly connected graph (in the directed sense).
3. There are no cycles.
4. If there are n words in the sentence (including *root*), then the graph has exactly $n - 1$ edges.

It is easy to show that 1 and 2 imply 3, and that 2 implies 4. In particular, a dependency graph that satisfies these constraints must be a tree. Thus we can say that dependency graphs satisfying these properties satisfy the *tree constraint*, and call such graphs *dependency trees*.

Directed edges in a dependency graph represent modification, e.g., a verb and its karta karaka (roughly, subject), a noun and a modifying adjective, etc. This relationship is often called the *head-modifier* or the *governor-dependent* relationship. The head is the parent and the modifier is the child or argument. We will always refer to words in a dependency relationship as the head and modifier.

The dependency structures can be principled showing only one kind of modification such as grammatical, or syntactic and even semantic properties of the head-modifier relationships. They are unfortunately mixed up sometimes, showing even grammatical phrasal categories as labels as in Figure 2.3.

2.2 Approaches

In general, dependency parsing can be broadly divided into grammar-driven and data-driven dependency parsing (Carroll, 2000). Most of the modern grammar-driven dependency parsers parse by eliminating the parses which do not satisfy some set of grammatical constraints. The problem of parsing,

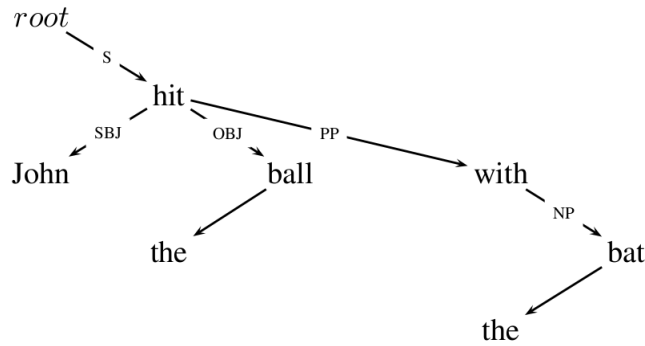


Figure 2.3 An example of a labeled dependency graph.

in this approach, is viewed as a constraint-satisfaction problem (Bharati and Sangal, 1993; Bharati et al., 2002)(Martins et al., 2009). Data-driven dependency parsers are different from the grammar driven parsers in that they use a corpus to induce a probabilistic model for disambiguation. Nevertheless many data-driven parsers also combine dependency formalism with the probabilistic model (Nivre et al. (2007b) and the references therein).

We used two state-of-the-art data-driven dependency parsers namely, MaltParser and MSTParser for our experiments. A brief description about these parsers (drawn from the original papers) can be seen in the following sections.

2.3 MaltParser (A Transition-based Dependency Parser)

MaltParser (Nivre et al., 2007b) implements the transition-based approach to dependency parsing, which has two essential components:

- A transition system for mapping sentences to dependency trees
- A classifier for predicting the next transition for every possible system configuration

Given these two components, dependency parsing can be realized as deterministic search through the transition system, guided by the classifier. With this technique, parsing can be performed in linear time for projective dependency trees and quadratic time for arbitrary (possibly non-projective) trees (Nivre, 2008).

2.3.1 Transition Systems

MaltParser comes with a number of built-in transition systems. We describe the arc-eager projective system first described in (Nivre, 2003). Other systems are minor variations of these two. For a more detailed analysis of this and other transition systems for dependency parsing, see Nivre (2008).

The arc-eager algorithm builds a labeled dependency graph in one left-to-right pass over the input. A configuration in the arc-eager projective system contains a stack holding partially processed tokens, an input buffer containing the remaining tokens, and a set of arcs representing the partially built dependency tree. There are four possible transitions (where *top* is the token on top of the stack and *next* is the next token in the input buffer):

- LEFT-ARC (*r*): Add an arc labeled *r* from *next* to *top*; pop the stack.
- RIGHT-ARC (*r*): Add an arc labeled *r* from *top* to *next*; push *next* onto the stack.
- REDUCE: Pop the stack.
- SHIFT: Push *next* onto the stack.

Consider an example English sentence “Economic news had little effect on financial markets .” taken from the Penn Treebank (Marcus et al., 1993). Figure 2.4 shows the dependency tree for the above mentioned sentence.

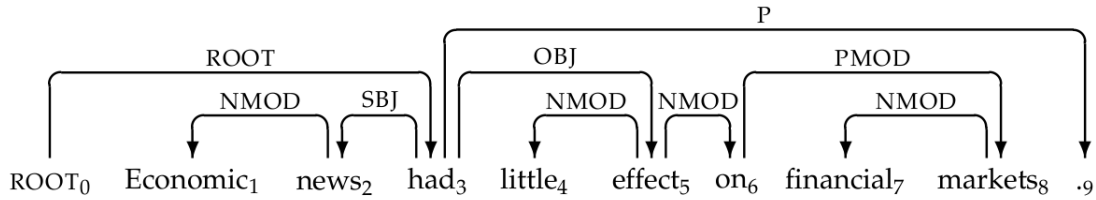


Figure 2.4 Dependency graph for an English sentence from the Penn Treebank.

Figure 2.5 shows the sequence of steps for parsing the example sentence described in Figure 2.4 using arc-eager algorithm.

Although this system can only derive projective dependency trees, the fact that the trees are labeled allows non-projective dependencies to be captured using the pseudo-projective parsing technique proposed in Nivre and Nilsson (2005). This is a way of dealing with non-projective structures in a projective data-driven parser. Training data is projectivized by a minimal transformation, lifting non-projective arcs one step at a time, and extending the arc label of lifted arcs using the encoding scheme called HEAD by Nivre and Nilsson (2005), which means that a lifted arc is assigned the label $r \uparrow h$, where r is the original label and h is the label of the original head in the non-projective dependency graph. Non-projective dependencies can be recovered by applying an inverse transformation to the output of the parser, using a left-to-right, top-down, breadth-first search, guided by the extended arc labels $r \uparrow h$ assigned by the parser.

MaltParser also provides an option for a non-projective transition system based on the method described by Covington (2001). This system uses a similar type of configuration of arc-eager described above, but adds a second temporary stack. Unlike the arc-eager, this allows the derivation of arbitrary non-projective dependency trees. There are again four possible transitions:

Transition	Configuration
	([0], [1, ..., 9], $\emptyset$)
SHIFT $\Rightarrow$	([0, 1], [2, ..., 9], $\emptyset$)
LEFT-ARC _{NMOD} $\Rightarrow$	([0], [2, ..., 9], $A_1 = \{(2, \text{NMOD}, 1)\}$)
SHIFT $\Rightarrow$	([0, 2], [3, ..., 9], A_1)
LEFT-ARC _{SBJ} $\Rightarrow$	([0], [3, ..., 9], $A_2 = A_1 \cup \{(3, \text{SBJ}, 2)\}$)
RIGHT-ARC _{ROOT} ^e $\Rightarrow$	([0, 3], [4, ..., 9], $A_3 = A_2 \cup \{(0, \text{ROOT}, 3)\}$)
SHIFT $\Rightarrow$	([0, 3, 4], [5, ..., 9], A_3)
LEFT-ARC _{NMOD} $\Rightarrow$	([0, 3], [5, ..., 9], $A_4 = A_3 \cup \{(5, \text{NMOD}, 4)\}$)
RIGHT-ARC _{OBJ} ^e $\Rightarrow$	([0, 3, 5], [6, ..., 9], $A_5 = A_4 \cup \{(3, \text{OBJ}, 5)\}$)
RIGHT-ARC _{NMOD} ^e $\Rightarrow$	([0, ..., 6], [7, 8, 9], $A_6 = A_5 \cup \{(5, \text{NMOD}, 6)\}$)
SHIFT $\Rightarrow$	([0, ..., 7], [8, 9], A_6)
LEFT-ARC _{NMOD} $\Rightarrow$	([0, ... 6], [8, 9], $A_7 = A_6 \cup \{(8, \text{NMOD}, 7)\}$)
RIGHT-ARC _{PMOD} ^e $\Rightarrow$	([0, ..., 8], [9], $A_8 = A_7 \cup \{(6, \text{PMOD}, 8)\}$)
REDUCE $\Rightarrow$	([0, ..., 6], [9], A_8)
REDUCE $\Rightarrow$	([0, 3, 5], [9], A_8)
REDUCE $\Rightarrow$	([0, 3], [9], A_8)
RIGHT-ARC _P ^e $\Rightarrow$	([0, 3, 9], [], $A_9 = A_8 \cup \{(3, \text{P}, 9)\}$)

Figure 2.5 Arc-eager transition sequence for the English sentence in Figure 2.4.

- LEFT-ARC (r): Add an arc labeled r from *next* to *top*; push *top* onto the second stack.
- RIGHT-ARC (r): Add an arc labeled r from *top* to *next*; push *top* onto the second stack.
- NO-ARC: Push *top* onto the second stack.
- SHIFT: Empty the second stack by pushing every word back onto the stack; then push *next* onto the stack.

2.3.2 Classifiers

Classifiers can be induced from treebank data using a wide variety of different machine learning methods. MaltParser uses support vector machines with a polynomial kernel, as implemented in the LIBSVM package (Chang and Lin, 2001). In addition to this, Malt also provides an option to use external libsvm packages. The task of the classifier is to map a high-dimensional feature vector representation of a parser configuration to the optimal transition out of that configuration.

Features are very crucial for any classifier. The features used in Malt are all symbolic and extracted from the following fields of the CoNLL data representation (Buchholz and Marsi, 2006): FORM, LEMMA, CPOSTAG, POSTAG, FEATS, and DEPREL.¹ Symbolic features are converted to numerical features using the standard technique of binarization. Features of the type DEPREL have a special status in that they are extracted during parsing from the partially built dependency graph and

¹Section 3.4.2 gives detailed description about the CoNLL format

may therefore contain errors, whereas all the other features have gold standard values during both training and parsing. Once we have the list of all possible features, then getting the best feature set is the next important step. General procedure for feature optimization in Malt is, base model is defined and using forward and backward feature selection algorithms, language-specific feature selection is done.

2.4 MST Parser (A Graph-based Dependency Parser)

MST Parser builds a complete graph over words of a sentence, each word being a node. It gives weightages to the edges of the graph based on prior learned contexts. It then proceeds to choose the maximum spanning tree as the output parse.

In what follows, $x = x_1 \dots x_n$ represents a generic input sentence, and y represents a generic dependency tree for sentence x . Considering y as the set of tree edges, $(i, j) \in y$ if there is a dependency in y from word x_i to word x_j . Score of the dependency tree is the sum of the scores of all edges in the tree. In particular, the score of an edge is the dot product between a high dimensional feature representation of the edge and a weight vector,

$$s(i, j) = w \cdot f(i, j)$$

Thus the score of a dependency tree y for sentence x is,

$$s(x, y) = \sum_{(i, j) \in y} s(i, j) = \sum_{(i, j) \in y} w \cdot f(i, j)$$

Assuming an appropriate feature representation as well as a weight vector w , dependency parsing is the task of finding the dependency tree y with highest score for a given sentence x .

For the rest of this section we assume that the weight vector w is known and thus we know the score $s(i, j)$ of each possible edge. In Section 3 we present a method for learning the weight vector.

2.4.1 Maximum Spanning Trees

MSTParser represents a generic directed graph $G = (V, E)$ by its vertex set $V = \{v_1, \dots, v_n\}$ and set $E \subseteq [1 : n] \times [1 : n]$ of pairs (i, j) of directed edges $v_i \rightarrow v_j$. Each such edge has a score $s(i, j)$. Since G is directed, $s(i, j)$ does not necessarily equal $s(j, i)$. A maximum spanning tree (MST) of G is a tree $y \subseteq E$ that maximizes the value $\sum_{(i, j) \in y} s(i, j)$ such that every vertex in V appears in y . The maximum projective spanning tree of G is constructed similarly except that it can only contain projective edges relative to some total order on the vertices of G .

For each sentence x we define the directed graph $G_x = (V_x, E_x)$ given by

$$V_x = \{x_0 = root, x_1, \dots, x_n\}$$

$$E_x = \{(i, j) : i \neq j, (i, j) \in [0 : n] \times [1 : n]\}$$

That is, G_x is a graph with the sentence words and the dummy root symbol as vertices and a directed edge between every pair of distinct words and from the root symbol to every word. It is clear that dependency trees for x and spanning trees for G_x coincide, since both kinds of trees are required to be rooted at the dummy root and reach all the words in the sentence. Hence, finding a (projective) dependency tree with highest score is equivalent to finding a maximum (projective) spanning tree in G_x .

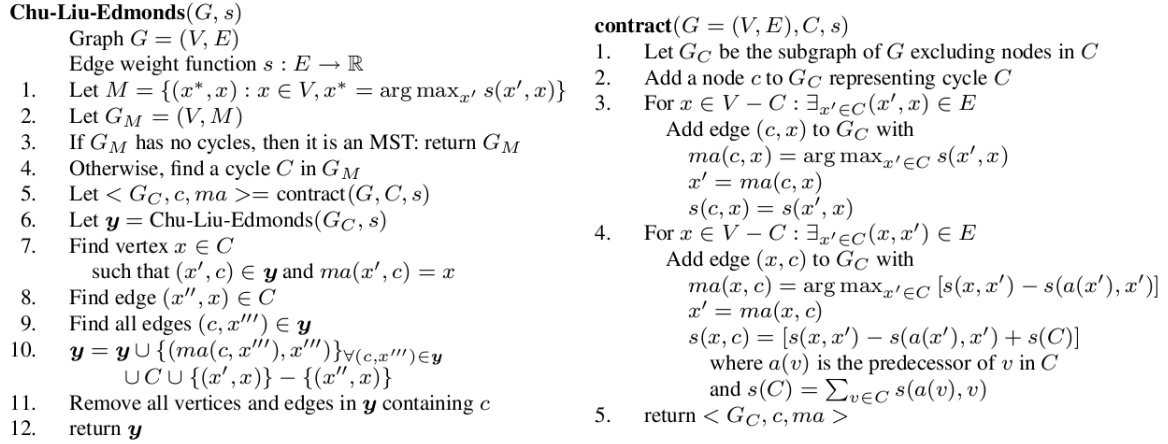
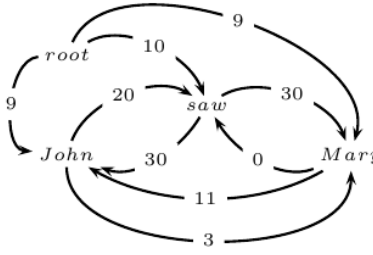


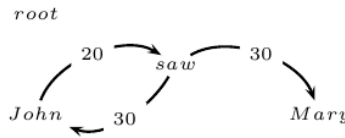
Figure 2.6 Chu-Liu-Edmonds algorithm for finding maximum spanning trees in directed graphs

2.4.2 Parsing Algorithm

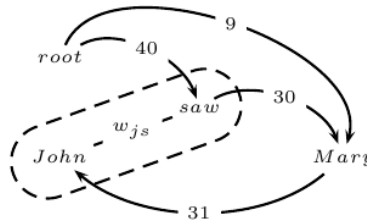
To find the highest scoring non-projective tree one simply need to search the entire space of spanning trees with no restrictions. Well-known algorithms exist for the less general case of finding spanning trees in undirected graphs (Cormen et al., 1990). Efficient algorithms for the directed case are less well known, but they exist. MSTParser uses the Chu-Liu-Edmonds algorithm (Chu and Liu, 1965; Edmonds, 1967), sketched in Figure 2.6. Informally, the algorithm has each vertex in the graph greedily select the incoming edge with highest weight. If a tree results, it must be the maximum spanning tree. If not, there must be a cycle. The procedure identifies a cycle and contracts it into a single vertex and recalculates edge weights going into and out of the cycle. It can be shown that a maximum spanning tree on the contracted graph is equivalent to a maximum spanning tree in the original graph (Georgiadis, 2003). Hence the algorithm can recursively call itself on the new graph. Naively, this algorithm runs in $O(n^3)$ time since each recursive call takes $O(n^2)$ to find the highest incoming edge for each word and to contract the graph. There are at most $O(n)$ recursive calls since we cannot contract the graph more than n times. To find the highest scoring non-projective tree for a sentence, x , the graph G_x is constructed and ran through the Chu-Liu-Edmonds algorithm. The resulting spanning tree is the best non-projective dependency tree. We illustrate here the application of the Chu-Liu-Edmonds algorithm to dependency parsing on the simple example $x = \text{'John saw Mary'}$, with directed graph representation G_x ,



The first step of the algorithm is to find, for each word, the highest scoring incoming edge



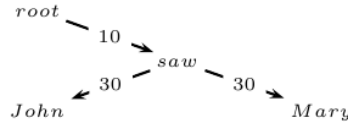
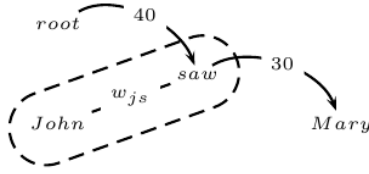
If the result were a tree, it would have to be the maximum spanning tree. However, in this case we have a cycle, so we will contract it into a single node and recalculate edge weights according to Figure 2.6.



The new vertex w_{js} represents the contraction of vertices ‘John’ and ‘saw’. The edge from w_{js} to ‘Mary’ is 30 since that is the highest scoring edge from any vertex in w_{js} . The edge from ‘root’ into w_{js} is set to 40 since this represents the score of the best spanning tree originating from root and including only the vertices in w_{js} . The same leads to the edge from ‘Mary’ to w_{js} . The fundamental property of the Chu-Liu-Edmonds algorithm is that an MST in this graph can be transformed into an MST in the original graph (Georgiadis, 2003). Thus, the algorithm is recursively called on this graph. Note that we need to keep track of the real endpoints of the edges into and out of w_{js} for reconstruction later. Running the algorithm, one must find the best incoming edge to all words.

This is a tree and thus the MST of this graph. We now need to go up a level and reconstruct the graph. The edge from w_{js} to ‘Mary’ originally was from the word ‘saw’, so we include that edge. Furthermore, the edge from root to w_{js} represented a tree from ‘root’ to ‘saw’ to ‘John’, so we include all those edges to get the final (and correct) MST,

In this manner, using Chu-Liu-Edmonds algorithm, non-projective dependency trees can be generated. Many languages that allow non-projectivity are still primarily projective. So, for projective dependency parsing, MSTParser uses Eisner algorithm (Eisner, 1996). It is well known that projective



dependency parsing using edge based factorization can be handled with the Eisner algorithm (Eisner, 1996). This algorithm has a runtime of $O(n^3)$ and has been employed successfully in both generative and discriminative parsing models (Eisner, 1996; McDonald et al., 2005). Furthermore, it is trivial to show that the Eisner algorithm solves the maximum projective spanning tree problem. The Eisner algorithm differs significantly from the Chu-Liu-Edmonds algorithm. First of all, it is a bottom-up dynamic programming algorithm as opposed to a greedy recursive one. A bottom-up algorithm is necessary for the projective case since it must maintain the nested structural constraint, which is unnecessary for the non-projective case.

In the preceding discussion, we have seen how MSTParser reduces the task of natural language dependency parsing to finding maximum spanning trees in directed graphs. This reduction results from edge-based factorization and is applied to projective languages with the Eisner parsing algorithm and non-projective languages with the Chu-Liu-Edmonds maximum spanning tree algorithm. A major advantage of this approach over other dependency parsing models is its uniformity and simplicity. By viewing dependency structures as spanning trees, a general framework for parsing trees for both projective and non-projective languages is provided. Furthermore, the resulting parsing algorithms are more efficient than lexicalized phrase structure approaches to dependency parsing, allowing the parser to search the entire space without any pruning. In particular the non-projective parsing algorithm based on the Chu-Liu-Edmonds MST algorithm provides true non-projective parsing. This is in contrast to other non-projective methods, such as that of Nivre and Nilsson (2005), who implement non-projectivity in a pseudo-projective parser with edge transformations. This formulation also dispels the notion that non-projective parsing is “harder” than projective parsing. In fact, it is easier since non-projective parsing does not need to enforce the non-crossing constraint of projective trees. As a result, non-projective parsing complexity is just $O(n^2)$, against the $O(n^3)$ complexity of the Eisner dynamic programming algorithm, which by construction enforces the non-crossing constraint.

2.4.3 Learning and Feature Selection

In the previous section we have seen that given the weight vector w and high dimensional feature representation of the edge $f(i, j)$, how to find the dependency tree y for a given sentence x . In this section, we see how the high dimensional feature representation and the weight vector are computed.

Margin Infused Relaxed Algorithm (MIRA) (Crammer and Singer, 2003; Crammer et al., 2003), an online large-margin learning algorithm, is used to compute the weight vector. Its power lies in the ability to define a rich set of features over parsing decisions, as well as surface level features relative to these decisions. For instance, it incorporates features over the part of speech of words occurring between and around a possible head-dependent relation. These features are highly important to overall accuracy since they eliminate unlikely scenarios such as a preposition modifying a noun not directly to its left, or a noun modifying a verb with another verb occurring between them.

MSTParser uses three different types of features namely, basic, extended, and second-order features. Consider an edge $(i, j) \in y$, where x_i is the modifier and x_j is the head. The basic set of features used by MSTParser are shown in Figure 2.7 a and b. The unigram features provide the information about the modifier and the head separately. The bigram features provide the conjoined information of the modifier and the head together.

a)	b)															
<table> <tr> <th>Basic Uni-gram Features</th></tr> <tr> <td>x_i-word, x_i-pos</td></tr> <tr> <td>x_i-word</td></tr> <tr> <td>x_i-pos</td></tr> <tr> <td>x_j-word, x_j-pos</td></tr> <tr> <td>x_j-word</td></tr> <tr> <td>x_j-pos</td></tr> </table>	Basic Uni-gram Features	x_i -word, x_i -pos	x_i -word	x_i -pos	x_j -word, x_j -pos	x_j -word	x_j -pos	<table> <tr> <th>Basic Bi-gram Features</th></tr> <tr> <td>x_i-word, x_i-pos, x_j-word, x_j-pos</td></tr> <tr> <td>x_i-pos, x_j-word, x_j-pos</td></tr> <tr> <td>x_i-word, x_j-word, x_j-pos</td></tr> <tr> <td>x_i-word, x_i-pos, x_j-pos</td></tr> <tr> <td>x_i-word, x_i-pos, x_j-word</td></tr> <tr> <td>x_i-word, x_j-word</td></tr> <tr> <td>x_i-pos, x_j-pos</td></tr> </table>	Basic Bi-gram Features	x_i -word, x_i -pos, x_j -word, x_j -pos	x_i -pos, x_j -word, x_j -pos	x_i -word, x_j -word, x_j -pos	x_i -word, x_i -pos, x_j -pos	x_i -word, x_i -pos, x_j -word	x_i -word, x_j -word	x_i -pos, x_j -pos
Basic Uni-gram Features																
x_i -word, x_i -pos																
x_i -word																
x_i -pos																
x_j -word, x_j -pos																
x_j -word																
x_j -pos																
Basic Bi-gram Features																
x_i -word, x_i -pos, x_j -word, x_j -pos																
x_i -pos, x_j -word, x_j -pos																
x_i -word, x_j -word, x_j -pos																
x_i -word, x_i -pos, x_j -pos																
x_i -word, x_i -pos, x_j -word																
x_i -word, x_j -word																
x_i -pos, x_j -pos																
c)	d)															
<table> <tr> <th>In Between POS Features</th></tr> <tr> <td>x_i-pos, b-pos, x_j-pos</td></tr> <tr> <th>Surrounding Word POS Features</th></tr> <tr> <td>x_i-pos, x_i-pos+1, x_j-pos-1, x_j-pos</td></tr> <tr> <td>x_i-pos-1, x_i-pos, x_j-pos-1, x_j-pos</td></tr> <tr> <td>x_i-pos, x_i-pos+1, x_j-pos, x_j-pos+1</td></tr> <tr> <td>x_i-pos-1, x_i-pos, x_j-pos, x_j-pos+1</td></tr> </table>	In Between POS Features	x_i -pos, b-pos, x_j -pos	Surrounding Word POS Features	x_i -pos, x_i -pos+1, x_j -pos-1, x_j -pos	x_i -pos-1, x_i -pos, x_j -pos-1, x_j -pos	x_i -pos, x_i -pos+1, x_j -pos, x_j -pos+1	x_i -pos-1, x_i -pos, x_j -pos, x_j -pos+1	<table> <tr> <th>Second-order Features</th></tr> <tr> <td>x_i-pos, x_k-pos, x_j-pos</td></tr> <tr> <td>x_k-pos, x_j-pos</td></tr> <tr> <td>x_k-word, x_j-word</td></tr> <tr> <td>x_k-word, x_j-pos</td></tr> <tr> <td>x_k-pos, x_j-word</td></tr> </table>	Second-order Features	x_i -pos, x_k -pos, x_j -pos	x_k -pos, x_j -pos	x_k -word, x_j -word	x_k -word, x_j -pos	x_k -pos, x_j -word		
In Between POS Features																
x_i -pos, b-pos, x_j -pos																
Surrounding Word POS Features																
x_i -pos, x_i -pos+1, x_j -pos-1, x_j -pos																
x_i -pos-1, x_i -pos, x_j -pos-1, x_j -pos																
x_i -pos, x_i -pos+1, x_j -pos, x_j -pos+1																
x_i -pos-1, x_i -pos, x_j -pos, x_j -pos+1																
Second-order Features																
x_i -pos, x_k -pos, x_j -pos																
x_k -pos, x_j -pos																
x_k -word, x_j -word																
x_k -word, x_j -pos																
x_k -pos, x_j -word																

Figure 2.7 Features used by MSTParser. where x_i is the head and x_j the modifier

Using just features over head-modifier pairs in the tree is not enough for high accuracy since all attachment decisions are made outside of the context in which the words occurred. To solve this problem, extended features are added, which can be seen in Figure 2.7c. The first new feature class recognizes word types that occur between the head and modifier words in an attachment decision. These features

take the form of POS trigrams: the POS of the head, that of the modifier, and that of a word in between, for all distinct POS tags for the words between the head and the modifier. These features were particularly helpful for nouns to select their heads correctly, since they help reduce the score for attaching a noun to another noun with a verb in between, which is a relatively infrequent configuration. The second class of extended features represents the local context of the attachment, that is, the words before and after the head-modifier pair. These features take the form of POS 4-grams: The POS of the head, modifier, word before/after head and word before/after modifier.

The extended features can be efficiently added since they are given as part of the input and do not rely on knowledge of dependency decisions outside the current edge under consideration. But sometimes dependency decisions outside the current edge would be useful in making the current edge decision. For this, MST provides option for second-order features as shown in Figure 2.7d. With the help of these features, one can consider the information about the siblings in the dependency tree.

2.5 Summary

In this Chapter, we gave a brief overview of what is dependency parsing and the data-driven dependency parsers used for our experiments. We started with a brief description on dependency parsing. Then we mentioned different approaches commonly followed for dependency parsing. We described two state-of-the-art data-driven dependency parsers used for our experiments in a very detailed manner. In the following section, we will see details about data, framework, and the formats used in our experiments.

Chapter 3

Hindi Treebank

In this chapter, we briefly describe about Hindi Language and Treebanks. First we talk briefly about Hindi language. Then we describe the paninian grammatical model used for dependency representation of Hindi. Following this, we describe different treebanks available for Hindi and different formats used for representing these treebanks.

3.1 Hindi Language

Hindi is the official language of the union of India, and is spoken by ~ 800 million people. It is a verb final language with free word order as SOV (Subject, Object, Verb) or OSV. This can be seen in (1), where (1a) shows the constituents in the default order, and the remaining examples show some of the word order variants of (1a) ¹.

- (1) a. malaya ne sameer ko kitaba dii.
Malay ERG Sameer DAT book gave
“Malay gave the book to Sameer” (S-IO-DO-V)
- b. malaya ne kitaba sameer ko dii. (S-DO-IO-V)
- c. sameer ko malaya ne kitaba dii. (IO-S-DO-V)
- d. sameer ko kitaba malaya ne dii. (IO-DO-S-V)
- e. kitaba malaya ne sameer ko dii. (DO-S-IO-V)
- f. kitaba sameer ko malaya ne dii. (DO-IO-S-V)

Hindi also has a rich case marking system, although case marking is not obligatory. For example, in (1), while the subject and indirect object are explicitly marked for the ergative ² (ERG) and dative (DAT) cases, the direct object is unmarked for the accusative.

¹S=Subject; IO=Indirect Object; DO=Direct Object; V=Verb; ERG=Ergative; DAT=Dative

²Hindi is split-ergative. The ergative marker appears on the subject of a transitive verb with perfect morphology.

3.2 Paninian Grammatical Model

Indian Languages (ILs) including Hindi are morphologically rich and have a relatively flexible word order. For such languages syntactic subject-object are not able to explain the varied linguistic phenomena. In fact, there is a debate in the literature whether the notions ‘subject’ and ‘object’ can at all be defined for ILs (Mohan, 1982). Behavioral properties are the only criteria based on which one can confidently identify grammatical functions in Hindi (Mohan, 1994); it can be difficult to exploit such properties computationally. Marking semantic properties such as thematic role as dependency relation is also problematic. Thematic roles are abstract notions and will require higher semantic features which are difficult to formulate and to extract as well. Paninian grammatical model (Kiparsky and Staal, 1969; Shastri, 1973) provides a level which while being syntactically grounded also helps in capturing semantics. In this section we briefly discuss the Paninian grammatical model for ILs and lay down some basic concepts inherent to this framework.

The Paninian framework considers information as central to the study of language. When a writer/speaker uses language to convey some information to the reader/hearer, he codes the information in the language string. Similarly, when a reader/hearer receives a language string, he extracts the information coded in it. The Paninian grammatical model is primarily concerned with: (a) how the information is coded and (b) how it can be extracted.

Two levels of representation can be readily understood in language: One, the actual language string (or sentence), two, what the speaker has in his mind. The latter can also be called as the meaning. Paninian framework has two other important levels: karaka level and vibhakti level

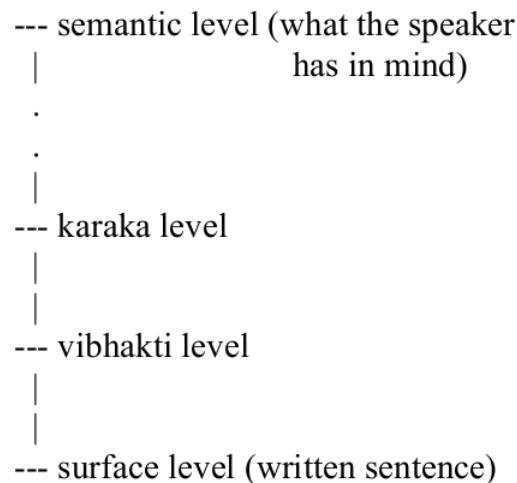


Figure 3.1 Levels of representation/analysis in the Paninian model

The surface level is the uttered or the written sentence. The vibhakti level is the level at which there are local word groups together with case endings, preposition or postposition markers. The vibhakti level abstracts away from many minor (including orthographic and idiosyncratic) differences among

languages. Above the vibhakti level is the karaka level. It includes karaka relations and a few additional relations such as taadaarthya (or purpose). The topmost level relates to what the speaker has in his mind. This may be considered to be the ultimate meaning level that the speaker wants to convey. One can imagine several levels between the karaka and the ultimate level, each containing more semantic information. Thus, karaka level is one in a series of levels, but one which has relationship to semantics on the one hand and syntax on the other.

At the karaka level, we have karaka relations and verb-verb relations, etc. Karaka relations are syntactico-semantic relations between the verbs and other related constituents (typically nouns) in a sentence. They capture a certain level of semantics which is somewhat similar to thematic relations but different from it (Bharati et al., 1995). This is the level of semantics that is important syntactically and is reflected in the surface form of the sentence(s). Begum et al. (2008) have subsequently proposed an annotation scheme based on Paninian framework. They have extended the original formulation to account for previously unhandled syntactic phenomenon.

The Paninian approach treats a sentence as a set of modifier-modified relations. A sentence is supposed to have a primary modified which is generally the main verb of the sentence. The elements modifying the verb participate in the action specified by the verb. The participant relations with the verb are called karaka. The notion of karaka will incorporate the ‘local’ semantics of the verb in a sentence, while also taking cue from the surface level morpho-syntactic information (Vaidya et al., 2009). There are six basic karakas, namely;

- k1: karta (This is akin to subject and agent, but different from them): the most independent participant in the action
- k2: karma (roughly the theme or object): the one most desired by the karta
- k3: karana (instrument): which is most essential for the action to take place
- k4: sampradaan (beneficiary): recipient or beneficiary of the action
- k5: apaadaan (source): movement away or separation from a source
- k7: adhikarana (location): location of the action in time and space

From the above description, it is easy to see that this analysis is a dependency based analysis (Kiparsky and Staal, 1969; Shastri, 1973), with verb as the root of the tree along with its argument structure as its children. The labels on the edges between a child-parent pair show the relationship between them. In addition to the above six labels many other have been proposed as part of the overall framework (Begum et al., 2008; Bharati et al., 2009b). Appendix C shows the complete set of dependency edge labels.

The analysis of sentence generally begins with verb’s demands (aakaankshaa) for its arguments. The arguments are identified, taking verb’s meaning into consideration. Their relationship with the verb is established using karaka and other relations. The discovery procedure of dependency relations depends

on the morpho-syntactic information. Only those elements that exhibit certain properties (yogyata) are selected. The verb generally selects karta or the karma based on its TAM (tense, aspect and modality) marker. This selection is shown syntactically either via agreement or some case markings. There exists, therefore, a TAM- vibhakti correspondence that can help identify certain relations. Other relations can also be indentified based on similar surface cues. In certain context, semantics constraints on a word can help identify its relationship with its parent, and can be thought as its yogyata.

In the following sections, we provide details of the treebanks annotated for Hindi using this Paninian grammatical model.

3.3 Treebanks

Two different treebanks are currently available for Hindi. Details about both the treebanks is provided in the following sections.

3.3.1 HyDT-Hindi

Begum et al. (2008) is the first dependency treebank annotated for Hindi. This treebank is the first ever attempt at constructing dependency treebanks for Indian Languages (IL). This treebank is called as HyDT-Hindi (Hyderabad Dependency Treebank-Hindi). A portion of the raw Hindi corpus obtained from CIIL (Central Institute for Indian Languages), Mysore, India is used for annotation. Details of the information encoded in this treebank is mentioned below.

POS Information: Each lexical item in a sentence is annotated with its part-of-speech (POS) tag. POS and chunk annotation guidelines (Bharati et al., 2006) are used for this purpose. List of POS tags used for annotation can be found in Appendix A.1.

Chunk Information: After annotation of POS tags, each sentence is manually chunked. A chunk is a minimal, non-recursive structure consisting of correlated groups of words (Bharati et al., 2006). A chunk represents a set of adjacent words which are in dependency relations with each other, and are connected to the rest of the words by incoming or outgoing dependency arcs. POS and chunk annotation guidelines (Bharati et al., 2006) are used for this purpose. List of chunk tags used for annotation can be found in Appendix A.2.

Dependency Information: After POS, and chunk annotation, dependency annotation is done following the set of dependency guidelines in Bharati et al. (2009b). This information is encoded at the syntactico-semantic level following the Paninian grammatical model (Begum et al., 2008; Bharati et al., 1995) described in section 3.2. Appendix C shows the complete list of dependency edge labels used for annotation.

Note that the manual dependency annotation is done at chunk-level only. The relations among the words in a chunk are not marked for now and hence allows to ignore local details while building the sentence level dependency tree. Thus, in the dependency tree each node is a chunk and the edge rep-

resents the relations between the connected nodes labeled with the karaka or other relations. All the modifier-modified relations between the heads of the chunks (inter-chunk relations) are marked in this manner.

This treebank comprises of around 2300 manually annotated sentences. Out of this data 1800 sentences were extracted and released for ICON-2009 Tools Contest. Average sentence length is 18.3 words/sentence and 8.99 chunks/sentence for this data.

3.3.2 Hindi Treebank (under development)

A new multi layered and multi representational treebank for Hindi (Bhatt et al., 2009; Xia et al., 2009) is being developed. It is considered to be multi-representational as linguistic information is represented in different forms, dependency and phrase structure. There is a clear distinction in how the same piece of linguistic information is shown in different forms. There is merit in both these forms of representation as discussed by Bhatt et al. (2009). On the other hand, multi-layered alludes to the different types of information stored in different forms. For instance, information is annotated at the morpho-syntactic (morphological, POS, chunk), syntactico-semantic (dependency relations) and lexical semantic levels (PropBank (Palmer et al., 2005)).

This treebank is currently under large-scale development. Manual dependency annotation is being done. Automatic conversion from dependency structure (DS) to phrase structure (PS) is planned. As described in the section 3.2, Paninian grammatical model is being used for dependency annotation. Different levels of information encoded at dependency representation of the treebank is described in the following sections.

POS Information: POS tags are annotated for each node in the sentence following the POS and chunk annotation guidelines (Bharati et al., 2006). List of POS tags used for annotation can be found in Appendix B. Note that this POS tagset is slightly different from the one used for *old treebank* as there are slight changes in the tagset. In some cases the tag name is changed whereas in some cases, the changes are more substantial. For example, POS tag for post-position markers is ‘PREP’ in the old guidelines, and ‘PSP’ is the new guidelines. Here only the name of the tag got changed. But in case of verbs, there is major change in the decision. In the old guidelines, VFM, VJJ, VRB, VNN are used for finite, adjectival non-finite, adverbial non-finite, and nominal non-finite verbs respectively. But in new guidelines, all these tags are merged into a single tag, VM, taking into account the nature of Hindi (and many other Indian languages) where the verbal form does not indicate finiteness by itself without the auxiliaries.

Morph Information: Information pertaining to the morphological features of the nodes is also encoded using the Shakti standard format (SSF) (Bharati et al., 2007). These morphological features have eight mandatory feature attributes for each node. These features are classified as root, category, gender, number, person, case, post position (for a noun) or tense aspect modality (for a verb) and suffix. More details can be found in the section 3.4.1, where a brief description of SSF format is given.

Chunk Information: After annotation of POS tags, chunk boundaries are marked with appropriate assignment of chunk labels (Bharati et al., 2006). List of Chunk tags used for annotation can be found in Appendix B.2. Note that similar to POS tagset, this chunk tagset is slightly different from the one used for *old treebank* as there are slight changes in the guidelines. For example, VG is the chunk tag for all types of chunks in the old guidelines. But as per new guidelines, VGF, VGNN, VGNF, and VGINF are the chunk tags for finite, gerund, non-finite and infinitival verb chunks respectively (For more details, refer Bharati et al. (2006)).

Dependency Relations: After POS, morph and chunk annotation, dependency annotation is done following the set of dependency guidelines in Bharati et al. (2009b). This information is encoded at the syntactico-semantic level following from the Paninian dependency framework (Begum et al., 2008; Bharati et al., 1995) described in section 3.2.

Other Features: In the dependency treebank, apart from POS, morph, chunk and dependency annotations, some special features for some nodes are marked. For example, for the main verb of a sentential clause, information about whether the clause is declarative, interrogative or imperative is marked. Similarly, whether the sentence is in active or passive voice is also marked.

Target of this treebank is 400k words. Currently, around 150k word data is available which is manually annotated and validated. Out of this data, first 90k data is released for ICON-2010 Tools Contest. There are around 4000 sentences in this data. Average sentence length is 22.69 words/sentence and 10.6 chunks/sentence for this data.

3.4 Representation

Both the treebanks are available in two different formats namely SSF and CoNLL. Actual annotation is done following Shakti Standard Format (SSF) (Bharati et al., 2007). The annotated data is also converted to CoNLL format for the convenience of the experiments using data-driven parsers like Malt and MST.

3.4.1 SSF Format

The SSF format has four columns. Token id, token/chunk boundaries, POS/Chunk tags and feature structure appear in the four columns respectively. Detailed description of SSF can be found in Bharati et al. (2007). Consider an example Hindi sentence shown below.

(1)	raama	PZala	KAwa	hE
	'Ram'	'fruit'	'eat'	PRES
	'Ram eats a fruit'			

For the above sentence, SSF representation with POS, chunk and dependency annotation would look like

```

1      ((      NP      <fs drel='k1:VGF' name='NP'>
1.1    rAma    NNP
      ))
2      ((      NP      <fs drel='k2:VGF' name='NP2'>
2.1    PZala   NN
      ))
3      ((      VGF      <fs name='VGF'>
3.1    KAwA    VM
3.2    hE      VAUX
      ))

```

The above SSF shows the relations between chunk heads. The fourth field of the SSF contains the dependencies. The head is identified by a unique id using an attribute value pair. This can be seen above at node no. 3, where 'name' is given an id 'VGF'. The dependents are then related to the head using 'drel' attribute. The value for drels is "dependency relation:head id".

As we can see this SSF representation provides information at chunk level only i.e., only inter-chunk dependency relations are available. Intra-chunk dependencies are not available. This is how HyDT-Hindi looks like.

Even the Hindi treebank (under development) looks similarly as far as manual annotation is considered. On top of it running of an automatic tool which identifies the intra-chunk relations is planned. In this way we can get complete sentence level annotation rather than only chunk level annotation. The expanded tree for the above sentence will look like below. Note that the chunk boundary and the chunk head information is retained in the expanded trees via 'chunkId' and 'chunkType' features.

```

1    rAma    NNP    <fs drel='k1:KAwA' name='rAma' chunkId='NP' chunkType='head:NP'>
2    PZala   NN     <fs drel='k2:KAwA' name='PZala' chunkId='NP2' chunkType='head:NP2'>
3    KAwA    VM     <fs name='KAwA' chunkId='VGF' chunkType='head:VGF'>
4    hE      VAUX   <fs drel='lwg__vaux:KAwA' name='hE' chunkType='child:VGF'>

```

In addition to the dependency information, one can also add other morphological information in the fourth field. The 2nd node in the above SSF with the morph information and head computation will look like:

```

2      ((      NP      <fs af='PZala,n,m,s,3,0,' drel='k2:VGF'>
2.1    PZala   NN      <fs af='PZala,n,m,s,3,0,'>
      ))

```

'af' above stands for abbreviated feature structure and represents the following info.

PZala,	n,	m,	s,	3,	,	0,	
root	cat	gen	num	per	cas	vib/tam	suf
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)

1. Root: Root form of the word
2. Category: Course grained POS
3. Gender: Masculine/Feminine/Neuter
4. Number: Singular/Plural
5. Person: First/Second/Third person
6. Case: Oblique/Direct case
7. Vibhakti (suffix/post-positions, etc.)/TAM (tense, aspect and modality)
8. Suffix: Suffix of the word

3.4.2 CoNLL Format

Apart from SSF format, both the treebanks are converted to CoNLL format ³. CoNLL format is the standard format being used in CoNLL Shared Tasks on dependency parsing. This is a ten column format. A short description of these columns is mentioned in Table 3.4.2.

Field number:	Field name:	Description:
1	ID	Token counter, starting at 1 for each new sentence
2	FORM	Word form or punctuation symbol
3	LEMMA	Lemma or stem of word form, or an underscore if not available
4	CPOSTAG	Coarse-grained POS tag
5	POSTAG	Fine-grained POS tag
6	FEATS	Unordered set of syntactic and/or morphological features, separated by a vertical bar ()
7	HEAD	Head of the current token, which is either a value of ID or zero ('0')
8	DEPREL	Dependency relation to the HEAD
9	PHEAD	Projective head of current token, which is either a value of ID or zero ('0')
10	PDEPREL	Dependency relation to the PHEAD

Table 3.1 Columns in CoNLL format

The CoNLL format for the above example sentence (1) will be:

³<http://nextens.uvt.nl/depparse-wiki/DataFormat>

1	rAma	rAma	n	NNP	gen-m num-s per-3 cas-\$ vib-0 suf-\$	3	k1	-	-
2	PZala	PZala	n	NN	gen-m num-s per-3 cas-\$ vib-0 suf-\$	3	k2	-	-
3	KAwa	KA	v	VM	gen-m num-s per-3 cas-\$ vib-wA_hE suf-wA	0	main	-	-
4	hE	hE	v	VAUX	gen-m num-s per-3 cas-\$ vib-hE suf-hE	3	lwg__vaux	-	-

Each row/line represents a node in the sentence. Each sentence is separated by a blank line. The format can handle UTF. Out of the ten columns, the columns ID, FORM, CPOSTAG, POSTAG, HEAD and DEPREL are mandatory and the rest are optional. In case of optional columns, if the information is not available, an underscore is present. Except FEATS column, all other columns are fixed. But, in FEATS column we can have any useful information other than the information encoded in the fixed columns.

Consider the third row in the the CoNLL format presented above. ‘3’ is the ID of the node in the sentence. ‘KAwa’ is the word and ‘KA’ is the root form of the word ‘KAwa’. ‘v’ and ‘VM’ are the coarse-grained POS tag and fine-grained POS tags respectively. As this node is the root of the sentence, HEAD and DEPREL are ‘0’ and ‘main’ respectively. All these are fixed columns. FEATS column is used to represent the extra information in the form of gender, number, person, case, vibhakti and suffix.

As the actual annotation is done using SSF, we have written scripts to convert the SSF data into CoNLL format for convenience of the data-driven parsers used.

Chapter 4

Hindi Dependency Parsing: Chunk-level

4.1 Introduction

The dependency parsing community has for the last few years shown considerable interest in parsing morphologically rich languages with flexible word order. This is partly due to the increasing availability of dependency treebanks for such languages, but it is also motivated by the observation that the performance obtained for these languages has not been very high (Nivre et al., 2007a). Attempts at handling various non-configurational aspects in these languages have pointed towards shortcomings in traditional parsing methodologies (Tsarfaty and Sima'an, 2008; Eryiğit et al., 2008; Seddah et al., 2009; Husain et al., 2009; Gadde et al., 2010). Among other things, it has been pointed out that the use of language specific features may play a crucial role in improving the overall parsing performance. Different languages tend to encode syntactically relevant information in different ways, and it has been hypothesized that the integration of morphological and syntactic information could be a key to better accuracy. However, it has also been noted that incorporating these language specific features in parsing is not always straightforward and many intuitive features do not always work in expected ways.

In this chapter, we present our work on data-driven dependency parsing for Hindi. We explore two data-driven parsers, namely MaltParser and MSTParser for our experiments. We study the role of different morphosyntactic features in Hindi dependency parsing. These experiments led to state-of-the-art dependency parser for Hindi.

4.2 Approach

We used two data driven parsers described in Chapter 2, namely Malt ¹ (Nivre et al., 2007b), and MST ² (McDonald et al., 2006) for our experiments.

Malt is a classifier based Shift/Reduce parser. It uses arc-eager, arc-standard, covington projective and convington non-projective algorithms for parsing (Nivre et al., 2006). History-based feature models

¹Malt Version 1.3.1

²MST Version 0.4b

are used for predicting the next parser action (Black et al., 1992). Support vector machines are used for mapping histories to parser actions (Kudo and Matsumoto, 2002). It uses graph transformation to handle non-projective trees (Nivre and Nilsson, 2005).

MST uses Chu-Liu-Edmonds (Chu and Liu, 1965; Edmonds, 1967) Maximum Spanning Tree algorithm for non-projective parsing and Eisner’s algorithm for projective parsing (Eisner, 1996). It uses online large margin learning as the learning algorithm (McDonald et al., 2005).

Malt provides an xml file where we can specify the features for the parser. But for MST, these features are hard coded. Accuracy of the labeler of MST is very low. We tried to modify the code but couldn’t get better results. So, we used maximum entropy classification algorithm, MaxEnt³ for labeling. First we ran MST for unlabeled dependency tree. On the output of MST we used maximum entropy algorithm for labeling. This is similar to the work of Dai et al. (2009).

4.3 Settings

4.3.1 Data Settings

We have considered Hindi data of ICON09 Tools Contest (Husain, 2009) for our experiments. This data is a subset of old hindi treebank (Begum et al., 2008). This data comprises of 1500 sentences for training, 150 for development and 150 for testing. Average sentence length is 18.3 words/sentence and 8.99 chunks/sentence for this data. This data has only chunk-level annotation. So, the task would be given a sentence with gold POS and chunk information, we have to identify the relation between chunk heads. In other words, parsing is at the chunk level rather than complete sentence level.

As both the parsers take CoNLL format as input, we have taken data in CoNLL format for our experiments. The FEATS column of each node in the data has 6 fields. These are six morphological features namely category, gender, number, person, vibhakti⁴ or TAM⁵ markers of the node. We experimented considering different combinations of these fields for both the parsers. In all the experiments, vibhakti and TAM fields gave better results than others. This is similar to the settings of Bharati et al. (2008a). They showed that for Hindi, vibhakti and TAM markers help in dependency parsing where as gender, number, person markers won’t.

We also explored using TAM classes instead of TAM markers (Bharati et al., 2008a). TAM markers which behave similarly are grouped into a class. This reduced the number of features and training time but there isn’t any significant improvement in the accuracies. For example, genitive vibhakti markers (kA, ke, kI) are the major clue for ‘r6’ relation. Instead of considering 3 different TAM labels (kA, ke, kI), we grouped these three TAM labels into a single class (V-r6) as they behave similarly. But in case of pronouns the vibhakti feature doesn’t provide this information. But the category given by morph will

³<http://maxent.sourceforge.net/>

⁴Vibhakti is a generic term for preposition, post-position and suffix.

⁵TAM is Tense, Aspect and Modality marker.

	POSTAG	CPOSTAG	FORM	LEMMA	DEPREL	FEATS	OTHERS
Stack: <i>top</i>	1	5	1	7		9	
Input: <i>next</i>	1	5	1	7		9	
Input: <i>next+1</i>	2	5	1	7			
Input: <i>next+2</i>	2						
Input: <i>next+3</i>	2						
Stack: <i>top-1</i>	3						
String: predecessor of <i>top</i>	3						
Tree: head of <i>top</i>	4						
Tree: leftmost dep of <i>next</i>	4	5	6				
Tree: rightmost dep of <i>top</i>					8		
Tree: left sibling of rightmost dep of <i>top</i>					8		
Merge: POSTAG of <i>top</i> and <i>next</i>							10
Merge: FEATS and DEPREL of <i>top</i>							10

Table 4.1 Feature pool used for arc-eager algorithm of Malt.

have a special value ‘sh_P’ for such cases. Using this information and suffix of the pronoun, we gave ‘V-r6’ as the TAM class.

In the following sections, we describe different parser and feature settings explored for both the parsers.

4.3.2 Malt: General Settings

Malt provides options for four parsing algorithms arc-eager, arc-standard, covington projective, covington non-projective. We experimented with all the algorithms and arc-eager gave better performance over others consistently. It provides option for one learning algorithm which is libsvm. Tuning the SVM model was difficult; we tried various parameters but could not find any fixed pattern. Finally, we tested the performance by adapting the CoNLL shared task 2007 (Nivre et al., 2007b) SVM settings used by the same parser for various languages (Hall et al., 2007).

4.3.3 Malt: Feature Selection

We have taken wide variety of features and grouped them into 10 groups (indicated by numbers 1 - 10 in Table 4.1). In Table 4.1, first column lists different features and first row represents different columns in the CoNLL format. We used 5-fold cross-validation on the combined training and development sets from the ICON09 tools contest to select the pool of features depicted in Table 4.1. For this forward-feature selection technique was used to incrementally add different feature groups and analyzed their impact on parsing accuracy. The result is shown in Figure 4.1.

Experiment 1: Experiment 1 uses a baseline model with only four basic features: POSTAG and FORM of *top* and *next*. This results in a labeled attachment score (LAS) of 41.7% and an unlabeled attachment score (UAS) of 68.2%.

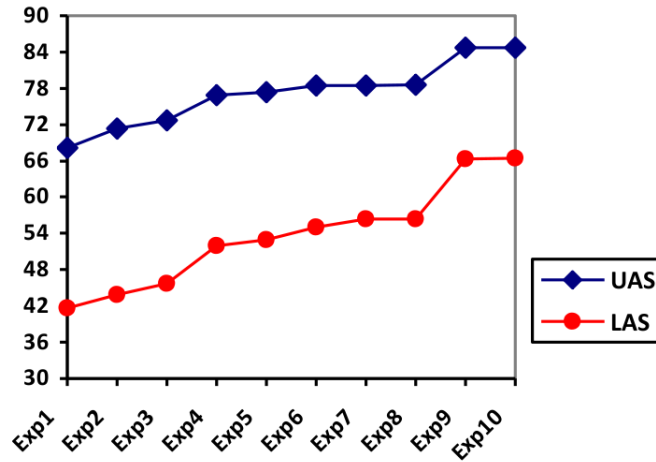


Figure 4.1 UAS and LAS of experiments 1-10; 5-fold cross-validation on training and development data of the ICON09 tools contest

Experiments 2–3: In experiments 2 and 3, the POSTAG of contextual words of next and top are added. Of all the contextual words, *next+1*, *next+2*, *next+3*, *top-1* and predecessor of *top* were found to be useful⁶. Adding these contextual features gave a modest improvement to 45.7% LAS and 72.7% UAS.

Experiment 4: In experiment 4, we used the POSTAG information of nodes in the partially built tree, more specifically the syntactic head of *top* and the leftmost dependent of *next*. Using these features gave a large jump in accuracy to 52% LAS and 76.8% UAS. This is because partial information is helpful in making future decisions. For example, a coordinating conjunction can have a node of any POSTAG category as its child. But all the children should be of same category. Knowing the POSTAG of one child therefore helps in identifying other children as well.

Experiments 5–7: In experiments 5, 6 and 7, we explored the usefulness of CPOSTAG, FORM, and LEMMA attributes. These features gave small incremental improvements in accuracy; increasing LAS to 56.4% and UAS to 78.5%. It is worth noting in particular that the addition of LEMMA attributes only had a marginal effect on accuracy, given that it is generally believed that this type of information should be beneficial for richly inflected languages.

Experiment 8: In experiment 8, the DEPREL of nodes in the partially formed tree is used. The rightmost child and the left sibling of the rightmost child of *top* were found to be useful. This is because, if we know the dependency label of one of the children, then the search space for other children gets reduced. For example, a verb cannot have more than one k1 or k2. If we know that the parser has assigned k1 to one of its children, then it should use different labels for the other children. The overall effect on parsing accuracy is nevertheless very marginal, bringing LAS to 56.5% and UAS to 78.6%.

⁶ The predecessor of top is the word occurring immediately before top in the input string, as opposed to top-1, which is the word immediately below top in the current stack

Experiment 9: In experiment 9, the FEATS attribute of top and next is used. This gave by far the greatest improvement in accuracy with a huge jump of around 10% in LAS (to 66.3%) and slightly less in UAS (to 84.7%). Recall that FEATS consists of two important morphosyntactic features, namely, case markers (as suffixes or postpositions) and TAM markers. These feature help because (a) case markers are important surface cues that help identify various dependency relations, and (b) there exists a direct mapping between many TAM labels and the nominal case markers because TAMs control the case markers of some nominals. As expected, our experiments show that the parsing decisions are certainly more accurate after using these features. In particular, (a) and (b) are incorporated easily in the parsing process.

In a separate experiment we also added some other morphological features such as gender, number and person for each node. Through these features we expected to capture the agreement in Hindi. The verb agrees in gender, number and person with the highest available karaka. However, incorporating these features did not improve parsing accuracy and hence these features were not used in the final setting. We will have more to say about agreement in section 4.5.

Experiment 10: In experiment 10, finally, we added conjoined features, where the conjunction of POS of next and top and of FEATS and DEPREL of top gave slight improvements. This is because a child-parent pair type can only take certain labels. For example, if the child is a noun and the parent is a verb, then all the dependency labels reflecting noun, adverb and adjective modifications are not relevant. Similarly, as noted earlier, certain case-TAM combinations demand a particular set of labels only. This can be captured by the combination tried in this experiment.

Experiment 10 gave the best results in the cross-validation experiments. The settings from this experiment were used to get the final performance on the test data. Table 4.3 shows the final results.

4.3.4 MST+MaxEnt: MST Settings

MST provides options for two algorithms, projective and non-projective. It also provides options to select features over single edge (order=1) and over pairs of adjacent edges in the tree (order=2). We can also specify k-best parses while training using training-k attribute. algorithm=non-projective, training-k=5 and order=2 gave the best accuracy for MST.

With the original MST parser, the labeled accuracy is very low. This is because only minimal features are used for labeling. Features considering FEATS column of CoNLL format are not used. Vibhakti and TAM markers which are crucial in labeling are not considered by the parser as they are present in FEATS column. We modified the code so that vibhakti and TAM markers which are crucial for labeling are used (Bharati et al., 2008a). We tried to add much richer context features like sibling features, but modification of the code is little complex. For this we have to modify the entire data structure being used for labeling task. Because of this we used MaxEnt for labeling.

4.3.5 MST+MaxEnt: MaxEnt Settings

Unlabeled dependency tree given by MST is given to MaxEnt for labeling. We did several experiments with different options provided by the MaxEnt tool. Best results are obtained when number of iterations is 50. As we have complete tree information we have taken edge features as well as context features. Details of the nodes and the features experimented are

Nodes

- CN: Current node
- PN: Parent node
- RLS: Right-most left sibling
- LRS: Left-most right sibling
- CH: Children

Features

- W: Lexical item
- R: Root form of the word
- P: Part-of-speech tag
- CP: Coarse POS tag
- VT: Vibhakti or TAM markers
- D: Direction of the dependency arc
- SC: Number of siblings
- CC: Number of children
- DS: Difference in positions of node and its parent
- PL: POS list from dependent to tree's root through the dependency path

Table 4.2, shows the best settings using MaxEnt for labelling.

4.4 Experiments and Results

We merged both the training and development data and did 5-fold cross-validation for tuning the parsers. We extracted best settings from the cross validation experiments. These settings are applied on the test data. Table 4.3, shows the cross-validated and test data results using both the parsers. Out of both the parsers, Malt gave better accuracies. Though UAS of MST+MaxEnt is slightly better than Malt, LAS of Malt is far better than MST+MaxEnt.

Features
D, SC, DS, CC
CN: W,R,P,CP,VT,R+P
PN: R,P,CP,VT
RLS: R,CP,VT
LRS: R,CP,VT
CH: R,VT

Table 4.2 MaxEnt Settings (CN: W; represents lexical item (W) of the current node (CN))

<i>System</i>	<i>Cross-validation</i>			<i>Test-set</i>		
	<i>UAS</i>	<i>LAS</i>	<i>LS</i>	<i>UAS</i>	<i>LAS</i>	<i>LS</i>
<i>Malt</i>	84.7	66.5	69.1	90.1	74.5	76.4
<i>MST+MaxEnt</i>	84.9	66.0	68.8	91.3	72.8	75.3

Table 4.3 Results of both Malt and MST+MaxEnt on cross-validated and test data sets.

4.5 Error Analysis

In this section we provide a detailed error analysis on the test data using Malt, which gave the state-of-the-art performance, and suggest possible remedies for problems noted. We note here that other than the reasons mentioned in this section, small treebank size could be another reason for low accuracy of the parser. The training data used for the experiments only had $\sim 28.5k$ words. With recent work on Hindi Treebanking (Bhatt et al., 2009) we expect to get more annotated data in the near future.

Figure 4.2 shows the precision and recall of some important dependency labels in the test data. As the parsing is at chunk level only, the labels in the treebank are syntactico-semantic in nature. Morpho-syntactic features such as case markers and/or TAM labels help in identifying these labels correctly. But lack of nominal postpositions can pose problems. Recall that many case markings in Hindi are optional. Also recall that the verb agrees with the highest available karaka. Since agreement features do not seem to help, if both k1 and k2 lack case markers, k1-k2 disambiguation becomes difficult (considering that word order information cannot help in this disambiguation). In the case of k1 and k2, error rates for instances that lack post-position markers are 60.9% (14/23) and 65.8% (25/38), respectively.

Table 4.2 shows the confusion matrix for some important labels in the test data. As the present information available for disambiguation is not sufficient, we can make use of some semantics to resolve these ambiguities. Bharati et al. (2008a) and Ambati et al. (2009b) have shown that this ambiguity can be reduced using minimal semantics. They used six semantic features: human, non-human, in-animate, time, place and abstract. Using these features they showed that k1-k2 and k7p-k7t ambiguities can be resolved to a great extent. Of course, automatically extracting these semantic features is in itself a

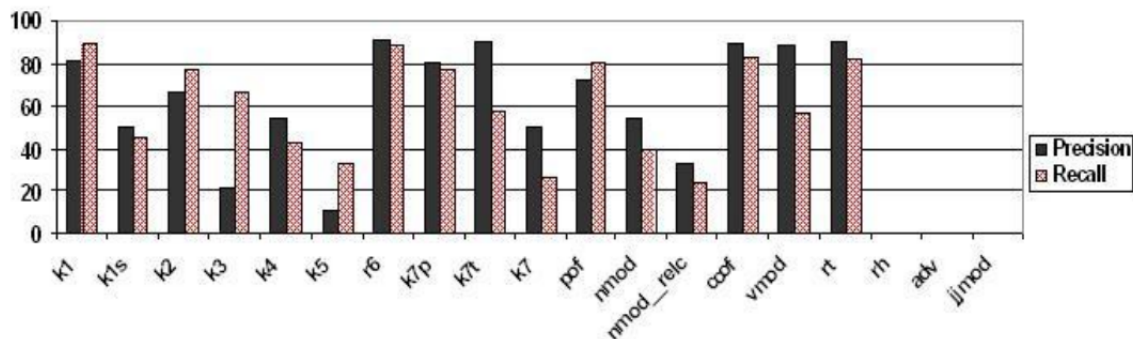


Figure 4.2 Precision and Recall of some important dependency labels

label	Correct	Incorrect							
		k1	k1s	k2	pof	k7p	k7t	k7	others
k1	184	5	3	8	3		1		3
k1s	12	6		1	6				1
k2	126	14		1	7	5			11
pof	54	1	8	4					
k7p	54	3		7			1	2	3
k7t	27	3		3	3		1		10
k7	3	2			2	4			

Table 4.4 Confusion matrix for important labels. The diagonal under ‘Incorrect’ represents attachment errors.

challenging task, although vrelid (2008) has shown that animacy features can be induced automatically from data.

In section 4.3.3, we mentioned that a separate experiment explored the effectiveness of morphological features like gender, number and person. Counter to our intuitions, these features did not improve the overall accuracy. Accuracies on cross-validated data while using these features were less than the best results with 66.2% LAS and 84.6% UAS. Agreement patterns in Hindi are not straightforward. For example, the verb agrees with k2 if the k1 has a post-position; it may also sometimes take the default features. In a passive sentence, the verb agrees only with k2. The agreement problem worsens when there is coordination or when there is a complex verb. It is understandable then that the parser is unable to learn the selective agreement pattern which needs to be followed. Similar problems with agreement features have also been noted by Goldberg and Elhadad (2009).

In the following sections, we analyze the errors due to different constructions and suggest possible remedies.

4.5.1 Simple Sentences

A simple sentence is one that has only one main verb. In these sentences, the root of the dependency tree is the main verb, which is easily identified by the parser. The main problem is the correct identification of the argument structure. Although the attachments are mostly correct, the dependency labels are error prone. Unlike in English and other more configurational languages, one of the main cues that help us identify the arguments is to be found in the nominal postpositions. Also, as noted earlier these postpositions are many times controlled by the TAM labels that appear on the verb. There are four major reasons for label errors in simple sentences: (a) absence of postpositions, (b) ambiguous postpositions, (c) ambiguous TAMs, and (d) inability of the parser to exploit agreement features. For example in (2), ‘raama’ and ‘phala’ are arguments of the verb ‘khaata’. Neither of them has any explicit case marker. This makes it difficult for the parser to identify the correct label for these nodes. In (3a) and (3b) the case marker ‘se’ is ambiguous. It signifies *instrument* in (3b) and *agent* in (3a).

- (2) raama phala khaata hai
 ‘Ram’ ‘fruit’ ‘eat’ ‘is’
 ‘Ram eats a fruit’
- (3) a. raama se phala khaayaa nahi gaya
 ‘Ram’ INST ‘fruit’ ‘eat’ ‘not’ PAST
 ‘Ram could not eat the fruit’
- b. raama chamach se phala khaata hai
 ‘Ram’ ‘spoon’ INST ‘fruit’ ‘eat’ ‘is’
 ‘Ram eats fruit with spoon’

4.5.2 Embedded Clauses

Two major types of embedded constructions involve participles and relative clause constructions. Participles in Hindi are identified through a set of TAM markers. In the case of participle embeddings, a sentence will have more than one verb, i.e., at least one participle and the matrix verb. Both the matrix (finite) verb and the participle can take their own arguments that can be identified via the case-TAM mapping discussed earlier. However, there are certain syntactic constraints that limit the type of arguments a participle can take. There are two sources of errors here: (a) argument sharing, and (b) ambiguous attachment sites.

Some arguments such as place/time nominals can be shared. Shared arguments are assigned to only one verb in the dependency tree. So the task of identifying the shared arguments, if any, and attaching them to the correct parent is a complex task. Note that the dependency labels can be identified based on the morphosyntactic features. The task becomes more complex if there is more than one participle in a sentence. 12 out of 130 instances (9.23%) of shared arguments has an incorrect attachment.

Many participles are ambiguous and making the correct attachment choice is difficult. Similar participles, depending on the context, can behave as adverbials and attach to a verb, or can behave as adjectives and attach to a noun. Take (4) as a case in point.

- (4) maine daurte hue kutte ko dekhaa
 ‘I’-ERG (while) ‘running’ ‘dog’ ACC ‘saw’

In (4) based on how one interprets ‘daurte hue’, one gets either the reading that ‘I saw a running dog’ or that ‘I saw a dog while running’. In case of the adjectival participle construction (VJJ), 2 out of 3 errors are due to wrong attachment.

4.5.3 Coordination

Coordination poses problems as it often gives rise to long-distance dependencies. Moreover, the treebank annotation treats the coordinating conjunction as the head of the coordinated structure. Therefore, a coordinating conjunction can potentially become the root of the entire dependency tree. This is similar to Prague style dependency annotation (Hajičová, 1998). Coordinating conjunctions pose additional problems in such a scenario as they can appear as the child of different heads. A coordinating conjunction takes children of similar POS category, but the parent of the conjunction depends on the type of the children.

- (5) a. raama aur shyaama ne khaana khaayaa
 ‘Ram’ ‘and’ ‘Shyam’ ERG ‘food’ ‘ate’
 ‘Ram and Shyam ate the food.’
 b. raama ne khaanaa khaayaa aur paanii piyaa
 ‘Ram’ ERG ‘food’ ‘ate’ ‘and’ ‘water’ ‘drank’
 ‘Ram ate food and drank water.’

In (5a), ‘raama’ and ‘shyaama’ are children of the coordinating conjunction ‘aur’, which gets attached to the main verb ‘khaayaa’ with the label *kI*. In effect, syntactically ‘aur’ becomes the argument of the main verb. In (5b), however, the verbs ‘khaayaa’ and ‘piyaa’ are the children of ‘aur’. In this case, ‘aur’ becomes the root of the sentence. Identifying the nature of the conjunction and its children becomes a challenging task for the parser. Note that the number of children that a coordinating conjunction can take is not fixed either. The parser could identify the correct head of the conjunctions with an accuracy of 75.7% and the correct children with an accuracy of 85.7%.

The nature of the conjunction will also affect the dependency relation it has with its head. For example, if the children are nouns, then the conjunction behaves as a noun and can potentially be an argument of a verb. By contrast, if the children are finite verbs, then it behaves as a finite verb and can become the root of the dependency tree. Unlike nouns and verbs, however, conjunctions do not have morphological features. So a child-to-head feature percolation should help make a coordinating node more transparent. For example, in (5a) the Ergative case ‘ne’ is a strong cue for the dependency label

k1. If we copy this information from one of its children (here ‘shyaama’) to the conjunct, then the parser can possibly make use of this information.

4.5.4 Complex Predicates

Complex predicates are formed by combining a noun or an adjective with a verbalizer ‘kar’ or ‘ho’. For instance, in ‘taariif karanaa’ (to praise), ‘taariif’ (praise) is a noun and ‘karanaa’ (to do) is a verb. Together they form the main verb. Complex predicates are highly productive in Hindi. Combination of the light verb and the noun/adjective is dependent on not only syntax but also semantics and therefore its automatic identification is not always straightforward (Butt, 1995). A noun-verb complex predicate in the treebank is linked via the dependency label *pof*. The parser makes mistakes in identifying *pof* or misclassifies other labels as *pof*. In particular, the confusion is with *k2* and *k1s* which are object/theme and noun complements of *k1*, respectively. These labels share similar contextual features like the nominal element in the verb complex. Table 4.4 includes the confusion matrix for *pof* errors.

4.5.5 Non-Projectivity

As noted earlier, MaltParser’s arc-eager parsing algorithm can be combined with the pseudo-projective parsing techniques proposed in Nivre and Nilsson (2005), which potentially helps in identifying non-projective arcs. The Hindi treebank has 14% non-projective arcs (Mannem et al., 2009). In the test set, there were a total of 11 non-projective arcs, but the parser did not find any of them. This is consistent with earlier results showing that pseudo-projective parsing has high precision but low recall, especially when the percentage of non-projective relations is small (Nilsson et al., 2007).

Non-projectivity has proven to be one of the major problems in dependency parsing, especially for free word-order languages. In Hindi, the majority of non-projective arcs are inter-clausal (Mannem et al., 2009), involving conjunctions and relative clauses. There have been some attempts at handling inter-clausal non-projectivity in Hindi. Husain et al. (2009) proposed a two-stage approach that can handle some of the inter-clausal non-projective structures. We can exploit those methods to handle non-projective cases.

4.5.6 Long-Distance Dependencies

Previous results on parsing other languages have shown that MaltParser has lower accuracy on long-distance dependencies. Our results confirm this. Errors in the case of relative clauses and coordination can mainly be explained in this way. For example, there are 8 instances of relative clauses in the test data. The system could identify only 2 of them correctly. These two are at a distance of 1 from its parent. For the remaining 6 instances the distance to the parent of the relative clause ranges from 4 to 12.

Figure 4.3 shows how parser performance decreases with increasing distance between the head and the dependent. Recently, Husain et al. (2009) have proposed a two-stage setup to parse inter-clausal and intra-clausal dependencies separately. They have shown that most long distance relations are inter-clausal, and therefore, using such a clause motivated parsing setup helps in maximizing both short distance and long distance dependency accuracy. In a similar spirit, Gadde et al. (2010) showed that using clausal features helps in identifying long distance dependencies. They have shown that providing clause information in the form of clause boundaries and clausal heads can help a parser make better predictions about long distance dependencies.

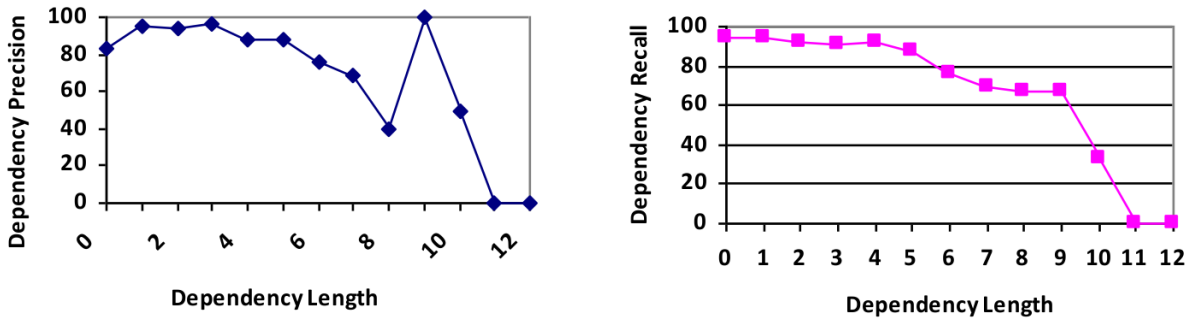


Figure 4.3 Dependency arc precision/recall relative to dependency length, where the length of a dependency from w_i to w_j is $|i - j|$ and roots are assumed to have distance 0 to their head

4.6 Summary

In this chapter we have presented our experiments on building a dependency parser for Hindi using Malt and MST. We did step by step analysis of the importance of different linguistic features in data-driven parsing of Hindi. Our main finding is that the combination of case markers on nominals with TAM markers on verbs is crucially important for syntactic disambiguation, while the inclusion of features such as person, number gender that help in agreement has not yet resulted in any improvement. We have also presented a detailed error analysis and discussed possible techniques targeting different error classes. We plan to use these techniques to improve our results in the near future.

Chapter 5

Linguistic Constraints in Dependency Parsing

5.1 Introduction

Due to the availability of dependency treebanks, there are several recent attempts at building dependency parsers. Two CoNLL shared tasks (Buchholz and Marsi, 2006; Nivre et al., 2007a) were held aiming at building state-of-the-art dependency parsers for different languages. Recently in NLP Tools Contest in ICON-2009 (Husain, 2009), rule-based, constraint based, statistical and hybrid approaches were explored towards building dependency parsers for three Indian languages namely, Telugu, Hindi and Bangla. In all these efforts, state-of-the-art accuracies are obtained by two data-driven parsers, namely, Malt (Nivre et al., 2007b) and MST (McDonald et al., 2006). The major limitation of both these parsers is that they won't take linguistic constraints into account explicitly. But, in real-world applications of the parsers, some basic linguistic constraints are very useful. If we can make these parsers handle linguistic constraints also, then they become very useful in real-world applications.

In this chapter, we see how to incorporate linguistic constraints in statistical dependency parsers. We consider a simple constraint that a verb should not have multiple subjects/direct objects as its children. We motivate the need of this linguistic constraint taking a machine translation system which uses dependency parser output as an example application. We propose two approaches to handle this case. We evaluate our approaches on the state-of-the-art dependency parsers for Hindi and Czech and analyze the results.

5.2 Motivation

In this section we take Machine Translation (MT) systems that use dependency parser output as an example and explain the need of linguistic constraints. We take a simple constraint that a verb should not have multiple subjects/direct objects as its children in the dependency tree. Indian Language to Indian

Language Machine Translation System¹ is one such MT system which uses dependency parser output. In this system the general framework has three major components.

1. dependency analysis of the source sentence
2. transfer from source dependency tree to target dependency tree, and
3. sentence generation from the target dependency tree

In the transfer part several rules are framed based on the source language dependency tree. For instance, for Telugu to Hindi MT system, based on the dependency labels of the Telugu sentence, post-positions markers that need to be added to the Hindi words are decided. Consider the following example,

- (1) Telugu: raamu oka pamdu tinnaadu
 ‘Ramu’ ‘one’ ‘fruit’ ‘ate’
 Hindi: raamu ne eka phala khaayaa
 ‘Ramu’ ‘ERG’ ‘one’ ‘fruit’ ‘ate’
 English: Ramu ate a fruit.

In the above Telugu sentence, ‘raamu’ is the karta karaka (roughly subject) of the verb ‘tinnaadu’. While translating this sentence to Hindi, the post-position marker ‘ne’ is added to the karta karaka. If the dependency parser marks two karta karakas, both the words will have ‘ne’ marker. This affects the comprehensibility. If we can avoid such instances, then the output of the MT system will be improved. Figure 5.1 describes the Telugu to Hindi MT system framework for the above mentioned example.

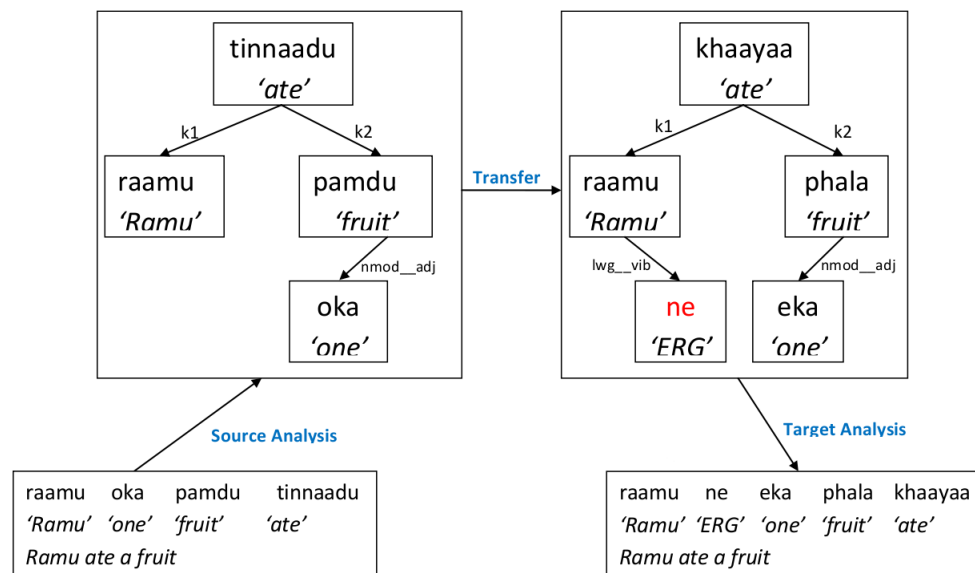


Figure 5.1 Telugu to Hindi MT system

¹<http://sampark.iiit.ac.in/>

This problem is not due to morphological richness or free-word-order nature of the target language. Consider an example of free-word-order language to fixed-word-order language MT system like Hindi to English MT system. The dependency labels help in identifying the position of the word in the target sentence. Consider the example sentences given below.

(2a) raama seba khaatha hai
 'Ram' 'apple' 'eats' 'is'
 'Ram eats an apple'

(2b) seba raama khaatha hai
 'apple' 'Ram' 'eats' 'is'
 'Ram eats an apple'

Though the source sentence is different, the target sentence is same. Even though the source sentences are different, the dependency tree is same for both the sentences. In both the cases, 'raama' is the karta karaka (roughly subject) and 'seba' is the karma karaka (roughly object) of the verb 'khaatha'. This information helps in getting the correct translation. If the parser for the source sentence assigns the label 'karta karaka (roughly subject)' to both 'raama' and 'seba', the MT system can not give the correct output. Figure 5.2 describes the Hindi to English MT system for the above mentioned example.

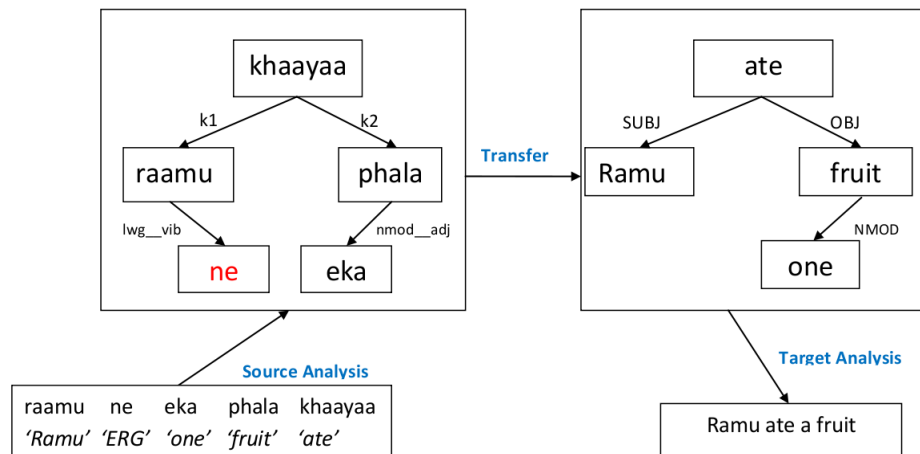


Figure 5.2 Hindi to English MT system

There were some attempts at handling these kind of linguistic constraints using integer programming approaches (Riedel et al., 2006; Bharati et al., 2008b). In these approaches dependency parsing is formulated as solving an integer program as McDonald et al. (2006) has formulated dependency parsing as Spanning Tree problem. All the linguistic constraints are encoded as constraints while solving the integer program. In other words, all the parses that violate these constraints are removed from the solution list. The parse which satisfies all the constraints is considered as the dependency tree for the

sentence. In the following section, we describe two new approaches to avoid multiple subjects/direct objects for a verb.

5.3 Approaches

In this section, we describe the two different approaches for avoiding the cases of a verb having multiple subjects/objects as its children in the dependency tree.

5.3.1 Naive Approach (NA)

In this approach we first run a parser on the input sentence. Instead of first best dependency label, we extract the k-best labels for each token in the sentence. For each verb in the sentence, we check if there are multiple children with the dependency label ‘subject’. If there are any such cases, we extract the list of all the children with label ‘subject’. we find the node in this list which appears left most in the sentence with respect to other nodes. We assign ‘subject’ to this node. For the rest of the nodes in this list we assign the second best label and remove the first best label from their respective k-best list of labels. We check recursively, till all such instances are avoided. We repeat the same procedure for ‘direct object’.

Main criterion to avoid multiple subjects/direct objects in this approach is position of the node in the sentence. Consider the example sentence of (2a),

Suppose the parser assigns the label ‘k1 (roughly subject)’ to both the nouns, ‘raama’ and ‘seba’. Then naive approach assigns the label k1 (roughly subject) to ‘raama’ and second best label to ‘seba’ as ‘raama’ precedes ‘seba’. In this manner we can avoid a verb having multiple children with dependency labels subject/direct object.

Limitation to this approach is word-order. The algorithm described here works well for fixed word order languages. For example, consider a language with fixed word order like English. English is a SVO (Subject, Verb, Object) language. Subject always occurs before the object. So, if a verb has multiple subjects, based on position we can say that the node that occurs first will be the subject. But if we consider a free-word order language like Hindi, this approach wouldn’t work always.

Consider (2a) and (2b). In both these examples, ‘raama’ is the k1 (roughly subject) of the verb ‘khaatha’ and ‘seba’ is the k2 (roughly direct object) of the verb ‘khaatha’. The only difference in these two sentences is the word order. In (2a), k1 (roughly subject) precedes k2. Whereas in (2b), k2 precedes k1. Suppose the parser identifies both ‘raama’ and ‘seba’ as k1. NA can correctly identify ‘raama’ as the k1 in case of (2a). But in case of (2b), ‘seba’ is identified as the k1. To handle these kind of instances, we proposed a probabilistic approach.

5.3.2 Probabilistic Approach (PA)

The probabilistic approach is similar to naive approach except that the main criterion to avoid multiple subjects/direct objects in this approach is probability of the node having a particular label. Whereas in naive approach, position of the node is the main criterion to avoid multiple subjects/direct objects. In this approach, for each node in the sentence, we extract the k-best labels along with their probabilities. Similar to NA, we first check for each verb if there are multiple children with the dependency label 'subject'. If there are any such cases, we extract the list of all the children with label 'subject'. We find the node in this list which has the highest probability value. We assign 'subject' to this node. For the rest of the nodes in this list we assign the second best label and remove the first best label from their respective k-best list of labels. We check recursively, till all such instances are avoided. We repeat the same procedure for 'direct object'.

Consider (2a) and (2b). Suppose the parser identifies both 'raama' and 'seba' as k1 (roughly subjects). Probability of 'raama' being a k1 will be more than 'seba' being a k1. So, the probabilistic approach correctly marks 'raama' as k1 in both (2a) and (2b). But, NA couldn't identify 'raama' as k1 in (2b).

Figure 5.3, sketches the steps involved in both Navie Approach and Probabilistic Approach.

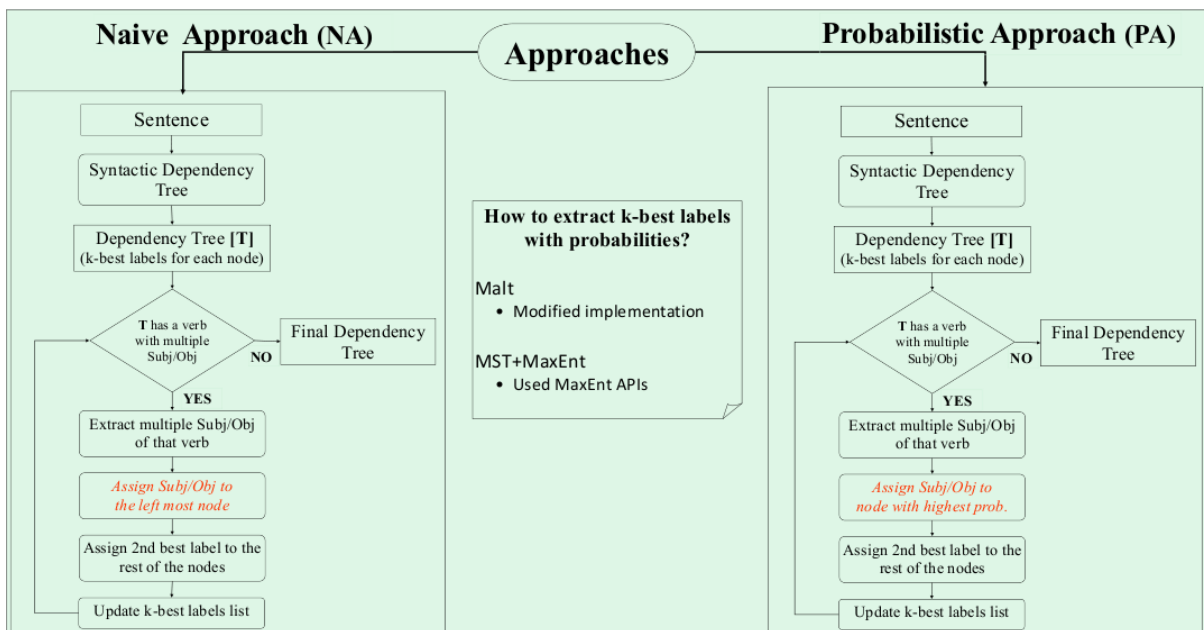


Figure 5.3 Approaches: Naive Approach and Probabilistic Approach

	Total Instances
Malt	39
MST + MaxEnt	51

Table 5.1 No. of instances of multiple subjects/objects in the output of the state-of-the-art Hindi parser.

5.4 Experiments

We evaluate our approaches on the state-of-the-art parsers for two languages namely, Hindi and Czech. First we calculate the instances of multiple subjects/objects in the output of the state-of-the-art parsers for these two languages. Then we apply our approaches and analyze the results. We have considered ICON09 Toolscontest data for Hindi and CoNLL-2007 Shared Task data for Czech.

5.4.1 Hindi

For Hindi, as we have seen, dependency annotation is done using Paninian framework (Begum et al., 2008; Bharati et al., 1995). So, in Hindi, the roughly equivalent labels for subject and direct object are ‘karta (k1)’ and ‘karma (k2)’. ‘karta’ and ‘karma’ are syntactico-semantic labels which have some properties of both grammatical roles and thematic roles. k1 behaves similar to subject and agent. k2 behaves similar to direct object and patient (Bharati et al., 1995; Bharati et al., 2009b). Thus we consider only k1 and k2 labels which are roughly equivalent to subject and direct object. Annotation scheme is such that there wouldn’t be multiple k1/k2 for a verb in any case (Bharati et al., 2009b). For example, even in case of coordination, coordinating conjunction is the head and conjuncts are children of the coordinating conjunction. The coordinating conjunction is attached to the verb with k1/k2 label and the conjuncts get attached to the coordinating conjunction with a dependency label ‘ccof’.

As we have seen in the previous chapter, the state-of-the-art parsing accuracy for Hindi are obtained using Malt and MST+MaxEnt (Ambati et al., 2009a). We consider this as the baseline. We analyzed the outputs of Malt and MST+MaxEnt. In the output of Malt, there are 39 instances of multiple subjects/direct objects. There are 51 such instances in the output of MST+MaxEnt. Malt is good at short distance labeling and MST is good at long distance labeling (McDonald and Nivre, 2007). As ‘k1’ and ‘k2’ are short distance labels, Malt was able to predict these labels more accurately than MST. Because of this, the output of MST has higher number of instances of multiple subjects/direct objects than Malt.

Both the parsers output first best label for each node in the sentence. In case of Malt, we modified the implementation to extract all the possible dependency labels with their scores. As Malt uses libsvm for learning, we couldn’t able to get the probabilities. Though interpreting the scores provided by libsvm as probabilities is not the correct way, that is the only option currently available with Malt. In case of MST+MaxEnt, labeling is performed by MaxEnt. We used a java version of MaxEnt ² to extract all

²<http://maxent.sourceforge.net/>

	<i>Malt</i>			<i>MST+MaxEnt</i>		
	<i>UAS</i>	<i>LAS</i>	<i>LS</i>	<i>UAS</i>	<i>LAS</i>	<i>LS</i>
<i>Baseline</i>	90.14	74.48	76.38	91.26	72.75	75.26
<i>NA</i>	90.14	74.57	76.38	91.26	72.84	75.26
<i>PA</i>	90.14	74.74	76.56	91.26	73.36	75.87

Table 5.2 Comparison of NA and PA with previous best results for Hindi.

possible tags with their scores. We applied both the naive and probabilistic approaches to avoid multiple subjects/direct objects. We evaluated our experiments based on unlabeled attachment score (UAS), labeled attachment score (LAS) and labeled score (LS) (Nivre et al., 2007a). Results are presented in Table 5.2.

As expected, PA performs better than NA. With PA we got an improvement of 0.26% in LAS over the previous best results for Malt. In case of MST+MaxEnt we got an improvement of 0.61% in LAS over the previous best results. Note that in case of MST+MaxEnt, the slight difference between state-of-the-art results of (Ambati et al., 2009a) and our baseline accuracy is due different MaxEnt package used.

Improvement in case of MST+MaxEnt is greater than that of Malt. One reason is because of more number of instances of multiple subjects/direct objects in case of MST+MaxEnt. Other reason is use of probabilities in case MST+MaxEnt. Whereas in case of Malt, we interpreted the scores as probabilities which is not a good way to do. But, in case of Malt, that is the only option available.

5.4.2 Czech

In case of Czech, we replicated the experiments of (Hall et al., 2007) using latest version of Malt (version 1.3.1) and analyzed the output. We consider this as the baseline. The minor variation of the baseline results from the results of CoNLL-2007 shared task is due to different version Malt parser being used. Due to practical reasons we couldn't use the older version. In the output of Malt, there are 38 instances of multiple subjects/direct objects out of 286 sentences in the testing data. In case of Czech, the equivalent labels for subject and direct object are 'agent' and 'theme'.

Czech is a free-word-order language similar to Hindi. So as expected, PA performed better than NA. Interestingly, accuracy of PA is lower than the baseline. Main reason for this is scores of libsvm of Malt. We explain the reason for this using the following example, consider a verb 'V' has two children 'C1' and 'C2' with dependency label subject. Assume that the label for 'C1' is subject and the label of 'C2' is direct object in the gold-data. As the parser marked 'C1' with subject, this adds to the accuracy of the parser. While avoiding multiple subjects, if 'C1' is marked as subject, then the accuracy doesn't drop. If 'C2' is marked as direct object then the accuracy increases. But, if 'C2' is marked as subject and 'C1' is marked as direct object then the accuracy drops. This could happen if probability of 'C1' having

	<i>UAS</i>	<i>LAS</i>	<i>LS</i>
<i>Baseline</i>	82.92	76.32	83.69
<i>NA</i>	82.92	75.92	83.35
<i>PA</i>	82.92	75.97	83.40

Table 5.3 Comparison of NA and PA with previous best results for Czech.

subject as label is lower than ‘C1’ having subject as the label. This is because of two reasons, (a) parser itself wrongly predicted the probabilities, and (b) parser predicted correctly, but due to the limitation of libsvm, we couldn’t get the scores correctly.

5.5 Discussion and Future Work

Results show that the probabilistic approach performs consistently better than the naive approach. For Hindi, we were able to achieve an improvement 0.26% and 0.61% in LAS over the previous best results using Malt and MST+MaxEnt respectively. We were not able to achieve any improvement in case of Czech due to the limitation of libsvm learner used in Malt.

We plan to evaluate our approaches on all the data-sets of CoNLL-X and CoNLL-2007 shared tasks using Malt. Settings of MST parser are available only for CoNLL-X shared task data sets. So, we plan to evaluate our approaches on CoNLL-X shared task data using MST also. Malt has the limitation for extracting probabilities due to libsvm learner. Latest version of Malt (version 1.3.1) provides option for liblinear learner also. Liblinear provides option for extracting probabilities. So we can also use liblinear learning algorithm for Malt and explore the usefulness of our approaches. Currently, we are handling only two labels, subject and direct object. Apart from subject and direct object there can be other labels for which multiple instances for a single verb is not valid. We can extend our approaches to handle such labels also. We tried to incorporate one simple linguistic constraint in the statistical dependency parsers. We can also explore the ways of incorporating other useful linguistic constraints.

5.6 Conclusion

Statistical systems with high accuracy are very useful in practical real-world applications. If these systems can capture basic linguistic information, then the usefulness of the statistical system improves a lot. In chapter, we presented a new method of incorporating linguistic constraints into the statistical dependency parsers. We took a simple constraint that a verb should not have multiple subjects/direct objects as its children. We proposed two approaches, one based on position and the other based on probabilities to handle this. We evaluated our approaches on state-of-the-art dependency parsers for Hindi and Czech.

Chapter 6

Hindi Dependency Parsing: Down to Word level

In the previous two chapters (Chapters 4 and 5), we explored different features and approaches towards building a state-of-the-art dependency parser for Hindi. All these efforts, however, were finding inter-chunk dependency relations, given gold-standard POS and chunk tags. But, there isn't any attempt to go to word level parsing for Hindi sentences, that too using automatic tags/features rather than gold-standard ones. In this chapter, we describe our experiments for sentence parsing going down to word level rather than only chunk level for Hindi, which is the first known attempt in this area.

6.1 Introduction

In this chapter we systematically explore various strategies to incorporate local morphosyntactic features in word level dependency parsing for Hindi. These features are obtained using a shallow parser. We conducted experiments with two data-driven parsers, MaltParser (Nivre et al., 2007b) and MST-Parser (McDonald et al., 2006). We first explore which information provided by the shallow parser is most beneficial and show that local morphosyntactic features in the form of chunk type, head/non-head information, chunk boundary information, distance to the end of the chunk and suffix concatenation are very crucial in Hindi dependency parsing. We then investigate the best way to incorporate this information during dependency parsing. All the experiments were done on a part of multi-layered and multi-representational Hindi Treebank (Bhatt et al., 2009)¹.

The shallow parser performs three tasks, (a) it gives the POS tags for each lexical item, (b) provides morphological features for each lexical item, and (c) performs chunking. A chunk is a minimal (non-recursive) phrase consisting of correlated, inseparable words/entities, such that the intra-chunk dependencies are not distorted (Bharati et al., 2006). Together, a group of lexical items with some POS tag and morphological features within a chunk can be utilized to automatically compute local morphosyntactic information. For example, such information can represent the postposition/case-marking in the case of noun chunks, or it may represent the tense, aspect and modality (TAM) information in the case of verb chunks. In the experiments conducted, such local information is automatically computed

¹This Treebank is still under development. There are currently 27k tokens with complete sentence level annotation

and incorporated as a feature to the head of a chunk. In general, local morphosyntactic features correspond to all the parsing relevant local linguistic features that can be utilized using the notion of chunk. Previously, there have been some attempts at using chunk information in dependency parsing. Attardi and DellOrletta (2008) used chunking information in parsing English. They got an increase of 0.35% in labeled attachment accuracy and 0.47% in unlabeled attachment accuracy over the state-of-the-art dependency parser.

Among the three components (a-c, above), the parsing accuracy obtained using the POS feature is taken as baseline. We follow this by experiments where we explore how each of morph and chunk features help in improving dependency parsing accuracy. In particular, we find that local morphosyntactic features are the most crucial. In all the parsing experiments, at each step we explore all possible features and extract the best set of features. Best features of one experiment are used when we go to the next set of experiments. For example, when we explore the effect of chunk information, all the relevant morph information from previous set of experiments is taken into account.

6.2 Getting the best linguistic features

As mentioned earlier, a shallow parser consists of three main components, (a) POS tagger, (b) morphological analyzer and (c) chunker. In this section we systematically explore what is the effect of each of these components. We'll see in section 6.2.3 that the best features of a-c can be used to compute local morphosyntactic features that, as the results show, are extremely useful.

6.2.1 Using POS as feature (PaF):

In this experiment we only use the POS tag information of individual words during dependency parsing. First a raw sentence is POS-tagged. This POS-tagged sentence is then given to a parser to predict the dependency relations. Figure 6.1, shows the steps involved in this approach for (1).

(1) raama ne aka seba khaayaa
 'Ram' ERG 'one' 'apple' 'ate'
 Ramu ate an apple.

In (1) above, 'NN', 'PSP', 'QC', 'NN' and 'VM' are the POS tags ² for *raama*, *ne*, *eka*, *seba* and *khaayaa* respectively. This information is provided as a feature to the parser. The result of this experiment forms our baseline accuracy.

²NN: Common noun, PSP: Post position, QC: Cardinal, VM: Verb. A list of complete POS tags can be found here: <http://ltrc.iit.ac.in/MachineTrans/research/tb/POS-Tag-List.pdf>. The POS/chunk tag scheme followed in the Treebank is described in Bharati et al. (2006).

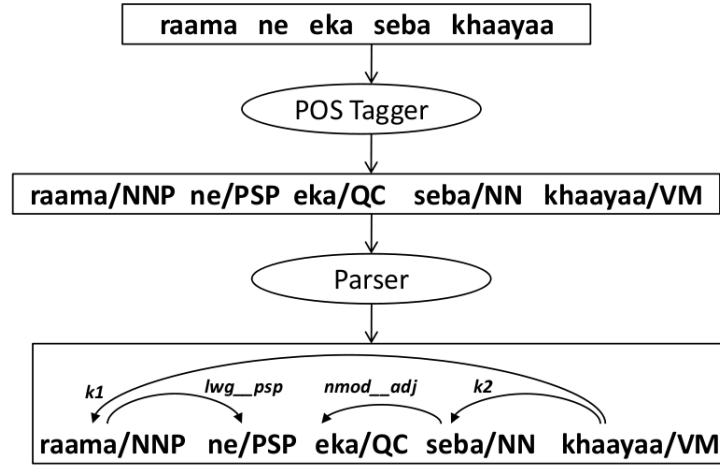


Figure 6.1 Dependency parsing using only POS information from a shallow parser

6.2.2 Using Morph as feature (MaF):

In addition to POS information, in this experiment we also use the morph information for each token. This morphological information is provided as a feature to the parser. Morph has the following information

- Root: Root form of the word
- Category: Course grained POS
- Gender: Masculine/Feminine/Neuter
- Number: Singular/Plural
- Person: First/Second/Third person
- Case: Oblique/Direct case
- Suffix: Suffix of the word

Take *raama* in (1), its morph information comprises of root = ‘raama’, category = ‘noun’ gender = ‘masculine’, number = ‘singular’, person = ‘third’, case = ‘direct’, suffix = ‘0’. Similarly, *khaayaa* (‘ate’) has the following morph information. root = ‘khaa’, category = ‘verb’ gender = ‘masculine’, number = ‘singular’, person = ‘third’, case = ‘direct’, suffix = ‘yaa’.

Through a series of experiments, the most crucial morph features were selected. Root, case and suffix turn out to be the most important features. Results are discussed in the experiments section.

6.2.3 Using local morphosyntax as feature (LMSaF)

Along with POS and the most useful morph features (root, case and suffix), in this experiment we also use local morphosyntactic features that reflect various chunk level information. These features are:

- Type of the chunk
- Head/non-head of the chunk
- Chunk boundary information
- Distance to the end of the chunk
- Suffix concatenation

In (1), there are two noun chunks and one verb chunk. *raama* and *seba* are the heads of the noun chunks. *khaayaa* is the head of the verb chunk. We follow standard IOB³ notation for chunk boundary. *raama*, *eka* and *khaayaa* are at the beginning (B) of their respective chunks. *ne* and *seba* are inside (I) their respective chunks. *raama* is at distance 1 from the end of the chunk and *ne* is at a distance 0 from the end of the chunk.

Once we have a chunk and morph feature like suffix, we can perform suffix concatenation automatically. A group of lexical items with some POS tags and suffix information within a chunk can be utilized to automatically compute this feature. This feature can, for example, represent the postposition/case-marking in the case of noun chunk, or it may represent the tense, aspect and modality (TAM) information in the case of verb chunks. Note that, this feature becomes part of the lexical item that is the head of a chunk. Take (2) as a case in point:

- (2) [NP *raama*/NNP *ne*/PSP] [NP *seba*/NN] [VGF *khaa*/VM *liyaa*/VAUX]
 ‘Ram’ ERG ‘apple’ ‘eat’ ‘PRFT’
 Ram ate an apple.

The suffix concatenation feature for *khaa*, which is the head of the VGF chunk, will be ‘0+yaa’ and is formed by concatenating the suffix of the main verb with that of its auxiliary. Similarly, the suffix concatenation feature for *raama*, which is head of the NP chunk, will be ‘0+ne’. This feature turns out to be very important. This is because in Hindi (and many other Indian languages) there is a direct correlation between the TAM markers and the case that appears on some nominals (Bharati et al., 1995). In (2), for example, *khaa liyaa* together gives the past perfective aspect for the verb *khaanaa* ‘to eat’. Since, Hindi is split ergative, the subject of the transitive verb takes an ergative case marker when the verb is past perfective. Similar correlation between the case markers and TAM exist in many other cases. Figure 6.2, shows the approach for the example sentence (1).

³Inside, Outside, Beginning of the chunk

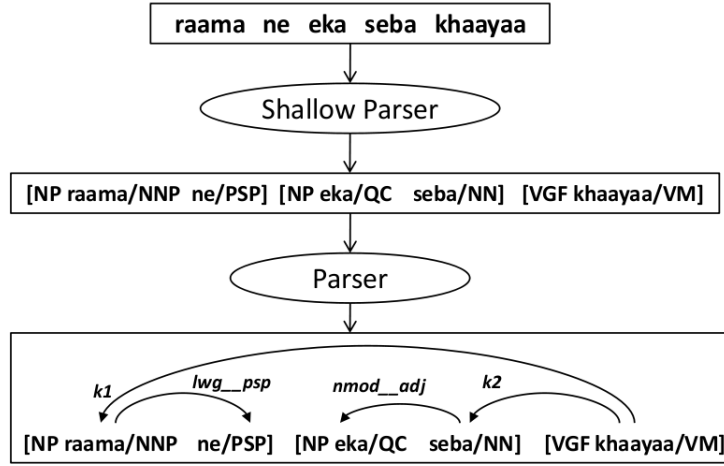


Figure 6.2 Dependency parsing using shallow parser information

6.2.4 An alternative approach to use best features: A 2-stage setup (2stage)

So far we have been using various information such as POS, chunk, etc. as features. Rather than using them as features and doing parsing at one go, we can alternatively follow a 2-stage setup. In particular, we divide the task of parsing into:

- Intra-chunk dependency parsing
- Inter-chunk dependency parsing

We still use POS, best morphological features (case, suffix, root) information as regular features during parsing. But unlike LMSaF mentioned in previous section, where we gave local morphosyntactic information as a feature, we divided the task of parsing into sub-tasks. A similar approach was also proposed by Bharati et al. (2009a). During intra-chunk dependency parsing, we try to find the dependency relations of the words within a chunk. Following which, chunk heads of each chunk within a sentence are extracted. On these chunk heads we run an inter-chunk dependency parser. For each chunk head, in addition to POS tag, useful morphological features, any useful intra-chunk information in the form of lexical item, suffix concatenation, dependency relation are also given as a feature. Among the intra-chunk information used for inter-chunk parsing, only suffix concatenation turned out to be very useful. Intra-chunk dependency relations didn't give any improvement in the accuracy.

Figure 6.3 shows the steps involved in this approach for (1). There are two noun chunks and one verb chunk in this sentence. *raama* and *seba* are the heads of the noun chunks. *khaaya* is the head of the verb chunk. The intra-chunk parser attaches *ne* to *raama* and *eka* to *seba* with dependency labels 'lwg_psp' and 'nmod_adj'⁴ respectively. Heads of each chunk along with its POS, morphological features, local

⁴nmod_adj is an intra-chunk label for quantifier-noun modification. lwg_psp is the label for post-position marker. Details of the labels can be seen in the intra-chunk guidelines (Bharati et al., 2009b).

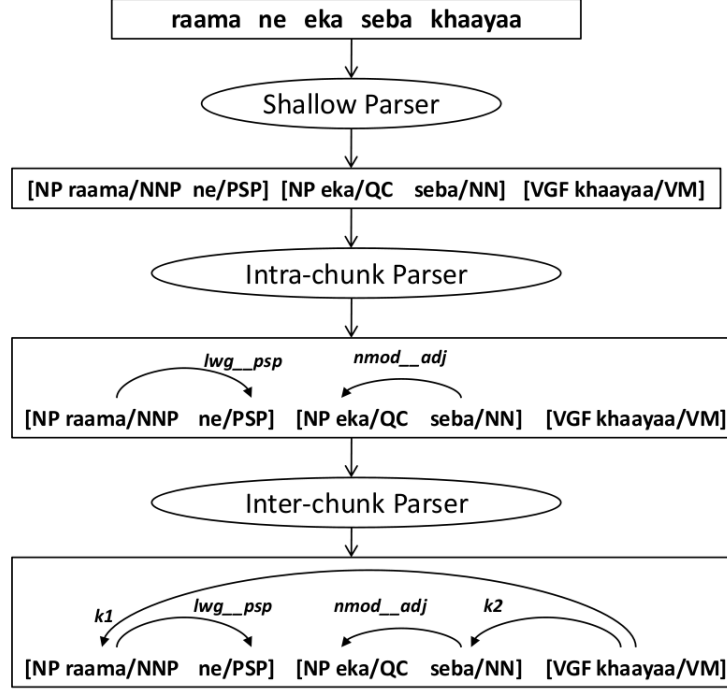


Figure 6.3 Dependency parsing using only POS information from a shallow parser

morphosyntactic features and intra-chunk features are extracted and given to inter-chunk parser. Using this information the inter-chunk dependency parser marks the dependency relations between chunk heads. *khaaya* becomes the root of the dependency tree. *raama* and *seba* are attached to *khaaya* with dependency labels ‘k1’ and ‘k2’⁵ respectively.

6.3 Experimental Setup

In this section we describe the data and the parser settings used for our experiments.

6.3.1 Data

For our experiments we took 1228 dependency annotated sentences (27k tokens), which have complete sentence level annotation from the new multi-layered and multi-representational Hindi Treebank (Bhatt et al., 2009). This treebank is still under development. Average length of these sentences is 22 tokens/sentence and 10 chunks/sentence. We divided the data into two sets, 1000 sentences for training and 228 sentences for testing.

⁵k1 (karta) and k2 (karma) are syntactico-semantic labels which have some properties of both grammatical roles and thematic roles. k1 behaves similar to subject and agent. k2 behaves similar to object and patient (Bharati et al., 1995; Vaidya et al., 2009). For complete tagset, see (Bharati et al., 2009b)

	<i>Malt</i>						<i>MST+MaxEnt</i>					
	<i>Cross-validation</i>			<i>Test-set</i>			<i>Cross-validation</i>			<i>Test-set</i>		
	<i>UAS</i>	<i>LAS</i>	<i>LS</i>	<i>UAS</i>	<i>LAS</i>	<i>LS</i>	<i>UAS</i>	<i>LAS</i>	<i>LS</i>	<i>UAS</i>	<i>LAS</i>	<i>LS</i>
<i>PaF</i>	89.4	78.2	80.5	90.4	80.1	82.4	86.3	75.1	77.9	87.9	77.0	79.3
<i>MaF</i>	89.6	80.5	83.1	90.4	81.7	84.1	89.1	79.2	82.5	90.0	80.9	83.9
<i>LMSaF</i>	91.5	82.7	84.7	91.8	84.0	86.2	90.8	79.8	82.0	92.0	81.8	83.8
<i>2stage</i>	91.8	83.3	85.3	92.4	84.4	86.3	92.1	82.2	84.3	92.7	84.0	86.2

Table 6.1 Results of all the four approaches using gold-standard shallow parser information.

6.3.2 Parsers and settings

All experiments were performed using two data-driven parsers, MaltParser⁶, and MSTParser⁷.

Similar to previous Chapters, we used Malt and MST+MaxEnt for our experiments. As the training data size is small we did 5-fold cross validation on the training data for tuning the parameters of the parsers and for feature selection. Best settings obtained using cross-validated data are applied on test set. We present the results both on cross validated data and on test data.

For the Malt Parser, arc-eager algorithm gave better performance over others in all the approaches. Libsvm consistently gave better performance over liblinear in all the experiments. For SVM settings, we tried out different combinations of best SVM settings of the same parser on different languages in CoNLL-2007 shared task (Hall et al., 2007) and applied the best settings. For feature model, apart from trying best feature settings of the same parser on different languages in CoNLL-2007 shared task (Hall et al., 2007), we also tried out different combinations of linguistically intuitive features and applied the best feature model. The best feature model is same as the feature model used in Chapter 4, which is the best performing system in the ICON-2009 NLP Tools Contest (Husain, 2009).

For the MSTParser, non-projective algorithm, order=2 and training-k=5 gave best results in all the approaches. For the MaxEnt, apart from some general useful features, we experimented considering different combinations of features of node, parent, siblings, and children of the node. Note that the parser settings used in these experiments are similar to the best system obtained in Chapter 4 (Ambati et al., 2009a)

6.4 Results and Analysis

All the experiments discussed were performed considering both gold-standard shallow parser information and automatic shallow parser⁸ information. Automatic shallow parser uses a rule based system for morph analysis, a CRF+TBL based POS-tagger and chunker. The tagger and chunker are 93% and

⁶Malt Version 1.3.1

⁷MST Version 0.4b

⁸<http://ltrc.iiit.ac.in/analyzer/hindi/>

	<i>Malt</i>						<i>MST+MaxEnt</i>					
	<i>Cross-validation</i>			<i>Test-set</i>			<i>Cross-validation</i>			<i>Test-set</i>		
	<i>UAS</i>	<i>LAS</i>	<i>LS</i>	<i>UAS</i>	<i>LAS</i>	<i>LS</i>	<i>UAS</i>	<i>LAS</i>	<i>LS</i>	<i>UAS</i>	<i>LAS</i>	<i>LS</i>
<i>PaF</i>	82.2	69.3	73.4	84.6	72.9	76.5	79.4	66.5	70.7	81.6	69.4	73.1
<i>MaF</i>	82.5	71.6	76.1	84.0	73.6	77.6	82.3	70.4	75.4	83.4	72.7	77.3
<i>LMSaF</i>	83.2	73.0	77.0	85.5	75.4	78.9	82.6	71.3	76.1	85.0	73.4	77.3
<i>2stage</i>	79.0	69.5	75.6	79.6	71.1	76.8	78.8	66.6	72.6	80.1	69.7	75.4

Table 6.2 Results of all the four experiments using automatic shallow parser information.

87% accurate respectively. These accuracies are obtained after using the approach of Avinesh and Gali (2007) on larger training data. In addition, while using automatic shallow parser information to get the results, we also explored using both gold-standard and automatic information during training. As expected, using automatic shallow parser information for training gave better performance than using gold while training.

Table 6.1 and Table 6.2 shows the results of the four experiments using gold-standard and automatic shallow parser information respectively. We evaluated our experiments based on unlabeled attachment score (UAS), labeled attachment score (LAS) and labeled score (LS) (Nivre et al., 2007a). Best LAS on test data is 84.4% (with 2stage) and 75.4% (with LMSaF) using gold and automatic shallow parser information respectively. These results are obtained using MaltParser. In the following section we discuss the results based on different criterion.

POS tags provide very basic linguistic information in the form of broad grained categories. The best LAS for PaF while using gold and automatic tagger were 80.1% and 72.9% respectively. The morph information in the form of case, suffix and root information proved to be the most important features. But surprisingly, gender, number and person features didn't help. Agreement patterns in Hindi are not straightforward. For example, the verb agrees with k2 if the k1 has a post-position; it may also sometimes take the default features. In a passive sentence, the verb agrees only with k2. The agreement problem worsens when there is coordination or when there is a complex verb. It is understandable then that the parser is unable to learn the selective agreement pattern which needs to be followed.

LMSaF on the other hand encode richer information and capture some local linguistic patterns. The first four features in LMSaF (chunk type, chunk boundary, head/non-head of chunk and distance to the end of chunk) were found to be useful consistently. The fifth feature, in the form of suffix concatenation, gave us the biggest jump, and captures the correlation between the TAM markers of the verbs and the case markers on the nominals.

6.4.1 Feature comparison: PaF, MaF vs. LMSaF

Dependency labels can be classified as two types based on their nature, namely, inter-chunk dependency labels and intra-chunk labels. Inter-chunk dependency labels are syntacto-semantic in nature. Whereas intra-chunk dependency labels are purely syntactic in nature.

Figure 6.4, shows the f-measure for top six inter-chunk and intra-chunk dependency labels for PaF, MaF, and LMSaF using Maltparser on test data using automatic shallow parser information. The first six labels (k1, k2, pof, r6, ccof, and k7p) are the top six inter-chunk labels and the next six labels (lwg__psp, lwg__aux, lwg__cont, rsym, nmod__adj, and pof__cn) are the top six intra-chunk labels. First six labels (inter-chunk) correspond to 28.41% and next six labels (intra-chunk) correspond to 48.81% of the total labels in the test data. The figure shows that with POS information alone, f-measure for top four intra-chunk labels reached more than 90% accuracy. The accuracy increases marginally with the addition of morph and local morphosyntactic features. The results corroborates with our intuition that intra-chunk dependencies are mostly syntactic. For example, consider an intra-chunk label ‘lwg__psp’. This is the label for postposition marker. A post-position marker succeeding a noun is attached to that noun with the label ‘lwg__psp’. POS tag for post-position marker is PSP. So, if a NN (common noun) or a NNP (proper noun) is followed by a PSP (post-position marker), then the PSP will be attached to the preceding NN/NNP with the dependency label ‘lwg__psp’. As a result, providing POS information itself gave an f-measure of 98.3% for ‘lwg__psp’. With morph and local morphosyntactic features, this got increased to 98.4%. However, f-measure for some labels like ‘nmod__adj’ is around 80% only. ‘nmod__adj’ is the label for adjective-noun, quantifier-noun modifications. Low accuracy for these labels is mainly due to two reasons. One is POS tag errors. And the other is attachment errors due to genuine ambiguities such as compounding.

For inter-chunk labels (first six columns in the figure 6.4), there is considerable improvement in the f-measure using morph and local morphosyntactic features. As mentioned, local morphosyntactic features provide local linguistic information. For example, consider the case of verbs. At POS level, there are only two tags ‘VM’ and ‘VAUX’ for main verbs and auxiliary verbs respectively (Bharati et al., 2006). Information about finite/non-finiteness is not present in the POS tag. But, at chunk level there are four different chunk tags for verbs, namely VGF, VGNF, VGINF and VGNN. They are respectively, finite, non-finite, infinitival and gerundial chunk tags. The difference in the verbal chunk tag is a good cue for helping the parser in identifying different syntactic behavior of these verbs. Moreover, a finite verb can become the root of the sentence, whereas a non-finite or infinitival verb can’t. Thus, providing chunk information also helped in improving the correct identification of the root of the sentence.

Similar to Prague Treebank (Hajičová, 1998), coordinating conjuncts are heads in the treebank that we use. The relation between a conjunct and its children is shown using ‘ccof’ label. A coordinating conjunct takes children of similar type only. For example, a coordinating conjunct can have two finite verbs or two non-finite verbs as its children, but not a finite verb and a non-finite verb. Such instances are also handled more effectively if chunk information is incorporated. The largest increase in performance, however, was due to the ‘suffix concatenation’ feature. Significant improvement in the core inter-chunk

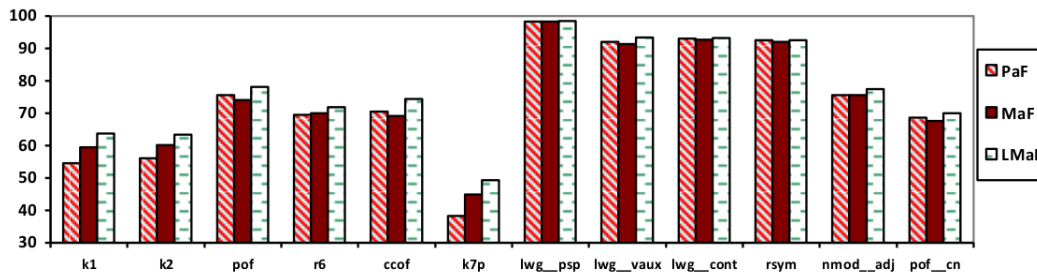


Figure 6.4 F-measure of top 6, inter-chunk and intra-chunk labels for PaF, MaF and LMSaF

dependency labels (such as k1, k2, k4, etc.) due to this feature is the main reason for the overall improvement in the parsing accuracy. As mentioned earlier, this is because this feature captures the correlation between the TAM markers of the verbs and the case markers on the nominals.

6.4.2 Approach comparison: LMSaF vs. 2stage

Both LMSaF and 2stage use chunk information. In LMSaF, chunk information is given as a feature whereas in 2stage, word level parsing is divided into intra-chunk and inter-chunk parsing. Both the approaches have their pros and cons. In LMSaF as everything is done in a single stage there is much richer context to learn from. In 2stage, we can provide features specific to each stage which can't be done in a single stage approach (McDonald et al., 2006). But in 2stage, as we are dividing the task, accuracy of the division and the error propagation might pose a problem. This is reflected in the results where the 2-stage performs better than the single stage while using gold standard information, but lags behind considerably when the features are automatically computed.

During intra-chunk parsing in the 2-stage setup, we tried out using both a rule-based approach and a statistical approach (using MaltParser). The rule based system performed slightly better (0.1% LAS) than statistical when gold chunks are considered. But, with automatic chunks, the statistical approach outperformed rule-based system with a difference of 7% in LAS. This is not surprising because, the rules used are very robust and mostly based on POS and chunk information. Due to errors induced by the automatic POS tagger and chunker, the rule-based system couldn't perform well. Consider a small example chunk given below.

((		NP
meraa	'my'	PRP
bhaaii	'brother'	NN
))		

As per the Hindi chunking guidelines (Bharati et al., 2006), *meraa* and *bhaaii* should be in two separate chunks. And as per Hindi dependency annotation guidelines (Bharati et al., 2009b), *meraa* is

attached to *bhaaii* with a dependency label ‘r6’⁹. When the chunker wrongly chunks them in a single chunk, intra-chunk parser will assign the dependency relation for *meraa*. Rule based system can never assign ‘r6’ relation to *meraa* as it is an inter-chunk label and the rules used cannot handle such cases. But in a statistical system, if we train the parser using automatic chunks instead of gold chunks, the system can potentially assign ‘r6’ label.

6.4.3 Parser comparison: MST vs. Malt

In all the experiments, results of MaltParser are consistently better than MST+MaxEnt. We know that Maltparser is good at short distance labeling and MST is good at long distance labeling (Mcdonald and Nivre, 2007). The root of the sentence is better identified by MSTParser than MaltParser. Our results also confirm this. MST+MaxEnt and Malt could identify the root of the sentence with an f-measure of 89.7% and 72.3% respectively. Presence of more short distance labels helped Malt to outperform MST. Figure 6.5, shows the f-measure relative to dependency length for both the parsers on test data using automatic shallow parser information for LMSaF.

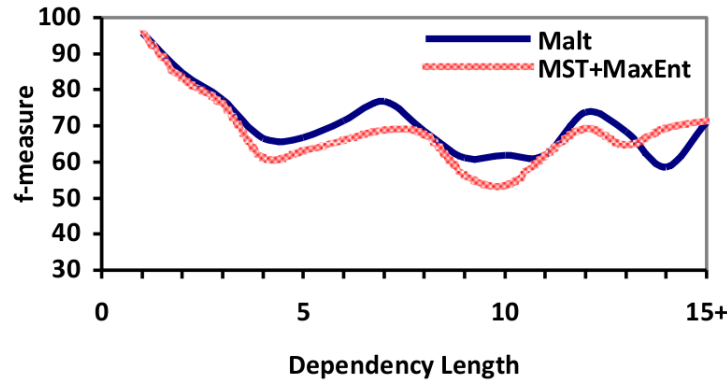


Figure 6.5 Dependency arc f-measure relative to dependency length

6.5 Discussion and Future Work

We systematically explored the effect of various linguistic features in word level Hindi dependency parsing. Results show that POS, case, suffix, root, along with local morphosyntactic features help dependency parsing. We then described two methods to incorporate such features during the parsing process. These methods can be thought as different paradigms of modularity. For practical reasons (i.e. given the POS tagger/chunker accuracies), it is wiser to use this information as features rather than dividing the task into two stages.

⁹‘r6’ is the dependency label for genitive relation

As mentioned earlier, this is the first attempt at sentence parsing for Hindi going down to word level. So, we cannot compare our results with previous attempts at Hindi dependency parsing, due to, (a) The data used here is different and (b) we produce sentence parses going down to word level rather than chunk level parses.

As mentioned in section 6.4.1, accuracies of intra-chunk dependencies are very high compared to inter-chunk dependencies. Inter-chunk dependencies are syntacto-semantic in nature. The parser depends on surface syntactic cues to identify such relations. But syntactic information alone is always not sufficient, either due to unavailability or due to ambiguity. In such cases, providing some semantic information can help in improving the inter-chunk dependency accuracy. There have been attempts at using minimal semantic information in dependency parsing for Hindi (Bharati et al., 2008a). Recently, Ambati et al. (2009b) used six semantic features namely, human, non-human, in-animate, time, place, and abstract for Hindi dependency parsing. Using gold-standard semantic features, they showed considerable improvement in the core inter-chunk dependency accuracy. Some attempts at using clause information in dependency parsing for Hindi (Gadde et al., 2010) have also been made. These attempts were at inter-chunk dependency parsing using gold-standard POS tags and chunks. We plan to see their effect in sentence parsing going down to word level using automatic shallow parser information also.

6.6 Conclusion

We explored two strategies to incorporate local morphosyntactic features in sentence level Hindi dependency parsing. These features were obtained using a shallow parser. We first explored which information provided by the shallow parser is useful and showed that local morphosyntactic features in the form of chunk type, head/non-head info, chunk boundary info, distance to the end of the chunk and suffix concatenation are very crucial for Hindi dependency parsing. We then investigated the best way to incorporate this information during dependency parsing. Further, we compared the results of various experiments based on various criteria and did some error analysis. This work was also the first attempt at sentence level parsing for Hindi going down to word level.

Chapter 7

Error Detection for Treebank Validation

As we have seen in the previous chapter (Chapter 6), with just 1000 sentences for training, we could build a dependency parser for Hindi with an accuracy of 75.4% LAS (Labeled Attachment Score). The training data size for Hindi is very low compared to other treebanks like Penn Treebank for English, Prague Dependency Treebank for Czech etc. Small treebank size is one of the most frequently stated reasons for low performance of the parsers (Nivre et al., 2007a; Nivre et al., 2007b; Bharati et al., 2008a). If we have comparable corpus for Hindi also then we could have higher parsing accuracies for Hindi. As mentioned in Chapter 6, the treebank used for experiments is a part of the new multi-layered and multi-representational Hindi Treebanking project (Bhatt et al., 2009) whose target is 400k words ($\sim 16k$ sentences) which is comparable to the treebanks for English and Czech. But, annotation process of this treebank is little slow. If we can identify the factors which are effecting the annotation process then we can work towards handling them.

When we analyzed the complete annotation process, we observed that the validation process is the most time-consuming part in the Hindi Treebank annotation. But, to have the annotated corpora free of anomalies (errors) and inconsistencies, experts need to validate them. As the data is already annotated carefully (which is a time-consuming task), we need tools that can supplement the validators' task with a view of making the overall task fast, without compromising on reliability. With the help of such a tool, a validator can directly go to error instances and correct them. Therefore, we need the tool to have high recall. It is easy to see that a human validator can reject un-intuitive errors (false positives) without much effort; one can therefore compromise a little bit on precision.

Here, we propose a tool to detect errors in the Treebank. We classify the identified errors under specific categories for the benefit of the validators, who may choose to correct a specific type of error at one time. We used a combination of a rule-based and hybrid system to do this task. Rule-based system works on the development of robust (high precision) rules. The rules are formed after a thorough study of the annotation guidelines and the framework. Whereas the hybrid system is a combination of statistical system with a rule-based post-processing module. The statistical system helps in detecting a wide array of potential errors and suspect cases. The rule-based post-processing module then prunes out the false positives, with the help of robust and efficient rules thereby ensuring higher precision value.

7.1 Related Work

Validation and correction tools are an important part for making treebanks error-free and consistent. With an increase in demand for high quality annotated corpora over the last decade, major research works in the field of developing lexical resources have focused on detection of errors. One such approach for treebank error detection has been employed by Dickinson and Meurers (2003a; 2003b; 2005) and Boyd et al. (2008). The underlying principle in these works is to detect ‘variation n-grams’ in syntactic annotation across large corpora. These variations could be present for a continuous sequence of words (POS and chunks) or for a non-continuous sequence of words (dependency structures). More the number of variations for a particular contiguous or non-contiguous sequence of tokens (or words), greater the chance of a particular variation being an error. They use these statistical patterns (n-grams) to detect anomalies in POS annotation in corpora such as the Penn treebank (Marcus et al., 1993), TIGER corpus (Brants et al., 2002) etc. For discontinuous patterns as found most commonly in dependency annotation (Boyd et al., 2008), they tested their strategy on Talbanken05 (Nivre et al., 2006) apart from the corpora mentioned above. This we believe was the first mainstream work on error detection in dependency annotation.

Some other earlier methods employed for error detection in syntactic annotation (mainly POS and chunk), are by Eskin (2000) and van Halteren (2000). Based on large corpora, van Noord (2004) and de Kok et al. (2009) employed error mining techniques. The basic underlying strategy was to obtain a set of parsed and un-parsed sentences using a wide-coverage parser and compute suspicion ratio for detecting errors. Other examples of detection of annotation errors in treebanks include (Kaljurand, 2004; Kordoni, 2003).

Most of the aforementioned techniques work well with large corpora in which the frequency of occurrence of words is very high. Hence, none of them account for data sparsity except for de Kok et al. (2009). Moreover, the techniques employed by van Noord (2004) and de Kok et al. (2009) rely on the output of a reliable state-of-the-art parser which may not be available for many languages just as in the case of Hindi, the language in question for our work.

Our work is focused on detecting errors even when the annotated data available is very small in size and, in the process, addresses the problem of data sparsity. We employ a combination of a rule-based approach with a hybrid approach for error detection. Moreover, unlike earlier efforts, our work focuses on reduction of validation time and effort during treebank construction. So, our focus is on high recall with reasonable precision.

7.2 Hindi Dependency Annotation

During dependency annotation, Part-Of-speech (POS), morph, chunk and inter-chunk dependency relations are annotated. Some special features are also annotated for some specific nodes. In this section

we briefly describe the information encoded in the dependency representation of the treebank. We also describe the possible errors at each level of annotation.

7.2.1 Part-Of-Speech (POS):

POS tags are annotated for each node following the POS and chunk annotation guidelines (Bharati et al., 2006).

In POS errors we try to identify whether the Part-Of-Speech (POS) tag is correct or not for each lexical item. For example, in the example sentence given below ‘chala’ should be the main verb (VM) instead of an auxiliary verb (VAUX).

raama	ghara	chala	gayaa.
NNP	NN	VAUX	VAUX
‘Ram’	‘home’	‘walk’	‘went’.
“Ram went home”.			

7.2.2 Morph:

Information pertaining to the morphological features of the nodes is also encoded using the Shakti standard format (SSF) (refer, Bharati et al. (2007)). These morphological features have eight mandatory feature attributes for each node. These features are classified as root, category, gender, number, person, case, post position (for a noun) or tense aspect modality (for a verb) and suffix.

Errors in the eight attribute values as mentioned in the previous section are classified as morph errors.

7.2.3 Chunk:

After annotation of POS tags, chunk boundaries are marked with appropriate assignment of chunk labels (Bharati et al., 2006). This information is also stored in SSF (Bharati et al., 2007).

There can be two types of chunk errors. One is chunk type and the other is chunk boundary. In chunk type we identify whether the chunk label is correct or not. In chunk boundary we identify whether the node should belong the same chunk or different chunk. For example, consider the following chunk,

((		NP
meraa	‘my’	PRP
bhaaii	‘brother’	NN
))		

In Hindi, ‘meraa’ and ‘bhaaii’ should be in two separate noun chunks (refer Bharati et al. (2006)). So, in the above example, the chunk label of ‘bhaaii’ is correct, but the boundary is wrong.

7.2.4 Dependency Relations:

After POS, morph and chunk annotation, inter-chunk dependency annotation is done following the set of dependency guidelines in Bharati et al. (2009b). This information is encoded at the syntactico-semantic level following the Paninian dependency framework (Begum et al., 2008; Bharati et al., 1995).

In dependency errors we try to identify whether a node is attached to its correct parent and whether its dependency label is correct or not. In addition to dependency relation errors, we also identify errors in general linguistic constraints and framework specific errors. For example, the tree well-formedness assumption in dependency analysis. Framework specific example would be that children of a conjunct should be of similar type (Bharati et al., 2009b). For example, a conjunct can have two nouns as its children but not a noun and a verb as its children.

7.2.5 Other Features:

In the dependency treebank, apart from POS, morph, chunk and inter-chunk dependency annotations, some special features for some specific nodes are marked. For example, for the main verb of a sentential clause, information about whether the clause is declarative, interrogative or imperative is marked. Similarly, whether the sentence is in active or passive voice is also marked.

Errors in the special features discussed above are classified under other feature errors.

Our aim is to identify above mentioned errors in the Hindi dependency treebank. Since the errors are of different types, some are easy to identify using simple rules (eg: POS errors) while others are difficult to detect (eg: dependency errors). Therefore, the current work addresses the problem by adopting different approaches for detecting different types of errors. In this work we try to identify POS, chunk and dependency errors only. Morph and other feature errors are currently not handled.

7.3 Approaches

We describe both rule-based and hybrid approaches in detail in the following subsections. Based on the nature of errors we use the rule based system and/or the hybrid system to identify the errors. The entire framework is sketched in Figure 7.1.

7.3.1 Rule-Based System

In this system we use certain generic rules to identify the errors. The main idea behind framing these generic rules is that particular tags (POS/chunk/dependency) demand particular patterns and vice versa. For example, if the POS-tag is “SYM” (tag for a punctuation marker, see Bharati et al. (2006)) then the lexical item should not contain any character in the Unicode range of Hindi or digits. Similarly, if the lexical item is a digit, then the POS tag should be QC (short form of Cardinals).

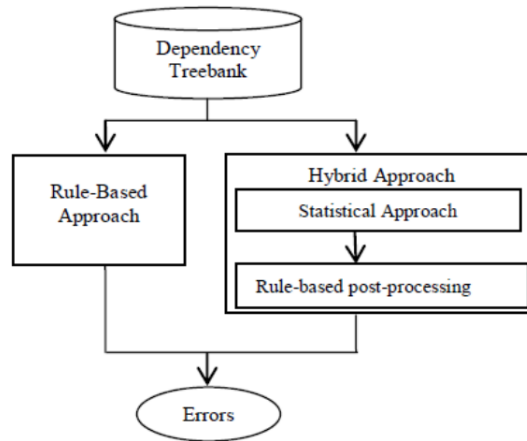


Figure 7.1 Error detection framework.

We used the annotation guidelines (Bharati et al., 2006; Bharati et al., 2009b) as an initial step to frame the rules. The guidelines, apart from providing description of the tags, give pointers to annotators, in the form of linguistic cues to identify the tags, exceptional cases, common confusing patterns and common error-prone patterns. Rules for identifying the errors are framed using such information. More rules were later formulated using the development data. Further, we extracted mismatches in the annotated and validated sets of the development data. These mismatches are basically errors made by annotators which were corrected by validators. Analysis of these mismatches helped in framing additional rules. Figure 7.2, shows an example case of a POS tag error. In the example sentence depicted below, “Ram ate two apples”, the rule that, a numeral should have its POS tag either as ‘QC’, or as ‘QF’ (refer, Bharati et al. (2006)) comes in handy while detecting the error. Therefore, the word “do” which had been erroneously tagged as a demonstrative (DEM) in the sentence, is identified as an error which can be then promptly corrected by the human validator.

Identification of Errors using Rules				
1	raama	‘Ram’	NNP	
2	ne	‘ERG’	PSP	
3	<i>do</i>	<i>‘two’</i>	<i>DEM</i>	◀◀
4	seba	‘apple’	NN	
5	khaaye	‘ate’	VM	
“Ram ate two apples.”				

Figure 7.2 Error detection at POS level by rule-based approach.

The nature of the rules varies for different levels of annotation, as the context required would be different. For example POS tagging rules are based on current lexical item, POS tags of previous and/or next words etc., whereas in case of dependency tags, rules are framed on features of current node, its parent, siblings, children and sometimes even a complete tree/sub-tree.

Figure 7.3, shows an example case of a chunk error. The rule states that the head for a particular chunk should be from a pre-defined set of POS tags and labels in Bharati et al. (2006). For instance, an ‘NP’ chunk should have noun as its head. Whereas, in the example shown above, NP has an adjective (JJ) as the head. This is an error and is detected by the tool. This gives a cue to the validator that either the word is erroneously tagged or the chunk label is incorrect. In this case, however, the validator corrects the chunk label from NP to JJP.

Identification of Chunk Errors using Rules					
1	((	NP			
1.1	pulisa	'police'	NN		
1.2	ne	'ERG'		PSP	
	)	)			
2	((	NP			
2.1	riporta	'report'	NN		
2.2	ko	'DAT'		PSP	
	)	)			
3	((	NP			◀◀◀
3.1	daayara	'register'	JJ		
	)	)			
4	((	VGF			
4.1	kiyaa	'did'	VM		
	)	)			
'The police registered a report.'					

Figure 7.3 Error detection at chunk level by rule-based approach.

There are robust and effective rules to find out errors at the dependency level as well. An example sentence with its dependency tree is shown below.

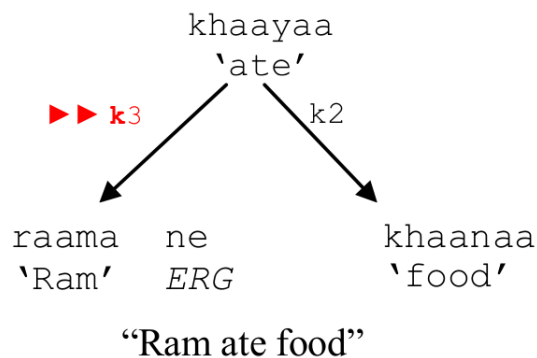


Figure 7.4 Error detection at dependency level by rule-based approach.

The rule that the dependency label for a noun chunk with an ergative case marker (in this case *ne*) must be *kI* (refer Bharati et al. (2009b)) works well and detects the error in the arc between the two nodes as shown by the pointer in Figure 7.4.

The rule based system is high on precision. However, it leaves out a number of errors which are relatively more difficult to identify with generic linguistic context. Therefore, to improve the recall of the error detection tool we clubbed the rule-based system with a hybrid system.

7.3.2 Hybrid System

In the hybrid approach we use a statistical system to identify the errors. The main aim of the statistical system is to achieve a high recall at the cost of precision. On the detected errors of the statistical system we run another rule-based module as a post-processing step to reduce the number of false positives and thereby increase the precision. In the statistical system we explored frequency based and probability based approaches. Details of these two approaches are described as follows.

7.3.2.1 Frequency Based Approach

Statistically, low frequency is a sign of possible error. In this approach we calculate the frequencies of patterns and their tag pairs, where a tag can be either a POS tag or a chunk tag or an inter-chunk dependency label. For POS, word level patterns are considered. For chunks, lexical items and the POS tags of the sequence of words within the chunk are considered. For inter-chunk dependencies, chunk tag, lexical item and POS tag sequence within the respective child and parent chunks are considered as the pattern.

Once we get the frequencies at each level, we keep some threshold on the frequency and all the pairs less than that threshold are considered as possible errors. This threshold is decided after experiments with the development data and it can vary with annotation level. For all the pairs greater than the threshold, if a pattern has multiple tags, then there might be a possibility of error. So, for such pairs, if the frequency of a pair is less than certain percentage of the total instances of that pattern, then it is considered as a possible error.

The above approach works well at POS level as the pattern contains only lexical item. But, when it comes to chunk and dependency levels, sparsity creates problems as the patterns are complex. Probability of occurrence of the same pattern is very low because of which a lot of valid instances get identified as errors. To resolve this, instead of original patterns, we find similarity between patterns and merge similar patterns. Again, the similarity criterion varies with annotation type. The main motive of this similarity criterion is to merge the patterns which look similar. On these merged patterns, we apply the frequency based approach to detect the errors. This approach of finding similarity will reduce the count of correct patterns being identified as errors but not completely remove it. To further reduce the negative effect of sparsity on these merged patterns, we use certain robust rules to remove false positives from

the errors list. So, a robust rule is capable of overriding a low frequency based pattern induction and can remove such a pattern from the final selection.

Figure 7.5, explains the complete approach taking inter-chunk dependency as an example. There are 6 different pairs of inter-chunk dependency patterns and the dependency labels. Out of these six, only the fourth pair is a valid error. But, as the frequencies are low, all the 6 pairs got identified as errors. After finding similarity between patterns and merging similar patterns, 6 pairs got reduced to 3 pairs. The similarity criterion used here is,

For both child and parent chunks, consider POS type of the head of the chunk and lexical item and POS tags of functional words.

We consider the POS tags NN (common noun), NNP (proper noun) and PRP (pronoun) as of same POS type 'NOUN'. Based on this similarity criterion, all the first five patterns got merged into a single pattern ($NP : NOUN \text{ } ne - PSP - > VGF : VM$). But the first four patterns have the same tag 'k1' and the fifth pattern has a separate tag 'vmod'. Due to this, the first five patterns got merged into two pairs instead of one. As there aren't any patterns similar to the sixth pattern it remained as a separate pair.

Out of these 3 pairs, 2 pairs are identified as errors by the frequency-based approach. Now we apply a rule-based post-processing module to reduce the false positives. After applying the rule,

If the child is an adverbial chunk (RBP) and the parent is a verbal chunk (VGF), then the dependency label can be "adv"

the number of errors reduced from two to one. In this manner the statistical approach tries for high recall at the cost of precision. The rule-based post-processing mechanism tries to increase the precision by reducing the number of false positives. Thus, we can use the hybrid approach to identify errors.

One major difference between our frequency based approach and the previous works by Dickinson and Meurers (2003a; 2003b; 2005) and Boyd et al. (2008) is the use of similarity criteria. As they worked on large data sets they just took the n-gram counts. But as we are working on small data sets, similarity based comparisons proved to be more helpful than mere counts. This is an attempt to handle sparsity.

One limitation of the frequency based hybrid approach is that we can't give richer context due to the problem of sparsity. To find whether the dependency label is correct or not, apart from node and its parent information, sibling and child information is also helpful. Current state-of-the-art dependency parsers use these features for dependency labeling (McDonald et al., 2006; Ambati et al., 2009a). Finding similarity between patterns and merging similar patterns would not help when we wish to take a much richer context. For this purpose, we also explored a probability based approach.

7.3.2.2 Probability Based Hybrid Approach

In the probability based hybrid approach, we first extract the contextual features which help in identifying the correct tag. These features might vary across POS, chunk and dependency levels. At POS level apart from the word, we consider prefixes and suffixes of the current word, previous and next words and

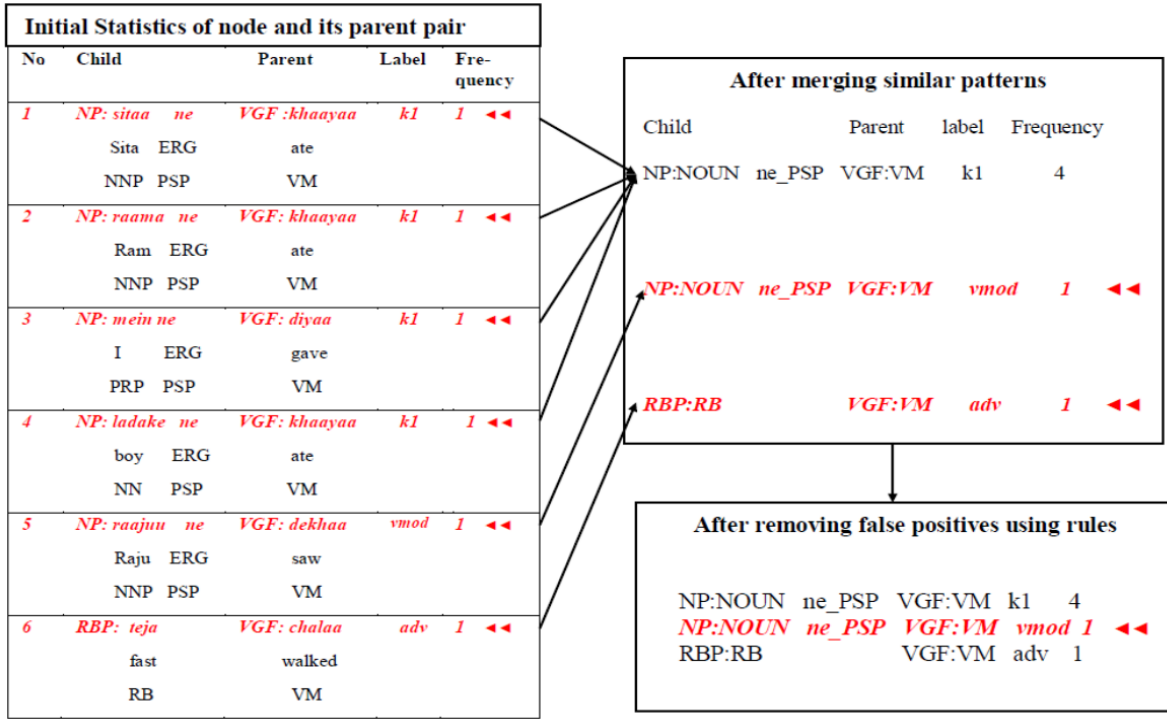


Figure 7.5 Error detection in inter-chunk dependencies by frequency based hybrid approach.

their POS tags. At the chunk level, apart from the word and its POS tag, we consider previous and next words, their POS tags and chunk tags. At the dependency level, apart from node and its parent features, we use sibling and child features with their respective dependency labels. The main aim is to find the probability of the correct tag in the given context.

Using these contextual features from the training data we create a model using maximum entropy classification algorithm ¹ (MAXENT), which gives the probabilities of the all possible output tags for a given context. For each node in the test data, we first extract the context information and the input tag of that node. We then extract the list of all possible dependency tags with their probabilities for this context using the trained model. From this list we extract first best and second best tags and their corresponding probabilities.

If the input tag doesn't match with the first best tag, and if the probability of the first best tag is greater than a particular threshold, we then consider it as a possible error node. These could be the cases which require much richer context to find the correct tag.

If both the input tag and the first best tag given by the model match we then fix a maximum and minimum threshold on the probability values. If the probability of the first best tag is greater than the maximum threshold we do not consider it as a potential error. The chance of it being an error is very low as the system is very confident about its decision. If the probability of the first best tag is less than

¹<http://maxent.sourceforge.net/>

the minimum threshold, it is considered as a possible error. This could either be the case of an error pattern or a correct but less frequent pattern. If it is the latter, then the rule-based post-processing tool will remove this false positive.

If the probability value lies between the maximum and minimum thresholds, we calculate the difference between the probabilities of the first and second best tags. If the difference is less than a particular value, it means that there is high ambiguity between these two tags. As there is high ambiguity there is a greater chance of making an error. Hence, we identify this case as a possible error.

```

find_errors(input)
  for each sentence in the input:
    for each node in the sentence:
      1. Get the node's context
      2. Get the input_tag
      3. tags_probs = get_probabilities(context);
      4. 1stBestTag = tags_probs[0][0];
      5. 1stBestProb = tags_probs[0][1];
      6. 2ndBestTag = tags_probs[1][0];
      7. 2ndBestProb = tags_probs[1][1];
      8. if input_tag != 1stBestTag:
          if 1stBestProb > thres_minX:
              mark node as error node;
      9. else:
          if 1stBestProb < thres_max:
              if 1stBestProb < thres_min:
                  mark node as error node;
              else if 1stBestProb-2ndBestProb < thres_dif:
                  mark node as error node;

get_probabilities(context)
  1. Load the trained maxent model
  2. Predict probabilities of all tags for the context.
  3. Store the tags and their probabilities in an array.
  4. return the array

```

Figure 7.6 Algorithm employed for PBHA.

Figure 7.6 shows the procedure of probability based hybrid approach (PBHA). Similar to FBHA discussed previously, we use a rule-based post processing module to reduce the number of false positives. Use of richer contextual information and probabilities to detect errors makes this approach more effective from the previous approaches employed for error detection (Dickinson and Meurers, 2003a; Dickinson and Meurers, 2003b; Dickinson and Meurers, 2005; Boyd et al., 2008). Using this approach, not only can one detect errors but also classify them under specific categories like ambiguous cases, less frequent cases etc.

<i>Type of Error</i>	<i>Total instances</i>	<i>Total Errors</i>	<i>Recall of the tool</i>
<i>POS Errors</i>	13922	16	12/16 = 75%
<i>Chunk Errors</i>	7113	24	15/24 = 62.5%
<i>Dependency Errors</i>	7113	843	218/843 = 25.86%

Table 7.1 Error Detection using rule-based system at different levels.

7.4 Experiments and Results

We evaluated the performance of our system using a 65k-token manually annotated and validated sample of data (2694 sentences) derived from the Hindi dependency treebank. We divided the data into 40k, 10k and 15k for training, development and testing respectively. For the rule-based system, training and development data was used to frame the rules. In the case of hybrid approach, we used training data to train the models and development data to tune the parameters like threshold values. Rules meant for pruning false positives were also framed using this data.

We ran the rule-based tool on the test data. Details of the type and number of errors identified by the rule based system are presented in Table 7.1. Using our rule-based system we detected 75%, 62.5% and 25.86% of errors at POS, chunk and dependency levels respectively. Currently in the treebank, dependency annotation is done at inter-chunk level only. So, dependency errors only represent inter-chunk dependency errors.

During annotation at POS and chunk levels, CRF+TBL based automatic POS-tagger and chunker which are 93% and 87% accurate respectively are used. These accuracies are obtained after using the approach of Avinesh and Gali (2007) on larger training data. This tagged data is then given to the human annotators for correction. In this process, most of the errors generated by the automatic taggers are corrected. This is the reason for less number of POS and chunk errors in the annotated data. As the number of errors is low and as these can be identified based on some standard rules, rule-based system performs quite well in this case. We also tried both the hybrid approaches, but the number of false positives is so high that the hybrid approaches are practically of no use in case of POS and chunk levels.

But at dependency level there aren't any reliable parsers currently available for Hindi, thus the annotators have to annotate data manually. As more complex linguistic information is being annotated, the chance of making errors is more. As the number of errors is large we need tools to detect the errors so that the validation process becomes faster. With the rule based system we were able to identify only 25.86% of the dependency errors. We then tried out both the frequency based and probability based hybrid approaches to detect errors at dependency level. Results are presented in Table 7.2.

With frequency based hybrid approach we were able to identify only 18.74% of the errors. The precision recorded for this approach was also quite low. But with the probability based hybrid approach, we detected 57.06% of the errors with a reasonable precision value. Note that, our main aim is to achieve a high recall value. The false positives can be easily discarded by the validators.

<i>Approach</i>	<i>Total Errors (Total instances)</i>	<i>System output</i>	<i>Correct Errors</i>	<i>Recall</i>
<i>FBHA</i>	843 (7113)	2546	158	18.74%
<i>PBHA</i>	843 (7113)	2000	481	57.06%

Table 7.2 Error Detection at dependency level using both the frequency based and probability based hybrid approaches.

<i>Approach</i>	<i>Total Errors</i>	<i>System output</i>	<i>Correct Errors</i>	<i>Recall</i>
<i>Rule Based Approach</i>	843	218	218	25.86%
<i>PBHA</i>	843	2000	481	57.06%
<i>Combining both the Approaches</i>	843	2165	646	76.63%

Table 7.3 Error Detection at dependency level using both the frequency based and probability based hybrid approaches.

When we combined the outputs of both the rule-based and probability based hybrid approaches, we could identify 76.63% of the errors at the dependency level. Results are shown in Table 7.3.

7.5 Discussion and Future Work

One basic difference between our approach and the other previous approaches is that we use a combination of a rule-based system and a hybrid system to detect errors. Out of two hybrid approaches explored, PBHA performs considerably better than FBHA in detecting errors at dependency level. The former approach identifies 38% more errors than the latter. As we have less amount of data, our hypothesis for FBHA, “Low frequency is a possible sign of error” didn’t work. Unsurprisingly, several valid patterns had low counts. Major advantage of PBHA over FBHA is the use of richer context. Richer context helped PBHA to predict the errors more accurately. But in FBHA we couldn’t use it because of sparsity issues.

Although we worked and presented our results only on the Hindi Treebank, our approach can be generalized to any language and to any framework. Rules used in rule based system and the post-processing step of the two hybrid approaches are language and framework dependent. These rules can be changed based on the language and the framework being used. In statistical part of the hybrid approach, tuning of parameters like the threshold values depend on the size of data, apart from the language and the framework.

The tool is being constantly improved. We are analyzing the errors which are missed out by the rule-based system and planning to improve the rules, if possible. For the hybrid approach, we intend to provide a mechanism where validators can update the rules of rule-based post-processing module. They can add these rules for the pattern identified as an error owing to a low number of instances in the

training data. This sort of feedback from the validators would help improve the precision of the hybrid system. Recall of the system can be improved with more data being available for training the models. We would also like to evaluate our system on the time taken for validation. That is the reduction in the validation time using this tool.

This tool can also help in improving the guidelines which subsequently improves the annotation. While correcting the errors if the validator comes across some ambiguous decisions or some common errors or comes up with new decisions, guidelines can be modified accordingly to reflect the changes. Data annotated based on new guidelines will reduce the occurrence of these errors and eventually the quality of annotation of individual as well as entire data will improve. Figure 7.7, shows the complete cycle of this process.

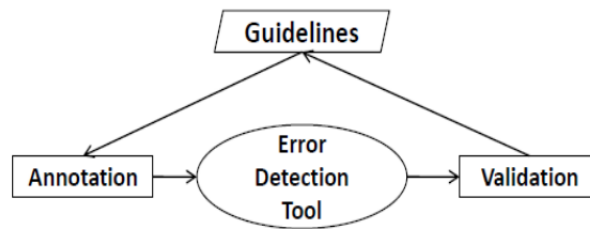


Figure 7.7 Cycle - Improving guidelines for better annotation

7.6 Conclusion

We have designed and built a new tool which uses both rule-based and hybrid systems to detect errors. We tested it on Hindi dependency treebank data and were able to detect 75%, 62.5% and 76.63% of errors in POS, chunk and dependency annotation respectively. For detecting POS and chunk errors, we used the rule-based system. For dependency errors, we used the combination of both rule-based and hybrid systems. This tool can be generalized to detect errors in annotation of any language/framework. Proposed approach works well even when the size of the data is low.

Chapter 8

Conclusions and Future Work

Hindi is a morphologically rich, free word order language. Parsing morphologically rich, free word order languages (MoR-FWO) is a challenging task. In this work we presented our experiments which led to state-of-the-art dependency parser for Hindi. We did a series of experiments exploring the role of different morphological and syntactic features in Hindi dependency parsing using two data-driven parsers, Malt and MST. With just 1500 sentences training data, we were able to build a dependency parser with state-of-the-art accuracy of 74.5% Labelled Attachment Score (LAS) and 90.1% Unlabelled Attachment Score (UAS). We also did a detailed error analysis isolating specific linguistic phenomena and/or other factors that impede the overall parsing performance, and suggested possible remedies for these problems. Some of these problems are explored/being explored as separate experiments and some are yet to be explored. One can build a single final model merging all these efforts.

During error analysis we found that some of the basic linguistic constraints are violated in the parser produced by these data driven parsers. This motivated us to build a linguistically sound parser without compromising on the accuracy. We considered a simple linguistic constraint that a verb should not have multiple karta karaka (roughly subject)/karma karaka (roughly object) as its child in the dependency tree and proposed two approaches to handle this constraint. We evaluated these two approaches on the state-of-the-art dependency parsers for Hindi and Czech. This is just an initial step towards incorporating linguistic constraints into statistical dependency parsers. Currently, we are handling only two labels, subject and direct object. Apart from subject and direct object there can be other labels for which multiple instances for a single verb is not valid. We can extend our approaches to handle such labels also. We can also explore the ways of incorporating other useful linguistic constraints.

After building state-of-the-art inter-chunk dependency parser for Hindi, we also presented our preliminary work on parsing of Hindi going down to word level. We extracted 1000 sentences, which are completely annotated down to word level, from a new multi-layered and multi-representational Hindi Treebank which is being developed. We did a step by step analysis of the importance of different features like, POS, morph and chunk information for sentence level parsing of Hindi. We were able to achieve an accuracy of 75.4% Labelled Attachment Score (LAS) and 85.5% Unlabelled Attachment Score (UAS). With more training data and more experiments we hope to achieve even better accuracies.

During analysis we found that accuracies of intra-chunk dependencies are very high compared to inter-chunk dependencies. There are attempts at showing the importance of semantic features (Bharati et al., 2008a; Ambati et al., 2009b) and clausal features in inter-chunk dependency parsing for Hindi. We plan to see their effect in complete sentence parsing using automatic shallow parser information also.

Next to dependency parser for Hindi, the second major contribution of this thesis is error detection tool for Hindi Treebank validation. We proposed a new tool which uses both rule-based and hybrid systems to detect errors. We tested it on Hindi dependency treebank data and were able to detect 75%, 62.5% and 76.63% of errors in POS, chunk and dependency annotation respectively. For detecting POS and chunk errors, we used the rule-based system. For dependency errors, we used the combination of both rule-based and hybrid systems. This tool can be generalized to detect errors in annotation of any language/framework. Proposed approach works well even when the size of the data is low. This is an initial work in this area. We are planning to improve the

The tool is being constantly improved. We are planning to improve both the rule-based and hybrid systems. We would also like to evaluate our system on the time taken for validation. That is the reduction in the validation time using this tool. This tool can also help in improving the guidelines which subsequently improves the annotation.

Finally, in this thesis, we presented our work on inter-chunk dependency parsing and initial efforts towards linguistically sound parser and sentence level parser going down to word level. We also presented our error detection tool which is being used in validation of the Hindi treebank.

Related Publications

1. **Bharat Ram Ambati**, Phani Gadde and Karan Jindal. (2009). Experiments in Indian Language Dependency Parsing. In *Proceedings of ICON09 NLP Tools Contest: Indian Language Dependency Parsing*. Hyderabad, India.
2. **Bharat Ram Ambati**, Samar Husain, Joakim Nivre and Rajeev Sangal. (2010). On the Role of Morphosyntactic Features in Hindi Dependency Parsing. In *Proceedings of NAACL-HLT 2010 workshop on Statistical Parsing of Morphologically Rich Languages (SPMRL 2010)*, Los Angeles, CA.
3. **Bharat Ram Ambati**. (2010). Importance of linguistic constraints in statistical dependency parsing. In *Proceeding of ACL 2010 Student Research Workshop (SRW)*, Uppsala, Sweden.
4. **Bharat Ram Ambati**, Samar Husain, Sambhav Jain, Dipti Misra Sharma and Rajeev. (2010). Two methods to incorporate ‘local morphosyntactic’ features in Hindi dependency parsing. In *Proceedings of NAACL-HLT 2010 workshop on Statistical Parsing of Morphologically Rich Languages (SPMRL 2010)*, Los Angeles, CA.
5. **Bharat Ram Ambati**, Mridul Gupta, Samar Husain and Dipti Misra Sharma. (2010). A high recall error identification tool for Hindi treebank validation. In *Proceedings of The 7th International Conference on Language Resources and Evaluation (LREC)*, Valleta. Malta.
6. **Bharat Ram Ambati**, Mridul Gupta, Samar Husain and Dipti Misra Sharma. Error Detection for Treebank Validation. (*To be submitted*).

Appendix A

Old Tagsets

The following is the list of POS and Chunk tags used for HyDT-Hindi treebank.

A.1 Old POS Tagset

No.	Description	Code (Example)
1.	Noun	NN (ladaZkA, naxI, vicAra, kaTorawA)
2.	Proper Noun	NNP (rAma, BAjapA)
3.	Pronoun	PRP (jo, vo, vaha, “jisa” ladaZke ne, jisane)
4.	Verb Finite Main	VFM (vaha “pItA” hE, vaha ladaZkA “hE”)
5.	Verb Auxillary	VAUX (KA/VFM cukA/VAUX hE/VAUX)
6.	Verb NonFinite Adjectival	VJJ (kAwe/VJJ hue/VAUX)
7.	Verb NonFinite Adverbial	VRB (KAkara, pIwe/VRB hue/VAUX)
8.	Verb NonFinite nominal	VNN (pInA)
9.	Adjective	JJ (aXikawara, sarvowwama:)
10.	Adverb	RB (XIre/RB XIre/RB, wejI/RB se/RP)
11.	Noun location	NLOC (upara, Age, pehele, bAxa)
12.	Postposition	PREP (ne, ke/PREP liye/PREP)
13.	Particle	RP (mItA sA/RP, waka/RP, hI/RP, wo/RP, BI/RP)
14.	Conjunct	CC (Ora, yA, ki)
15.	Question words	QW (kyA/QW, kEsA/QW)
16.	Quantifier	QF (jyAxA/QF, WodA/QF, kama/QF, bahuwa/QF)
17.	Number Quantifiers	QFNUM (wIsarA, wInoM, wIna)
18.	Intensifier	INTF (“bahuwa” jyAxA, “Ora” jyAxA)
19.	Negative	NEG (nA, nahIM)
20.1	Compound Common Nouns	NNC (kenxra/NNC sarakAra/NN)
20.2	Compound Proper Nouns	NNPC (SrI/NNC pI/NNPC ke/NNPC miSrA/NNP)

21.	Interjection words	UH (hAM and interjections)
22.	Symbol	SYM
23.	Demonstrative	DEM (vaha)
24.	Null Element	NULL

A.2 Old Chunk Tag Set

SI No.	Tag Name	Description
1	NP	Minimal NP (acCA ladakA, nIII kitAba)
2	VG	verb and its axiliaries (A rahA hai, bETA hai)
3	JJP	adjectival chunk (sundara, bhauwa sundara) this chunk will be formed for adjectives which occur away from the expected prenominal position, eg laDakA, sundara aur catura. Here, ‘sundara aur catura’ occur post nominally. This expression can not be part fo an NP.
4	RBP	adverbial chunk (very fast, quickly) eg He runs very fast
5	BLK	Entities such as interjections and discourse markers that cannot fall into any of the above mentioned chunks will be kept within a separate chunk. eg. ((oh_INJ))_BLK, ((arre_INJ))_BLK
6	CCP	This is a special chunk name. Primarily used for ‘conjuncts’. Eg. Vaha yahAz AyA thA aur muJase milA thA
7	NULL_*	This chunk is introduced during the task of dependency annotation. The chunk represents a missing entity which is necessary to make the dependency tree complete.
8	NEGP	When the negative element of the verb occurs away from it, it is placed in a negation phrase.

Appendix B

New Tagsets

B.1 New POS Tagset

The following is the list of POS and Chunk tags being used for Hindi treebank (under development).

Sl No.	Category	Tag name	Example
1.1	Noun	NN	
1.2	NLoc	NST	
2.	Proper Noun	NNP	
3.1	Pronoun	PRP	
3.2	Demonstrative	DEM	
4	Verb-finite	VM	
5	Verb Aux	VAUX	
6	Adjective	JJ	
7	Adverb	RB	*Only manner adverb
8	Post position	PSP	
9	Particles	RP	bhI, to, hI, jI, hA.N, na,
10	Conjuncts	CC	bole (Bangla)
11	Question Words	WQ	
12.1	Quantifiers	QF	bahut, tho.DA, kam (Hindi)
12.2	Cardinal	QC	
12.3	Ordinal	QO	
12.4	Classifier	CL	
13	Intensifier	INTF	
14	Interjection	INJ	
15	Negation	NEG	
16	Quotative	UT	ani (Telugu), mAne (Hindi)

17	Sym	SYM	
18	Compounds	*C	
19	Reduplicative	RDP	
20	Echo	ECH	
21	Unknown	UNK	

B.2 New Chunk Tagset

Sl. No	Chunk Type	Tag Name	Example
1	Noun Chunk	NP	Hindi: ((merA nayA ghara))_NP my new house
2.1	Finite Verb Chunk	VGF	Hindi: mEMne ghara para khAnA ((khAyA_VM))_VGF
2.2	Non-finite Verb Chunk	VGNF	Hindi: mEMne ((khAte khAte_VM))_VGNF ghode ko dekhA
2.3	Infinitival Verb Chunk	VGINF	Bangla : bindu Borabela ((snAna karawe))_VGINF BAlobAse
2.4	Verb Chunk (Gerund)	VGNN	Hindi: mujhe rAta meM ((khAnA_VM))_VGNN acchA lagatA hai
3	Adjectival Chunk	JJP	Hindi: vaha laDaZkI hE((suMdara_JJ sI_RP))_JJP
4	Adverb Chunk	RBP	Hindi : vaha ((dhIre-dhIre_RB))_RBP cala rahA thA
5	Chunk for Negatives	NEGP	Hindi: ((binA))_NEGP ((kucha))_NP ((bole))_VG ((kAma))_NP ((nahIM calatA))_VG
6	Conjuncts	CCP	Hindi: ((rAma))_NP ((Ora))_CCP ((SyAma))_NP
7	Chunk Fragments	FRAGP	Hindi: rAma (jo merA baDZA bhAI hE) ne kahA.
8	Miscellaneous	BLK	

For complete description, see the guidelines:
<http://ltrc.iiit.ac.in/MachineTrans/publications/technicalReports/tr031/posguidelines.pdf>

Appendix C

Dependency Tag Set

The following is the list of core dependency tags that are being used.

No.	Tag Name	Tag description	Example
1.1	k1	karta (doer/agent/subject)	rAma bETA hE
1.2	pk1	prayojaka karta (Causer)	Az ne bacce ko KanA KilAyA
1.3	jk1	prayojya karta (causee)	mAz ne AyA se bacce ko KAnA KilavAyA
1.4	mk1	madhyastha karta (mediator-causer)	mAz ne AyA se bacce ko KAnA KilavAyA
1.5	k1s	vidheya karta (karta samanadhikarana)	rAma buxXimAna hE
2.1	k2	karma (object/patient)	rAma rojZa eka seba KAwA hE
2.2	k2p	Goal, Destination	rAma Gara gayA
2.3	k2g	gauna karma (secondary karma)	ve loga gAMXIjI ko bApU BI kahawe hEM
2.4	k2s	karma samanadhikarana (object complement)	rAma mohana ko buxXimAna sama-JawA hE
3	k3	karana (instrument)	rAma ne cAkU se seba kAtA
4.1	k4	sampradaana (recipient)	rAma ne mohana ko Kira xI
4.2	k4a	anubhava karta (Experiencer)	muJako rAma buxXimAna lagawA hE
5.1	k5	apaadaana (source)	rAma ne cammaca se katorI se Kira KAyI
5.2	k5prk	prakruti apadana ('source material' in verbs denoting change of state)	jUwe camade se banawe hEM
6.1	k7t	kaalaadhikarana (location in time)	rAma xilli meM rahawA hE
6.2	k7p	deshadhikarana (location in space)	mejZa para kiwAba hE
6.3	k7	vishayaadhikarana (location elsewhere)	ve rAjanIwi para carcA kara rahe We

7	k*u	saadrihya (similarity)	[[k1u]] rAXA mIrA jEsI sunxara hE
8.1	r6	shashthi (possessive)	sammAna kA BAva
8.2	r6-k1,	karta or karma of a conjunct	[r6-k1] kala manxira kA uxGAtana huA
8.3	r6v	(‘kA’ relation between a noun and a verb)	rAma ke eka betI hE
9	adv	kriyaavisheshana (‘manner adverbs’ only)	vaha jalxI jalxI liKA rahA WA
10	sent-adv	Sentential Adverbs	isake alAvA, BAkaPA (mAovAxI) ke rAmabacana yAxava ko giraPZawAra kara liyA gayA
11	rd	prati (direction)	sIWa gAzva kI ora jA rahI WI
12	rh	hetu (cause-effect)	mEne mohana kI vajaha se kiwAba KArIxI
13	rt	taadarthya (purpose)	mEne mohana ke liye kiwAba KArIxI
14.1	ras-k*	upapada__sahakaarakatwa (associative)	rAma apane pIWaji ke sAWa bAjZara gayA
14.2	ras-neg	Negation in Associatives	rAma pIWaji ke binA gayA
15	rs	relation samanadhikaran (noun elaboration)	bAwa yaha hE ki vo kal nahIM AyegA
16	rsp	relation for duratives	1990 se lekara 2000 waka BArawa kI pragawi wejZa rahI
17	rad	Address words	mAz muJe kala xilli jAnA hE
18	nmod__relc, jjmod__relc, rbmod__relc	Relative clauses, jo-vo constructions	merI bahana [jo xilli meM rahawI hE] kala A rahI hE
19	nmod	Noun modifier (including participles)	pedZa para bETI cidZiyA gAnA gA rahI WI
20	vmod	Verb modifier	vaha KAwe hue gayA
21	jjmod	Modifiers of the adjectives	halkI nIII kiwAba
22	pof	Part of relation	rAma ravi kI prawIkSA kara rahA WA.
23	ccof	Conjunct of relation	rAma seba KAwa hE Ora sIWa xUXa pIWI hE
24	fragof	Fragment of	BAkaPA (mAovAxI) ke rAmabacana yAxava ko giraPZawAra kara liyA gayA

25	enm	Enumerator	Apa apanA kara samaya se xe sakawe hEM
----	-----	------------	---

For complete description, see: <http://ltrc.iiit.ac.in/MachineTrans/research/tb/DS-guidelines/DS-guidelines-ver2-28-05-09.pdf>

Bibliography

- [Ambati et al.2009a] Bharat Ram Ambati, Phani Gadde, and Karan Jindal. 2009a. Experiments in indian language dependency parsing. In *Proceedings of the ICON09 NLP Tools Contest: Indian Language Dependency Parsing*, pages 32–37.
- [Ambati et al.2009b] Bharat Ram Ambati, Pujitha Gade, Chaitanya GSK, and Samar Husain. 2009b. Effect of minimal semantics on dependency parsing. In *Proceedings of the RANLP09 Student Research Workshop*.
- [Attardi and DellOrletta2008] Giuseppe Attardi and Felice DellOrletta. 2008. Chunking and dependency parsing. In *LREC Workshop on Partial Parsing: Between Chunking and Deep Parsing*, Marrakech, Morocco.
- [Avinesh and Gali2007] PVS Avinesh and Karthik Gali. 2007. Part-of-speech tagging and chunking using conditional random fields and transformation based learning. In *IJCAI-07 Workshop on Shallow Parsing in South Asian Languages*.
- [Begum et al.2008] Rafiya Begum, Samar Husain, Arun Dhvaj, Dipti Misra Sharma, Lakshmi Bai, and Rajeev Sangal. 2008. Dependency annotation scheme for indian languages. In *Proceedings of The Third International Joint Conference on Natural Language Processing (IJCNLP)*, Hyderabad, India.
- [Bharati and Sangal1993] Akshar Bharati and Rajeev Sangal. 1993. Parsing free word order languages in the paninian framework. In *Proceedings of ACL*.
- [Bharati et al.1995] A. Bharati, V. Chaitanya, and R. Sangal. 1995. Natural language processing: A paninian perspective. *Prentice-Hall of India*, pages 65–106.
- [Bharati et al.2002] Akshar Bharati, Rajeev Sangal, and T. Papi Reddy. 2002. A constraint based parser using integer programming. In *Proceedings of ICON*.
- [Bharati et al.2006] Akshar Bharati, Rajeev Sangal, Dipti Misra Sharma, and Lakshmi Bai. 2006. Anncorra: Annotating corpora guidelines for pos and chunk annotation for indian languages. In *Technical Report (TR-LTRC-31), LTRC, IIIT-Hyderabad*.

- [Bharati et al.2007] Akshar Bharati, Rajeev Sangal, and Dipti Misra Sharma. 2007. Ssf: Shakti standard format guide. In *Technical Report (TR-LTRC-33)*, LTRC, IIIT-Hyderabad.
- [Bharati et al.2008a] Akshar Bharati, Samar Husain, Bharat Ambati, Sambhav Jain, Dipti M Sharma, and Rajeev Sangal. 2008a. Two semantic features make all the difference in parsing accuracy. In *Proceedings of the 6th International Conference On Natural Language Processing (ICON)*, pages 11–19, Pune, India. Macmillan Publishers India Ltd.
- [Bharati et al.2008b] Akshar Bharati, Samar Husain, Dipti M Sharma, and Rajeev Sangal. 2008b. A two-stage constraint based dependency parser for free word order languages. In *Proceedings of the COLIPS International Conference on Asian Language Processing (IALP)*, Chiang Mai, Thailand.
- [Bharati et al.2009a] Akshar Bharati, Samar Husain, Meher Vijay, Kalyan Deepak, Dipti Misra Sharma, and Rajeev Sangal. 2009a. Constraint based hybrid approach to parsing indian languages. In *Proceedings of The 23rd Pacific Asia Conference on Language, Information and Computation (PACLIC 23)*, Hong Kong.
- [Bharati et al.2009b] Akshar Bharati, Dipti Misra Sharma, Samar Husain, Lakshmi Bai, Rafiya Begum, and Rajeev Sangal. 2009b. Anncorra: Treebanks for indian languages, guidelines for annotating hindi treebank (version 2.0). <http://ltrc.iiit.ac.in/MachineTrans/research/tb/DS-guidelines/DS-guidelines-ver2-28-05-09.pdf>.
- [Bhatt et al.2009] Rajesh Bhatt, Bhuvana Narasimhan, Martha Palmer, Owen Rambow, Dipti Misra Sharma, and Fei Xia. 2009. A multi-representational and multi-layered treebank for hindi/urdu. In *Proceedings of the Third Linguistic Annotation Workshop at 47th ACL and 4th IJCNLP*, pages 186–189, Suntec, Singapore.
- [Black et al.1992] Ezra Black, Fred Jelinek, John Lafferty, David M. Magerman, Robert Mercer, and Salim Roukos. 1992. Towards history-based grammars: using richer models for probabilistic parsing. In *Proceedings of the workshop on Speech and Natural Language*, pages 134–139, Harriman, New York.
- [Boyd et al.2008] Adriane Boyd, Markus Dickinson, and Detmar Meurers. 2008. On detecting errors in dependency treebanks. *Research on Language and Computation*, 6(2):113–137.
- [Brants et al.2002] Sabine Brants, Stefanie Dipper, Silvia Hansen, Wolfgang Lezius, and George Smith. 2002. The tiger treebank. In *Proceedings of the First Workshop on Treebanks and Linguistic Theories (TLT 2002)*, Sozopol, Bulgaria.
- [Buchholz and Marsi2006] Sabine Buchholz and Erwin Marsi. 2006. Conll-x shared task on multilingual dependency parsing. In *Proceedings of the Tenth Conference on Computational Natural Language Learning*, pages 149–164, New York City, New York.

- [Butt1995] M. Butt. 1995. The structure of complex predicates in urdu. In *CSLI Publications*.
- [Chang and Lin2001] Chih-Chung Chang and Chih-Jen Lin. 2001. Libsvm: A library for support vector machines. *Software available at <http://www.csie.ntu.edu.tw/~cjlin/libsvm>*.
- [Chu and Liu1965] Y.J. Chu and T.H. Liu. 1965. On the shortest arborescence of a directed graph. *Science Sinica*, 14:1396–1400.
- [Covington2001] Michael A. Covington. 2001. A fundamental algorithm for dependency parsing. In *Proceedings of the 39th Annual ACM Southeast Conference*, pages 95–102.
- [Dai et al.2009] Qifeng Dai, Enhong Chen, and Liu Shi. 2009. An iterative approach for joint dependency parsing and semantic role labeling. In *CoNLL '09: Proceedings of the Thirteenth Conference on Computational Natural Language Learning*, pages 19–24, Boulder, Colorado.
- [de Kok et al.2009] Daniel de Kok, Jianqiang Ma, and Gertjan van Noord. 2009. A generalized method for iterative error mining in parsing results. In *Proceedings of Workshop on Grammar Engineering Across Frameworks (GEAF 2009), 47th ACL and 4th IJCNLP*, pages 71–79, Suntec, Singapore.
- [Dickinson and Meurers2003a] Markus Dickinson and W. Detmar Meurers. 2003a. Detecting errors in part-of-speech annotation. In *Proceedings of the tenth conference on European chapter of the Association for Computational Linguistics*, pages 107–114, Budapest, Hungary.
- [Dickinson and Meurers2003b] Markus Dickinson and W. Detmar Meurers. 2003b. Detecting inconsistencies in treebanks. In *Proceedings of the Second Workshop on Treebanks and Linguistic Theories (TLT 2003)*, pages 45–56, Växjö, Sweden.
- [Dickinson and Meurers2005] Markus Dickinson and W. Detmar Meurers. 2005. Detecting errors in discontinuous structural annotation. In *Proceedings of the 43rd Annual Meeting on Association for Computational Linguistics*, pages 322–329, Ann Arbor, Michigan.
- [Edmonds1967] J. Edmonds. 1967. Optimum branchings. *Journal of Research of the National Bureau of Standards*, 71B:233–240.
- [Eisner1996] Jason M. Eisner. 1996. Three new probabilistic models for dependency parsing: an exploration. In *Proceedings of the 16th International Conference on Computational Linguistics (COLING-96)*, pages 340–345, Copenhagen, Denmark.
- [Eryiğit et al.2008] Gülşen Eryiğit, Joakim Nivre, and Kemal Oflazer. 2008. Dependency parsing of turkish. *Computational Linguistics*, 34(3):357–389.
- [Eskin2000] E. Eskin. 2000. Automatic corpus correction with anomaly detection. In *Proceedings of the First Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-00)*, Seattle, Washington.

- [Gadde et al.2010] Phani Gadde, Karan Jindal, Samar Husain, Dipti Misra Sharma, and Rajeev Sangal. 2010. Improving data driven dependency parsing using clausal information. In *Human Language Technologies: The 2010 Annual Conference of the North American Chapter of the Association for Computational Linguistics*, pages 657–660, Los Angeles, California.
- [Goldberg and Elhadad2009] Yoav Goldberg and Michael Elhadad. 2009. Hebrew dependency parsing: Initial results. In *Proceedings of the 11th International Conference on Parsing Technologies (IWPT'09)*, pages 129–133, Paris, France, October.
- [Gorla et al.2008] Jagadeesh Gorla, Anil Kumar Singh, Rajeev Sangal, Karthik Gali, Samar Husain, and Sriram Venkatapathy. 2008. A graph based method for building multilingual weakly supervised dependency parsers. In *GoTAL*, pages 148–159.
- [Hajičová1998] Eva Hajičová. 1998. Prague dependency treebank: From analytic to tectogrammatical annotation. In *Proceedings of the First Workshop on Text, Speech, Dialogue*, pages 45–50, Brno, Czech Republic.
- [Hall et al.2007] Johan Hall, Jens Nilsson, Joakim Nivre, Gülsen Eryigit, Beáta Megyesi, Mattias Nilsson, and Markus Saers. 2007. Single malt or blended? a study in multilingual parser optimization. In *Proceedings of the CoNLL Shared Task Session of EMNLP-CoNLL 2007*, pages 933–939, Prague, Czech Republic, June.
- [Hudson1984] Richard Hudson. 1984. Word grammar. *Basil Blackwell, 108 Cowley Rd, Oxford, OX4 1JF, England*.
- [Husain et al.2009] Samar Husain, Phani Gadde, Bharat Ambati, Dipti Misra Sharma, and Rajeev Sangal. 2009. A modular cascaded approach to complete parsing. In *Proceedings of the COLIPS International Conference on Asian Language Processing 2009 (IALP)*, Singapore.
- [Husain2009] Samar Husain. 2009. Edependency parsers for indian languages. In *Proceedings of the ICON09 NLP Tools Contest: Indian Language Dependency Parsing*, India.
- [Kaljurand2004] K. Kaljurand. 2004. Checking treebank consistency to find annotation errors. <http://math.ut.ee/kaarel/NLP/Programs/Treebank/ConsistencyChecking/>.
- [Kiparsky and Staal1969] P. Kiparsky and J. F. Staal. 1969. ‘syntactic and relations in panini’. *Foundations of Language*, 5:84–117.
- [Kordoni2003] Valia Kordoni. 2003. Strategies for annotation of large corpora of multilingual spontaneous speech data. In *Proceedings of the Workshop on Multilingual Corpora: Linguistic Requirements and Technical Perspectives*, pages 53–57, Lancaster, UK.
- [Kudo and Matsumoto2002] Taku Kudo and Yuji Matsumoto. 2002. Japanese dependency analysis using cascaded chunking. In *proceedings of the 6th conference on Natural language learning*, pages 1–7, Morristown, NJ, USA.

- [Mannem et al.2009] Prashanth Mannem, Himani Chaudhry, and Akshar Bharati. 2009. Insights into non-projectivity in hindi. In *Proceedings of the ACL-IJCNLP 2009 Student Research Workshop*, pages 10–17, Suntec, Singapore.
- [Marcus et al.1993] Mitchell P. Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. 1993. Building a large annotated corpus of english: The penn treebank. *Computational Linguistics*, 19(2):313–330.
- [Mcdonald and Nivre2007] Ryan Mcdonald and Joakim Nivre. 2007. Characterizing the errors of data-driven dependency parsing models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing and Natural Language Learning*.
- [McDonald et al.2005] Ryan McDonald, Koby Crammer, and Fernando Pereira. 2005. Online large-margin training of dependency parsers. In *Proceedings of the 43rd Annual Meeting on Association for Computational Linguistics*, pages 91–98, Ann Arbor, Michigan.
- [McDonald et al.2006] Ryan McDonald, Kevin Lerman, and Fernando Pereira. 2006. Multilingual dependency analysis with a two-stage discriminative parser. In *Proceedings of the Tenth Conference on Computational Natural Language Learning (CoNLL-X)*, pages 216–220, New York City, New York.
- [Mel’čuk1988] Igor A. Mel’čuk. 1988. Dependency syntax: Theory and practice. *State University, Press of New York*.
- [Mohanani1982] K. P. Mohanan. 1982. Grammatical relations in malayalam. In *Joan Bresnan (ed.), The Mental Representation of Grammatical Relations*.
- [Mohanani1994] Tara Mohanan. 1994. Arguments in hindi.
- [Nilsson et al.2007] Jens Nilsson, Joakim Nivre, and Johan Hall. 2007. Generalizing tree transformations for inductive dependency parsing. In *Proceedings of the 45th Annual Meeting of the Association of Computational Linguistics*, pages 968–975, Prague, Czech Republic, June.
- [Nivre and Nilsson2005] Joakim Nivre and Jens Nilsson. 2005. Pseudo-projective dependency parsing. In *ACL ’05: Proceedings of the 43rd Annual Meeting on Association for Computational Linguistics*, pages 99–106, Ann Arbor, Michigan.
- [Nivre et al.2006] Joakim Nivre, Jens Nilsson, and Johan Hall. 2006. Talbanken05: A swedish treebank with phrase structure and dependency annotation. In *Proceedings of the fifth international conference on Language Resources and Evaluation (LREC-06)*, pages 24–26, Genoa, Italy.
- [Nivre et al.2007a] Joakim Nivre, Johan Hall, Sandra Kübler, Ryan McDonald, Jens Nilsson, Sebastian Riedel, and Deniz Yuret. 2007a. The CoNLL 2007 shared task on dependency parsing. In *Proceedings of the CoNLL Shared Task Session of EMNLP-CoNLL 2007*, pages 915–932, Prague, Czech Republic, June. Association for Computational Linguistics.

- [Nivre et al.2007b] Joakim Nivre, Johan Hall, Jens Nilsson, Atanas Chanev, Gülsen Eryigit, Sandra Kübler, Svetoslav Marinov, and Erwin Marsi. 2007b. Maltparser: A language-independent system for data-driven dependency parsing. *Natural Language Engineering*, 13(2):95–135.
- [Nivre2003] Joakim Nivre. 2003. An efficient algorithm for projective dependency parsing. In *Proceedings of the 8th International Workshop on Parsing Technologies (IWPT)*, pages 149–160.
- [Nivre2008] Joakim Nivre. 2008. Algorithms for deterministic incremental dependency parsing. *Computational Linguistics*, 34(4):513–553.
- [Palmer et al.2005] Martha Palmer, Paul Kingsbury, and Daniel Gildea. 2005. The proposition bank: An annotated corpus of semantic roles. *Computational Linguistics*, 31:71–106.
- [Riedel et al.2006] Sebastian Riedel, Ruket Çakıcı, and Ivan Meza-Ruiz. 2006. Multi-lingual dependency parsing with incremental integer linear programming. In *Proceedings of the Tenth Conference on Computational Natural Language Learning (CoNLL-X)*, pages 226–230, New York City, June.
- [Seddah et al.2009] Djamé Seddah, Marie Candito, and Benoît Crabbé. 2009. Cross parser evaluation : a french treebanks study. In *Proceedings of the 11th International Conference on Parsing Technologies (IWPT'09)*, pages 150–161, Paris, France, October.
- [Shastri1973] Charudev Shastri. 1973. Vyakarana chandrodyā (vol. 1 to 5). *Delhi: Motilal Banarsidass. (In Hindi)*.
- [Shieber1985] Stuart M. Shieber. 1985. Evidence against the context-freeness of natural language. *Linguistics and Philosophy*, 8:334–343.
- [Tsarfaty and Sima'an2008] Reut Tsarfaty and Khalil Sima'an. 2008. Relational-realizational parsing. In *Proceedings of the 22nd International Conference on Computational Linguistics*, pages 889–896.
- [Vaidya et al.2009] Ashwini Vaidya, Samar Husain, Prashanth Mannem, and Dipti Misra Sharma. 2009. A karaka-based dependency annotation scheme for english. In *proceedings of CICLing*, pages 41–52.
- [van Halteren2000] Hans van Halteren. 2000. The detection of inconsistency in manually tagged text. In *Proceedings of the 2nd Workshop on Linguistically Interpreted Corpora*, Luxembourg.
- [van Noord2004] Gertjan van Noord. 2004. Error mining for wide-coverage grammar engineering. In *Proceedings of the 42nd Meeting of the Association for Computational Linguistics (ACL'04)*, pages 446–453, Barcelona, Spain, July.
- [Xia et al.2009] Fei Xia, Owen Rambow, Rajesh Bhatt, Martha Palmer, and Dipti Misra Sharma. 2009. Towards a multi-representational treebank. In *Proceedings of the 7th International Workshop on Treebanks and Linguistic Theories (TLT 2009)*, Groningen, Netherlands.

[vrelid2008] L. vrelid. 2008. Argument differentiation: Soft constraints and data-driven models. *PhD Thesis, University of Gothenburg*.